Intrusion Detection Based on Federated Learning: A Systematic Review

JOSE LUIS HERNANDEZ-RAMOS, University of Murcia, Murcia, Spain
GEORGIOS KAROPOULOS, European Commission Joint Research Centre, Ispra, Italy
EFSTRATIOS CHATZOGLOU, University of the Aegean Department of Information and Communication Systems Engineering, Samos, Greece
VASILEIOS KOULIARIDIS, European Commission Joint Research Centre, Ispra, Italy
ENRIQUE MARMOL, University of Murcia, Murcia, Spain
AURORA GONZALEZ-VIDAL, University of Murcia, Murcia, Spain
GEORGIOS KAMBOURAKIS, University of the Aegean Department of Information and Communication Systems Engineering, Samos, Greece

The evolution of cybersecurity is closely linked to the development and improvement of artificial intelligence (AI). As a key tool for realizing more cybersecure ecosystems, Intrusion Detection Systems (IDSs) have evolved tremendously in recent years by integrating machine learning (ML) techniques to detect increasingly sophisticated cybersecurity attacks hidden in big data. However, traditional approaches rely on centralized learning, in which data from end nodes are shared with data centers for analysis. Recently, the application of federated learning (FL) in this context has attracted great interest to come up with collaborative intrusion detection approaches where data does not need to be shared. Due to the recent rise of this field, this work presents a complete, contemporary taxonomy for FL-enabled IDS approaches that stems from a comprehensive survey of the literature from 2018 to 2022. Precisely, our discussion includes an analysis of the main ML models, datasets, aggregation functions, as well as implementation libraries employed by the proposed FL-enabled IDS approaches. On top of everything else, we provide a critical view of the current state of the research around this topic, and describe the main challenges and future directions based on the analysis of the literature and our own experience in this area.

This study was supported by grant RYC2023-043553-I, funded by MICIU/AEI/10.13039/501100011033 and ESF+ as well as the HORIZON-MSCA-2021-SE-01-01 project Cloudstars (g.a. 101086248), the HORIZON-MSCA-2021-PF-01-01 project INCENTIVE (g.a. 101065524), a Leonardo Grant 2023 to Researchers and Cultural Creators from the BBVA Foundation, the project CNS2023-145059 (INTEGRATOR) funded by MICIU/AEI/10.13039/501100011033 and the European Union NextGenerationEU/PRTR, the project PCI2023-145989-2 (REMINDER) funded by MICIU/AEI/10.13039/501100011033 and the European Union NextGenerationEU/PRTR, as well as the ONOFRE4 Project PID2023-148104OB-C43 funded by MICIU/AEI/10.13039/501100011033/ and FEDER/EU.
Authors' Contact Information: Jose Luis Hernandez-Ramos, University of Murcia, Murcia, Murcia, Spain; e-mail: jluis.hernandez@um.es; Georgios Karopoulos, European Commission Joint Research Centre, Ispra, Lombardia, Italy; e-mail: georgios.karopoulos@ec.europa.eu; Efstratios Chatzoglou, University of the Aegean Department of Information and Communication Systems Engineering, Samos, North Aegean, Greece; e-mail: efchatzoglou@aegean.gr; Vasileios Kouliaridis, European Commission Joint Research Centre, Ispra, Italy; e-mail: vasileios.kouliaridis@ec.europa.eu; Enrique Marmol, University of Murcia, Murcia, Murcia, Spain; e-mail: enrique.marmol@um.es; Aurora Gonzalez-Vidal, University of Murcia, Murcia, Murcia, Spain; e-mail: aurora.gonzalez2@um.es; Georgios Kambourakis, University of the Aegean Department of Information and Communication Systems Engineering, Samos, North Aegean, Greece; e-mail: gkamb@aegean.gr.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2025 Copyright held by the owner/author(s).
ACM 0360-0300/2025/07-ART309
https://doi.org/10.1145/3731596

CCS Concepts: • **Computing methodologies** → **Machine learning**; **Distributed artificial intelligence**; • **Security and privacy** → **Intrusion detection systems;**

Additional Key Words and Phrases: Federated Learning, Intrusion Detection Systems, Cybersecurity, Machine Learning, Taxonomy, Aggregation Methods, Distributed Intelligence

ACM Reference Format:
Jose Luis Hernandez-Ramos, Georgios Karopoulos, Efstratios Chatzoglou, Vasileios Kouliaridis, Enrique Marmol, Aurora Gonzalez-Vidal, and Georgios Kambourakis. 2025. Intrusion Detection Based on Federated Learning: A Systematic Review. *ACM Comput. Surv.* 57, 12, Article 309 (July 2025), 65 pages. https://doi.org/10.1145/3731596

1 Introduction

Nowadays, it is clear that **Artificial Intelligence** (**AI**) and, in particular, the application of **Machine Learning** (**ML**) techniques will play a key role in the development of the next digital age [88]. Indeed, several worldwide initiatives, including the EU AI Act [53] and the AI Strategy in the U.S., demonstrate the need for regulatory frameworks to foster the development of compliant AI systems considering ethical, social, and legal aspects. In the field of cybersecurity, the application of ML is often seen as a double-edged sword in which potential attackers can develop increasingly sophisticated attacks that need to be mitigated through effective and efficient approaches [207]. One of the most prominent advances in this field recently has been the definition of **Federated Learning** (**FL**) [138] as a decentralized learning approach where end nodes are not required to share their data to build an ML model. Instead, the training of a certain model is performed throughout different rounds in which such model is built from the end nodes' updates by using a certain aggregation function. This distributed scheme is crucial for the development of ML-enabled systems and applications, while end users' privacy is well preserved. From a practical point of view, the development of FL will be driven by the increasing processing capabilities of end nodes to execute ML algorithms, as well as the need to reduce latency in the decision-making processes by processing data at the network edge [129].

While the deployment of FL approaches has been widely considered in recent years in a plethora of scenarios, including **Internet of Things** (**IoT**) [152], edge computing [231] and healthcare [153], the application of FL in the field of cybersecurity is intended to foster the development of collaborative approaches to realize more effective and efficient systems for the identification and mitigation of cyberattacks [63]. That is, FL could play a prominent role in large-scale cybersecurity scenarios where organizations could share information about threats and security attacks without the need to share their actual data. This naturally works in favor of cyber threat intelligence as well. In particular, the well-known **Intrusion Detection Systems** (**IDSs**) are essential to identify potential security threats in any IT system. Over the last few decades, IDSs have evolved from signature-based systems to sophisticated ML-based approaches with the capability to detect slow-rate, deceptive, or even unknown attacks [98, 179]. In this context, the application of FL is intended to mitigate privacy issues, otherwise present, due to the need of sharing network traffic data, which is normally required to identify cyberattacks.

Our contribution: The development of FL-driven IDSs has recently attracted immense interest from the research community, as widely recognized by recent surveys around this topic [10, 110]. However, as detailed below, such survey works lack exhaustiveness since they either concentrate on a certain scenario (e.g., IoT) or their analysis neglects key aspects of FL, such as the aggregation function used or the dataset employed. To cover this literature gap, our analysis is founded on a proposed taxonomy around the main aspects of FL-enabled IDSs, including ML model, aggregation function, dataset, as well as implementation libraries, which are key to assess the current

development of such systems. We scrutinize each of these aspects and elaborate on their impact on FL-enabled IDS. Moreover, our analysis encompasses key challenges and future paths that need to be considered for the development of FL-enabled IDSs. This analysis hinges in part on our previous experience in this area [32] and includes the study of some of the recently proposed solutions to tackle FL challenges, including communication overhead, security and privacy issues, as well as the lack of a standardized set of evaluation metrics to ease the comparison of FL-enabled IDS approaches. While these challenges need to be considered in the coming years, it is rather straightforward that the future of IDS and other security systems can take advantage of FL's decentralized nature to realize sophisticated security approaches without the need to share private information [100]. In summary, the contributions of our work are as follows: (a) A comprehensive taxonomy that categorizes FL-enabled IDS approaches based on key aspects such as ML models, aggregation functions, datasets, and implementation libraries; (b) A critical review of more than 100 relevant works, highlighting gaps in previous surveys, particularly in terms of overlooked aspects like aggregation functions and implementation libraries; (c) Review of the main datasets employed in the evaluation of FL-enabled IDS, highlighting the key characteristics of each dataset and how they influence the development and validation of systems in this area; (d) A detailed review of different ML models and aggregation functions applied in FL-based IDS, analyzing how these elements affect the performance and effectiveness of these systems; (e) Identification of key challenges in the field, including communication overhead, security, privacy issues, and the lack of standardized evaluation metrics. It also suggests future research directions based on both the literature review and the authors' own experience in the field.

Comparison with existing surveys: Table 1 in Appendix A provides a chronologically-ordered list of major past FL surveys, including its relationship with the work at hand. Despite the great number of surveys about FL, only a few recent works have partially analyzed the current landscape of approaches related to the application of FL specifically for the development of IDS approaches. Indeed, according to our analysis, many of these focus on broad topics, such as communication overhead, privacy mechanisms, as well as their application to specific contexts, such as **multi-access edge computing** (**MEC**), or IoT, but offer only limited coverage of intrusion detection aspects. A key contribution of our work is that, rather than merely identifying the gaps in existing surveys, we leverage their findings to uncover key trends in the evolution of FL and examine their implications for FL-enabled IDS.

To make this point clearer, Table 2 in Appendix A compares the current work with existing surveys on this topic, based on seven distinct criteria, namely, the definition of a taxonomy, the analysis of ML models, the employed datasets, as well as the use of aggregation functions and specific implementation libraries. Based on our analysis, with reference to the same table, a survey addresses a certain aspect if it offers a comprehensive description of the different alternatives, e.g., it analyzes a significant number of IDS datasets and discusses their impact on the development of FL-enabled IDSs. If a certain criterion is not comprehensively analyzed, say, only the most used aggregation functions are described, we consider that aspect partially addressed. If an aspect is not examined, even if it is used to classify existing works, then it is deemed as not covered.

As recapitulated in Table 2 in Appendix A, the following surveys either address a certain criterion partially or they are marginally in scope, or both. The survey on FL-enabled IDS given in [10] analyzes relevant concepts, challenges, and future directions. However, it does not provide a comprehensive review of existing approaches, and it does not define a taxonomy to classify such solutions. Furthermore, it does not refer to existing IDS datasets and their potential suitability for FL scenarios. Recently, the authors in [55] offered a comparative review of several FL-enabled IDS approaches, and discussed the main challenges associated with them. Although this contribution considers some of the aspects put forward by the present work, e.g., dataset, implementation library,

or ML technique, it does not provide a comprehensive analysis of these features and their impact on FL-enabled IDS. Moreover, the review scope is limited, only referring to 11 works. Additionally, the work in [110] defines a taxonomy to analyze FL-enabled IDS approaches from both a quantitative and qualitative perspective. The authors also highlight some of the main open problems, as well as relevant future directions. Nevertheless, they only analyze 22 works without thoroughly covering several key aspects of FL-powered IDS. More recently, [246] analyzes 21 works on this topic by providing a partial analysis on current datasets and challenges. However, the authors do not analyze ML techniques and aggregation functions, or their impact in the context of FL-enabled IDS. Such aspects are discussed by [97], which only analyzes eight existing FL-enabled IDS approaches. The authors focus on existing challenges but they do not describe the impact of using different ML techniques in such context.

Other works concentrated on the use of FL-enabled IDS in IoT. Particularly, the authors in [29] provided a review based on the different ML techniques used by such works. Furthermore, [57] offered an overview about the use of FL to strengthen cybersecurity in IoT, considering different applications, and the integration with other technologies, such as blockchain. The authors also provided a performance analysis on the application of FL to specific datasets, and a description of the main challenges of FL-enabled IDS in IoT. In a similar direction, our previous work [32] evaluated the use of FL in IDS for IoT with different data distributions, and provided an analysis of the main challenges and future perspectives in this ecosystem. Furthermore, [22] analyzed different aspects related to IDS in IoT scenarios, including an attack taxonomy and deployment options in FL settings. Indeed, FL is not the focus of such work as the authors only analyze four works proposing an FL-enabled IDS approach. In addition, [219] is focused on FL for malware analysis in IoT, even if several works on IDS are also analyzed according to the employed dataset and ML model.

A key missing element of previous surveys is their limited discussion of recent, innovative FL approaches and how they shape the evolution of FL-enabled IDS. In particular, advanced architectures, novel aggregation methods, and layered privacy mechanisms have emerged to tackle issues like highly **non-independent and identically distributed** (**non-IID**) data and adversarial threats more effectively. While existing works typically mention fundamental privacy safeguards or classical aggregations (e.g., FedAvg, FedProx), newer research often combines multiple strategies—sometimes including decentralized or blockchain-based frameworks—to better protect and scale collaborative intrusion detection. Collectively, these developments underscore why FL-enabled IDS has evolved so rapidly—and highlight the need for an up-to-date survey that analyzes the impact of such solutions in the context of IDS. Unlike these studies, the work at hand offers a comprehensive analysis of the main aspects that characterize FL-enabled IDS approaches that are used to compare more than 100 published works on this topic so far. This analysis captures the design of advanced aggregation functions, the use of a plethora of ML models, and an exhaustive examination of IDS datasets to be considered in FL settings that are not thoroughly considered by existing surveys. Through the analysis of such recent innovative approaches and their impact on FL-enabled IDS, we shed light on the key challenges confronting FL-based IDS in practice, thereby serving as a comprehensive reference for researchers and practitioners looking to push the boundaries of federated intrusion detection.

Literature search and article selection process: for searching the literature for relevant works, we first consider the set of references exploited in [32, 181]. Then, to dig deeper and find the most recent relevant works, we took advantage of the Scopus database. A first coarse search was done using the query "TITLE-ABS-KEY (("federated learning" OR "federated machine learning") AND ("intrusion detection")). After filtering the obtained results, we realized that some references that we had considered in our previous works were not obtained using the previous query. This is because some works use terms related to intrusion detection interchangeably; for instance, several

articles use relevant to IDS terms like “malware” or “attack detection”. As a result, we amended the query with the expression: TITLE-ABS-KEY ((“federated learning” OR “federated machine learning”) AND (“intrusion detection” OR “network anomaly detection” OR “attack detection” OR “malware detection”)), and removed some references, which were not actually related to intrusion detection. This for instance applies to works proposing anomaly detection schemes and evaluate them by means of non-IDS datasets. Figure 2 in Annex A illustrates the evolution in the number of published works related to the previous search, which demonstrates the increasing interest by the research community in the development of FL-enabled IDS approaches. Moreover, in the case of the identification of relevant surveys, we realized that their majority were not yet in Scopus database because they were very recent. Therefore, in this case, we used the Google Scholar database with the search (“federated learning” “intrusion detection” “survey”).

Structure of the survey: The rest of the article is organized as follows. The next section presents a background on FL and IDS, as well as an overarching taxonomy that is used to classify the existing literature. Section 3 elaborates on existing IDS datasets and evaluation aspects (i.e., evaluation metrics and FL implementations) related to FL-enabled IDS and elaborates on their use in the literature. Furthermore, Section 4 offers an overview of the ML/DL models that have been considered for FL-enabled IDS approaches. Section 5 discusses the different aggregation techniques used in FL-enabled IDS. Section 6 discusses key challenges in this field, while the last section concludes.

2 Background

For aiding the reader in assessing the contents of the subsequent sections, the current section provides an overview of the development of FL-enabled IDS and some of the main real-world applications that can leverage the benefits of such approach. Furthermore, it describes the taxonomy that is used in our work to classify the current literature around FL-enabled IDS. For a more detailed explanation of IDSs and FL concepts, please refer to Appendix B.

2.1 Significance of FL for IDS

As mentioned above, the role of FL in the development of the next generation of IDS is decisive to the implementation of approaches to detect cybersecurity attacks without the need to share private data. As demonstrated by the impressive interest in FL in recent years, the application of FL in the evolution of IDS goes beyond small-scale scenarios where specific attacks need to be detected; it can foster a collaborative sharing approach of cybersecurity information among companies and institutions from different countries without the need to share sensitive information [185]. Indeed, the unique challenges of modern, distributed networks make FL attractive for IDS as it allows privacy-preserving and collaborative model training across multiple nodes without centralized data sharing. Compared to centralized methods, FL’s decentralized nature reduces potential vulnerabilities and ensures compliance with data protection laws, such as the GDPR. Moreover, FL enhances capabilities by leveraging collaborative intelligence. In large-scale network environments, intrusion patterns often vary between nodes, making a single centralized model insufficient. FL allows each node to contribute to a shared model privately, expanding the system’s ability to detect a wide range of cyberattacks and adapt to specific network behaviors.

Another advantage of FL is the reduction of latency, which represents a critical factor in time-sensitive scenarios. Centralized systems may suffer from data-transfer delays and processing bottlenecks, while FL facilitates near real-time detection by allowing local data analysis. This responsiveness is essential in environments where threats must be mitigated immediately, including IoT networks or edge computing contexts. FL also offers scalability and resilience by accommodating heterogeneous networks and distributing computation across multiple devices, eliminating single points of failure. This approach also enables IDS to be deployed more efficiently across large-scale,

distributed environments, enabling flexible and cost-effective cybersecurity solutions. In addition to the expected benefits, the development of FL still needs to address several challenges in the coming years, including communication requirements and security aspects, which are detailed in Section 6.

2.2 Real-World Application Scenarios for FL-Based IDS

The deployment of FL-enabled IDS approaches in real-world scenarios has attracted a significant interest specially where data privacy, scalability, and real-time threat detection are paramount. Indeed, one of the most attractive aspects of FL-based IDS is its applicability in enterprise-scale ecosystems where multiple organizations prefer not to share sensitive intrusion data directly. For instance, large banks or healthcare institutions aiming to detect emerging threats can collaborate under a federated paradigm, ensuring confidentiality is maintained while building a more robust joint model. In a similar vein, consortium-based efforts like **Information Sharing and Analysis Centers** (**ISACs**) could adopt FL to develop collective defense strategies without disclosing raw logs or proprietary information. Recent blockchain-oriented approaches for federated aggregation [259] also demonstrate that enterprise environments can leverage decentralized ledgers to reduce reliance on a single trusted aggregator, ensuring scalability and trust among potentially competing entities.

A closely related application domain involves edge and IoT environments, where massive numbers of devices—ranging from smart sensors to connected vehicles—produce continuous streams of network traffic. Traditional, centralized IDS solutions may be infeasible at such a scale due to bandwidth constraints and latency requirements. Through an on-device or near-device model training, FL-based IDS can accelerate attack detection, reduce communication overhead, and preserve the data's sensitive nature (e.g., coming from IoT devices). In fact, several works analyze non-iid data distributions in IoT settings [258] and highlight how resource constraints or heterogeneous device capabilities can affect IDS performance. These findings underscore the practical importance of lightweight model designs and advanced aggregation strategies in large-scale IoT deployments.

Furthermore, industrial and critical infrastructure scenarios could also benefit from FL-based IDS techniques, given the severe potential impact of security breaches in power grids, water treatment facilities, or transportation systems. Here, collaborative training among geographically dispersed or otherwise isolated industrial control nodes can yield faster adaptation to novel attack patterns. For instance, in vehicular or smart-grid contexts, each node can locally process its operational data and only exchange model updates, thus minimizing both communication costs and exposure of sensitive operational details. As many organizations in the industrial sector impose strict data-protection rules, federated approaches remain attractive, combining local autonomy with global threat intelligence.

2.3 A Taxonomy for FL-Enabled IDS Approaches

With regard to general classifications on IDS systems [156], an FL-based IDS is an anomaly-based system, more particularly belonging to the ML subclass. Therefore, any FL-enabled IDS taxonomy can be seen as a sub-tree under the ML/anomaly-based leaf of the general IDS taxonomy. Under this assumption, Figure 1 presents a taxonomy of FL-based IDSs, taking also into account relevant taxonomies found in related work. Also with reference to Table 2, it is to be noted that only one recent work proposes a taxonomy for FL-based IDS systems [110], whereas other relevant surveys provide taxonomies for FL systems in general [29, 83, 148, 152]. We argue here that an FL-based IDS is a special case of an IDS, which is obviously clearly impacted by the aspects around FL; thus, the proposed taxonomy shown in Figure 1 is highly influenced by the aforementioned FL taxonomies rather than trying to come up with a completely different classification. Moreover, we elaborated on the existing taxonomies around IDS to incorporate any IDS-specific characteristics. As shown in Figure 1, the proposed taxonomy builds around seven criteria detailed below.

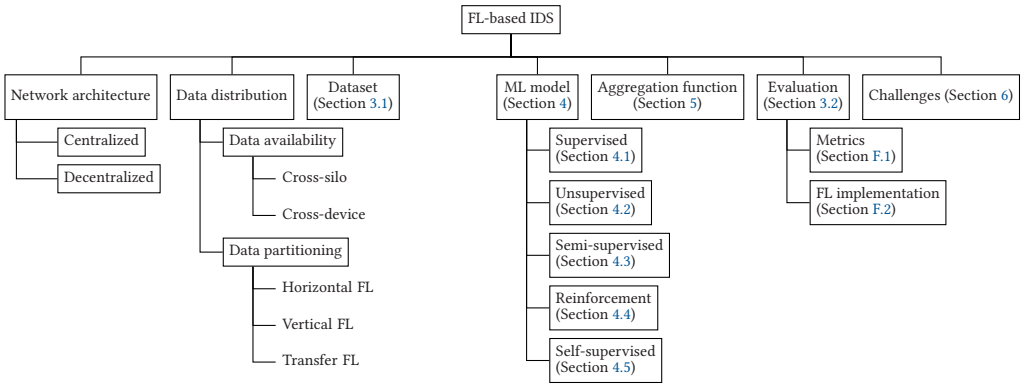


Fig. 1. Taxonomy of FL-based IDS.

The *network architecture* criterion shows where the aggregation takes place. Specifically, an IDS can be categorized as centralized, when a central server acts as the aggregator of the model parameters, or decentralized, when the model parameters are exchanged point-to-point among a subset of the system nodes. The next classification criterion concerns *data availability*, which is affected by the number and size of the client nodes. In the cross-silo category we find small client numbers that own a large amount of data and typically are data centers belonging to large corporations or organizations. In the cross-device case, there is a large number of low-end devices, such as smartphones or IoT devices, with a rather limited data volume and processing power. *Data partitioning* concerns the way data are split among participants on the sample and feature space. In horizontal FL, the clients have the same feature space but different sample space, whereas in vertical FL the sample space is the same and the feature space differs. **Federated transfer learning** (**FTL**) can be considered a hybrid category between the previous two, where both the sample and the feature spaces are different. In the context of IDS, horizontal FL allows for the collaborative training of a shared model where each participant contributes to the model updates without sharing raw data. The aggregation function typically used in horizontal FL is **Federated Averaging** (**FedAvg**), where local updates are averaged to improve the global model, which is then shared among participants. In horizontal FL, the source model is adapted to the target domain through methods such as fine-tuning or meta-learning. The aggregation function in horizontal FL may involve knowledge transfer mechanisms that are not limited to averaging model weights but can also include approaches like weighted ensemble methods that are tailored in accordance with the source and target domains. Vertical FL requires a meticulous design of the network architecture to account for the alignment and integration of different feature sets while preserving data privacy. Techniques such as secure multi-party computation and homomorphic encryption are often used to enable the secure aggregation of model updates without revealing the actual data to other participants or the central server. The *ML model* aspect considers the type of ML algorithm used, dividing them into four categories: supervised, unsupervised, semi-supervised, and **reinforcement learning** (**RL**). An IDS can further be classified depending on the employed ML algorithm, using existing taxonomies [60]. The *aggregation method* aspect classifies FL-based IDS systems according to the algorithm used for aggregating the received model updates. The next criterion, namely, *implementation framework*, concerns the framework used for implementing the FL solution. The *dataset* aspect is not actually a characteristic of the IDS, but it demonstrates which of the available datasets has been used for testing a proposed FL-based IDS and conducting the relevant experiments. For each taxonomy's criterion, we provide an analysis in the following sections, considering the

existing literature on FL-enabled IDS approaches. Based on the proposed taxonomy in Figure 1, Appendix H classifies the FL-enabled IDS approaches analyzed in this work. The next sections dig into the main aspects of the proposed taxonomy, including datasets, ML models, aggregation functions, as well as evaluation considerations.

3 Datasets and Evaluation of FL-Enabled IDS

The effectiveness of FL-driven IDS crucially depends on choosing suitable datasets and applying consistent evaluation methods. This section first reviews the datasets commonly used to benchmark IDS solutions in a federated context, then outlines the primary metrics and implementations for FL-based IDS. We note that comparing different proposals remains challenging, given the broad variability in datasets, metrics, and frameworks.

3.1 Datasets for FL-Enabled IDS

Appendix C describes and analyzes 36 pertinent network datasets that were proposed in the literature during the time span between 2017 and 2022. We classify these datasets in three categories, namely, Internet datasets (Appendix C.1), IoT datasets (Appendix C.2), and other datasets (Appendix C.3). Although none were explicitly created for FL-based IDS, several have been used successfully in that context (Figure 4 in Appendix C). To compare these datasets, we consider aspects typically employed in IDS benchmarking [179], such as the number of features and samples, attacks, as well as if the corpus was obtained through a realistic testbed. We also note whether each dataset can realistically simulate an FL environment, since many FL-enabled IDS works rely on artificial dataset splits (e.g., lack IP addresses) [32, 176]. The basic characteristics of each one of the considered datasets are sketched below, and additional details per corpus are provided in Appendix C.

Analysis: From Appendix C, several conclusions can be drawn. First, we observe that half of the 36 datasets examined (18) target IoT scenarios, reflecting the rising deployment of IoT technologies. Second, there is also an increase in proposed datasets related to vehicular environments, especially around CAN-based in-vehicle IDS. Third, regarding the benign/attack ratio, even though it is widely recognized that real-life scenarios are characterized by a higher proportion of benign traffic, we verified that there is a significant disparity between the different datasets, with some of them (e.g., Bot-IoT) containing more samples related to attack traffic. A fourth point is that most of the datasets do not contain modern attacks, while a significant portion of them embrace a small number of attacks. Another significant observation is that most newer datasets support more realistic splits (e.g., IP-based) for FL, though some (e.g., Kitsune, IoT-23) still require pre-processing to isolate each IP device. In the case of datasets related to in-vehicle networks, we verified that data often comes from a single vehicle, complicating realistic multi-client divisions. Others, like IoTID20, contain only two devices; IoT-23 has three benign devices but one attacker.

In addition to the analysis of the described datasets, Tables 3, 4, and 5 in Appendix C show which datasets are used by proposed FL-enabled IDS. We verified that only 56 works out of the 104 analyzed utilized recent datasets. Namely, Figure 4 in Appendix C provides the usage statistics of the datasets employed by existing FL-enabled IDS approaches. We see that the most used dataset is NSL-KDD [212] (used in 23 works so far), which was created in 2009 as an improved version of the KDDCup99 [4] dataset, which, although deprecated, is still widely used (11). However, these datasets lack IP info for realistic FL partitions [179]. Indeed, the proposed works (e.g., [12] and [173]) use different and artificial divisions of the dataset. Refs. [125, 182, 188] consider a variable number of nodes, and in some works (e.g., [130, 237]) the number of nodes is not clearly stated. With reference to KDDCup99, [62] and [72] use a fixed number of clients (2 and 4 respectively). A variable number of nodes is considered by [123], while this number remains unspecified in [202].

Beyond KDDCup99 and NSL-KDD, CIC-IDS2017 is also widely used (20). Some authors (e.g., [168]) leverage IP addresses to define up to 12 nodes. However, in other cases, including [161, 255]), it is unclear how the dataset is divided. Additionally, certain works [171] only consider a subset of the dataset's attacks. CIC-IDS2017, KDDCup99, and the WSN-DS dataset [18] are used in [41] by using different numbers of clients. Moreover, [170] is evaluated using two datasets independently: CIC-IDS2017 and ISCXBotnet2014 [28], which are divided considering a variable number of clients. Furthermore, the work in [54] uses **Transfer Learning** (**TL**) to obtain personalized models.

Another widely used dataset is UNSW-NB15 [149] (11), which was proposed in 2015 with 49 features to characterize network traffic considering 45 IP addresses, which can be used for FL purposes. For instance, the work in [254] uses two clients to simulate a federated TL scenario. Other more recent datasets, such as CIC-DDoS2019 and CSE-CIC-IDS2018 were used in 7 and 5 works, respectively. For example, [113] divides the CIC-DDoS2019 among five nodes, and [242] considers a variable number of clients. Furthermore, CSE-CIC-IDS2018, InSDN [52], and MQTTset [217] were used by [59] considering a variable number of nodes (5 to 15).

Regarding IoT datasets, in the case of Ton_IoT (7), [32] divides the dataset considering the nodes' IP address and select a subset of the dataset made up of 10 clients. The work in [46] also divides the dataset by IP address considering the network traffic of 13 nodes. However, the work in [105] only uses two clients to split the dataset, and [199] proposes a hierarchical approach making use of the NSL-KDD and Ton_IoT datasets, although the number of the participating devices is not mentioned. For N-BaIoT (5), [252] considers different data distributions using 27 and 89 nodes to partition the dataset. The work in [176], on the other hand, splits the dataset across eight nodes using the division of the network traffic as provided in the dataset.

Moreover, N-BaIoT, Kitsune, IoT-DDoS [5] and WUSTL [213]) are used by [243] independently considering three FL clients. Interestingly, the WUSTL dataset is a previous version of WUST-IIoT without IP address information, while the IoT-DDoS dataset is a subset of BoT-IoT made up of traffic only related to **distributed DoS** (**DDoS**) attacks. Moreover, the work in [78] uses BoT-IoT considering the attackers' IP address. Additionally, [56] proposed the Edge-IIoTset dataset, which is described in Section C.2 in Appendix C, and used for the evaluation of different FL-enabled scenarios.

For vehicular contexts (Section C.3 in Appendix C), [15] employs the OTIDS dataset, which is distributed among a variable number of nodes. However, this dataset comprises information generated by a single vehicle. This is also the case of [50] and the Car Hacking A&D Challenge dataset. Furthermore, [7] uses the Car Hacking dataset distributed among 20 nodes, while [216] centers on the detection of vehicle misbehavior using the VeReMi dataset, although it is artificially distributed among 10 nodes.

Based on industrial environments, [256] uses two datasets ([82] and [146]) considering a variable number of nodes. The work in [146] is also used by [112] and [20]. In the first, the authors consider a variable number of clients, while in the second the authors distribute the dataset among 100 nodes with 10% of them training in each round. Furthermore, [147] exploits another dataset on DoS attacks [58] associated with the ModBus protocol. Reference [158] focuses on a water treatment scenario using the **SecureWater Treatment** (**SWaT**) dataset [64]. The peculiarity of this work is that, unlike most of the proposed approaches, it considers a vertical FL scenario where clients do not share the same feature space.

Other datasets are scarcely used in the current literature. For instance, [81] uses the CTU-13 dataset [61], which concentrates on different botnets and is distributed among 200 clients. Furthermore, [255] uses the ISCXVPN2016 [49] and ISCXTor2016 [109] datasets. While the former dataset is split into two clients, it is unclear if this division can be done for the latter. A similar case is presented in [198], which uses the Microsoft Malware dataset [180] that is randomly divided

into six nodes of equal size. Other barely considered datasets are Drebin [24], Genome [257], and Contagio [1], which are used by [208] by modifying the number of clients. In the case of [163], the authors use the Maldroid [133], Drebin and Androzoo [16] datasets considering three clients. Furthermore, [164] uses the same datasets and the VirusShare repository [221] with a variable number of clients.

Other works generate their own datasets [114, 154, 205] in different scenarios, including detection of attacks on Android devices [76, 174], or even against solar farms [251]. Moreover, the work in [248] considers false data injection attacks on smart grids, while [196] uses the VirusTotal database [165], which also seems to be used by the same authors in [195]. On the downside, some of the aforementioned datasets are not publicly available, therefore it is difficult to compare the proposed approaches with the existing literature. Last but not least, there exist few other works, including [155] and [13], which do not provide details of the dataset being used.

Overall, based on the previous analysis, we verified that a significant number of proposed works are based on datasets prior to 2017 that contain obsolete attacks, which do not reflect modern communication networks. Furthermore, some datasets also lack essential network traffic information (e.g., IP addresses), making realistic FL client distribution difficult. Even when datasets allow for realistic divisions, most studies use random splits without making them publicly available, hindering reproducibility and fair comparisons. Additionally, many works create artificially balanced splits, contradicting the non-IID nature of FL scenarios. Furthermore, the approaches do not reflect heterogeneity, that is, considering scenarios where nodes employ dissimilar features to characterize network traffic. Actually, only one of the works analyzed [158] explicitly considers a vertical FL scenario. Last but not least, the datasets we presented above have been created in different environments, including generic Internet, IoT, **software-defined networking** (**SDN**), vehicular and **Unmanned Aerial Vehicle** (**UAV**) testbeds. FL-IDS research covers many different fields and is, thus, expected that using a relevant dataset will have an impact on the accuracy of the experimental results and the development of reliable IDS.

When it comes to dataset selection, it is preferable to use datasets that possess all or most of the aforementioned characteristics in FL IDS research, that is, contain recent data, provide network information such as IP addresses, divide data to different hosts, make this division publicly available, and reflect heterogeneity. Moreover, it is also important for these datasets to be labeled, in order to facilitate experimental evaluation, and contain a wide range of features to be as generic as possible. With reference to Table 3 in Appendix C, CIC-IDS2017, CSE-CIC-IDS2018, and CIC-DDoS2019 fulfill the above criteria and actually are the three most utilized generic datasets in FL IDS related works. Apart from these, the two CIDDS datasets have a potential but with the drawback of having less features (14); another option here is AWID3, which has the largest number of features (254) from all the datasets we presented above. With reference to Table 4 in Appendix C, for FL-based IDS in an IoT environment it would be beneficial to consider using an IoT dataset, such as ToN_IoT or X-IIoTID. Other recommended datasets in this category include N-BaIoT, Kitsune, and Bot-IoT, which, however, present a high ratio of attack to benign traffic. For vehicle IDS scenarios (Table 5 in Appendix C), VeReMi seems the one fulfilling most criteria even though the number of features used is only 17. For the rest, OTIDS has a high ratio of attack to benign traffic, whereas Car Hacking and Car Hacking A&D Challenge use a low number of features (5 and 6, respectively).

3.2 Evaluation of FL-Enabled IDS Approaches

This section analyzes the main aspects affecting the evaluation of FL-based systems and, in particular, their impact in the scope of FL-enabled IDS. For this purpose, Section F.1 in Appendix F describes the most commonly used evaluation metrics in ML and FL. In addition to the described metrics, the evaluation of FL systems, and therefore FL-enabled IDS, is characterized by some additional aspects

that need to be considered. Specifically, the number of training rounds has a significant impact on model convergence and the communication overhead between the aggregator and the clients. Furthermore, the number of epochs determines the iterations a client performs on its local dataset in each training round before sending weight updates to the aggregator. With reference to Section H, we note that there is no consensus about the number of training rounds/epochs to be considered, and in some cases, these values are not specified. Additionally, although not explicitly mentioned, there are specific metrics for RL settings, such as rewards. A reward is often represented as a signal (negative or positive) received by the agent after taking an action in the environment to promote its learning ability.

Furthermore, Section F.2 in Appendix F provides an overview of the main existing FL implementations. Indeed, Figure 7 provides a summarized overview on the use of such libraries in the scope FL-enabled IDS. The main goal is to understand how IDS models are evaluated and compared in the FL ecosystem (and for FL-enabled IDS in particular) taking into account the unique characteristics of FL and the associated challenges.

Analysis: as shown in the studies analyzed in Table 9, most of the metrics used to evaluate FL-enabled IDS correspond to the classification metrics described above. Specifically, Accuracy represents the most common metric, being used by 92 out of 104 studies (88.45%). However, this metric alone does not provide a reliable measure, especially in imbalanced datasets, which are common in FL scenarios and in real-life datasets.

Precision and recall are also widely used, in 56.2% and 53.3% of surveyed works, respectively. In practical terms, a high recall indicates that the model is effective at correctly identifying flows representing attacks. Moreover, high precision would indicate that when the model predicts an attack, it is likely to be correct.

The F1-score is also relatively common (70.1%) and it provides a single metric to evaluate the overall effectiveness of a classification model, considering both the correctness of positive predictions and the ability to capture positive instances. The **Area Under the Curve** (**AUC**) is a less frequent metric (3.8%) but often preferred over others, such as accuracy, in situations where there is an imbalanced class distribution or a need to evaluate the model's performance across different threshold settings. In fact, as described by [29], the use of ROC can provide a more accurate global view in FL-enabled IDS systems. It is also worth noting that only one study [167] uses the **Matthew Correlation Coefficient** (**MCC**) metric, which relates the set of **True Positive** (**TP**), **True Negative** (**TN**), **False Positive** (**FP**), and **False Negative** (**FN**) cases in a formula. This metric, along with the **Cohen Kappa Score** (**CKS**) metric, offers high reliability and simplicity of interpretation [29]. Other metrics employed in the different analyzed works are related to the efficiency of the proposed approach, including time or computation time (21%), latency (2%), overhead/communication overhead (11%), delay (2%), memory/CPU use (6%), power (3%), energy (3%), and area on-chip (1%).

The wide variety of metrics used, along with the issues mentioned in Section 3.1 regarding the division of datasets among different parties, makes comparing the different proposed approaches a complex and non-intuitive task. Additionally, as shown in Table 9, it is noteworthy that some studies do not explicitly indicate the number of rounds and epochs considered in the training process. This is coupled with a lack of details about the hyperparameters used in the employed model. In fact, the process of tuning the hyperparameters of the models used is essential for improving the metrics. As described in [47], which is devoted to analyzing the factors that may lead to a method being perceived as superior, *"quantifiable progress necessitates the use of shared metrics, which are an essential part of a benchmark. Their choice requires great care, as at the end of the day, metrics reflect the progress and dictate future research directions."* Based on the above discussion, we have shown that there is no agreement in FL research when it comes to reporting metrics, and in this sense, we

believe that the full context of the problem needs to be provided in order to promote transparency and avoid hindering the progress of this area of knowledge. In fact, as described in [29], future work should address this issue by defining a standardized set of metrics to facilitate the comparison and analysis of FL-enabled IDS.

Regarding the implementations used in the analyzed works, Figure 7 depicts the usage of different FL implementations previously described. Interestingly, only 22 out of 104 works employ libraries specifically designed for FL. Actually, some works do not provide details of the libraries used, while a significant portion references well-known ML libraries such as scikit-learn [103]. Furthermore, it is worth noting that the majority of the analyzed frameworks offer preliminary versions that may contain bugs or implementation weaknesses. These aspects also do not facilitate the analysis and comparison among the analyzed FL-enabled IDS.

4 ML/DL Models for FL-Enabled IDS

This section alongside Appendix D offers a comprehensive overview and analysis of the ML techniques used by existing FL-enabled IDS. In the following subsections, we briefly refer to five major categories, namely supervised, unsupervised, semi-supervised, reinforcement, and self-supervised learning techniques. The reader can refer to Table 6 in Appendix D for a schematic overview of the use of each ML technique in existing literature. Furthermore, a summarized overview of such information is provided in Figure 5.

4.1 Supervised Learning

Supervised learning uses labeled data for training. Common techniques [134] include Logistic Regression, **Decision Trees** (**DT**), **Random Forest** (**RF**), **Neural Networks** (**NN**), and **Support Vector Machines** (**SVM**). Based on the information collected in Annex D, most FL-enabled IDS approaches use NN, including feedforward (FNN), recurrent (RNN), and convolutional (CNN) architectures.

FNNs propagate signals in one direction. The work in [12] employs a simple MLP (an example of FNN) with two hidden layers (256 neurons) for federated mimic learning, where a master model is trained on private data and label a public dataset to train a student model. Meanwhile, [123] applies an MLP in a blockchain-enabled FL architecture for vehicular attack detection. In [176], four MLPs with varying configurations are tested.

As an alternative to FNNs, RNNs allow bidirectional signal flow. A well-known RNN is **Long short-term memory** (**LSTM**), whose hierarchical approach is used in [199] for FL-based medical environments and a 5-layer LSTM is used in [251] for false data injection attack detection in solar farms. **Gated recurrent units** (**GRUs**), another RNN type, are employed in [113] with two fully connected layers and two GRU layers, while [105] utilizes a GRU model with hidden layers of 16 and 8 neurons. In [210], a GRU with 256 hidden nodes is tested. The work in [147] combines LSTM with four GRU models, while [154] presents the DIoT system, where gateways generate GRU models for attack detection. In particular, several gateways are responsible for generating GRU models from the connected IoT devices, which are then aggregated by a service.

CNNs are widely used in FL-enabled IDS. For instance, [114] applies CNNs for intrusion detection in satellite-terrestrial networks, [205] employs a four-layer CNN for LAN-based attack detection, and [256] uses CNNs for payload classification as a basis for detecting potential attacks in an edge-enabled industrial environment. Similarly, [54] deploys CNNs on **Mobile Edge Computing** (**MEC**) nodes with TL for IoT networks, and [163] integrates TL and FL for collaborative learning. Moreover, in [51], a 16-layer CNN is utilized in combination with a simple 7-layer structure [193]. Studies like [132] use CNNs for DDoS attacks. For improving effectiveness, the authors group several traffic flows related to a certain attack to be used by the CNN. Moreover, a germinal CNN-based work is

presented in [130]; the same study was improved in [237] to consider every attack in the employed dataset. The study in [43] also uses a three-layer CNN to build a model which is then transferred to another domain for accelerating model training. The authors also rely on RL to build a client selection mechanism to improve system performance. Additionally, [81] exploits a CNN-based approach that includes a noise layer to implement a **Differential Privacy** (**DP**) mechanism for reinforcing privacy.

Other recent approaches consider the use of CNNs in combination with other models. For example, in [125], the authors combine CNNs and MLPs; CNNs are used to extract data features that are then used by an MLP model for classification. The study in [248] uses a combination of CNNs and LSTMs for false data injection attack detection in the context of smart grids, although further details about the model are not provided. Additionally, [253] compares the performance between a federated version of CNNs and LSTMs, resulting that the latter offers better performance over a dataset consisting of UNIX user logs. The use of a simple RNN is considered by [247] as a more effective alternative compared to MLPs and CNNs to detect DoS attacks using multiple datasets. In [112], the authors propose a model consisting of a CNN and a GRU, whose outputs are then passed to an MLP for intrusion detection in industrial environments. In particular, the input features are a one-dimensional vector representing the numerical features of network traffic data. This vector is processed by three convolutional blocks and two identical GRU layers. The output is fed to the MLP module, which has two fully connected layers and a dropout layer. Ultimately, a softmax layer provides the final classification result.

In addition, some approaches do not provide details on the type of NN being used. For instance, [150] proposes a security architecture for detecting jamming attacks in a UAV ecosystem. Moreover, [173] uses a simple NN with a hidden layer to analyze the impact of different data distributions. Also based on a (fully connected) NN, the work in [243] builds an efficient memory-based model based on the backpropagation algorithm.

The authors in [171] exploit a three-layer NN to design an attack detection approach based on the principles of never ending learning [144]; this is done to obtain knowledge about new threats. An NN-based model is also used by [216], although no further details about the used model are provided. Another NN is used by [182] based on a simple architecture with two hidden layers in an FL-enabled hierarchical IDS approach where edge nodes perform partial aggregations. Additionally, [78] proposes an NN model by reducing the number of layers and evaluating the impact of different numbers of neurons used in the hidden layer.

The authors in [225] rely on a lightweight NN with two hidden layers to propose a federated approach based on *contrastive learning* in which original traffic instances are converted into a new representation. The contribution in [108] considers a simple NN structure comprising two hidden layers with 64 neurons. The use of an NN is also considered by [160], which compares the performance and privacy properties between FL, TL, and Split Learning [197]. Also based on NNs, other works make use of **Deep NNs** (**DNNs**), which are based on the use of a greater number of hidden layers. For instance, the use of DNN combined with TL is proposed by [254] to tackle the problem of training data scarcity. In particular, first, FL is used to build a global model, which is reconstructed and re-trained in a second phase to improve detection effectiveness. In a similar approach, [161] uses a 6-layer DNN to pre-train a model on the network edge, which is then transferred and customized on different end nodes. Also through a DNN, [167] implements a model composed of 4 hidden layers with 100 neurons each. In [93], the authors rely on a 6-layer architecture to detect attacks in industrial environments. Moreover, [174] uses a DNN composed of five dense layers and five dropout layers for the detection of side channel attacks on Android devices. In [59], the authors compare federated approaches based on CNN, RNN, and DNN models with different configurations.

Other recent works consider certain variations of traditional NN approaches, such as **Binarized NNs (BNNs)** [77]. In particular, targeting IoT scenarios where gateways act as FL clients, [170] proposed a BNN as an efficient approach for federated training in terms of memory and communication costs. Furthermore, the authors in [255] propose a **multi-task learning (MTL)** [249] approach called MT-DNN-FL, which simultaneously detects network anomalies, recognizes VPN traffic, and classifies traffic using publicly accessible datasets. Recall that a **Multi-task NN (MTNN)** is an example of MTL techniques, which are typically used to train a model for different purposes simultaneously; therefore, this approach is especially useful in environments where there is a scarcity of data for each specific task.

In addition to NNs, other techniques have also been considered in the development of FL-enabled IDS approaches. For example, DTs are one of the simplest ML techniques. Based on DTs, the work in [158] uses the **Gradient Boosting Decision Tree (GBDT)** algorithm [245], which combines multiple DTs for creating a more accurate prediction model. Additionally, [242] uses a more lightweight approach, which significantly accelerates the training process compared to GBDT. However, according to the results presented, the proposed approach is not deployed in a federated scenario. The same classifier is used in [48] by adjusting its main hyperparameters to mitigate overfitting. In this direction, [15] proposes an RF model for intrusion detection in a vehicular scenario to extract and transform CAN ID sequences. In [136], the authors propose an RF approach on which the optimal number of DTs to be used is analyzed considering different datasets. Furthermore, [50] uses RF to ensemble local models that are trained using GRUs.

Other popular ML techniques are represented by LR and SVM [134]. Based on LR, [32] proposes an approach for creating a multi-class classification system using the softmax function on the Ton_IoT dataset (see Section 3.1). A similar approach is used by [181], which adds DP techniques using the same configuration. Also based on a similar environment, the authors in [46] employ LR for binary classification on a different subset of the same dataset. Regarding [188], the authors compare a simple LR model with a DNN-based approach , obtaining different results depending on the data distribution between participating nodes.

In the case of SVM, [76] proposes the use of a federated version of SVM to detect malware on Android devices. Additionally, [41] uses GRU in combination with SVM considering different datasets. In a smart metering environment, [143] employs a simple DNN architecture that is compared with SVM, KNN, and LR. A comparison of different techniques is also given in [72]. Specifically, the authors use MLP, DT, SVM, and RF in a federated environment where client updates are shared through a blockchain. In [56], the authors propose the Edge-IIoTset dataset (see Section 3.1), which is evaluated using different models, including DT, RF, SVM, DNN, and **K-Nearest Neighbors (KNN)**. Moreover, the work in [196] compares the use of different supervised approaches, including CNN, RF, LR, and KNN. The same work is extended in [195] considering additional models, such as Mobile-Net (a type of CNN for computer vision) and MLP, as well as an approach to reduce the communication cost.

Finally, one of the most recent ML techniques is represented by transformers [121], which are the basis of popular tools such as Chat-GPT. Although they are mostly known for their results in **Natural Language Processing (NLP)**, such models can also be considered for intrusion detection. In this direction, [7] uses transformers in a supervised approach to detect attacks in a vehicular environment through a training process using labeled data.

4.2 Unsupervised Learning

Our analysis reveals that unsupervised learning techniques [34] remain underexplored in FL-enabled IDS. It consists of finding patterns from unlabeled data, while some well-known techniques are *clustering* and *dimensionality reduction*. Clustering groups the data into clusters based on the

similarities among samples. One of the main clustering techniques is *k-means*, which is used by [233] using the cosine distance (instead of the commonest Euclidean distance) to measure the distance between different data. Additionally, the authors use a three-way clustering approach in which clusters are represented by three regions. The same technique is applied in [106] over a set of benign data from different datasets to create several classes of normal traffic. Furthermore, [234] uses the same approach to detect nodes sending false gradients during the training process.

Beyond clustering, one of the most widely used unsupervised techniques for IDS is **autoencoders** (**AEs**). With reference to IDS, an AE is trained with benign data to learn the underlying representation of normal traffic. Then, during the intrusion detection phase, a certain threshold is established so that in case the reconstruction error exceeds the threshold, the sample is considered as anomalous. Based on AEs, the goal of [168] was to evaluate different AE configurations by modifying the number of hidden layers and neurons on the CIC-IDS2017 dataset. The authors also use blockchain to support the auditing of models throughout the training process. AEs are also considered by [62], which describes a system based on the **Gaussian Mixture Model** (**GMM**). Related to this work, the study in [40] proposes an AE- and GMM-based approach considering data from different domains. Additionally, the authors propose an IDS optimization strategy to cope with the impact of different client data distributions. The contribution in [235] relies on a combination of AEs and NNs. First, an AE is used to learn from unlabeled data and produce labeled data that is then used as input for an NN. Furthermore, [176] assesses three different configurations of AEs, considering different numbers of hidden layers and neurons per layer on the N-BaIoT dataset. In this case, the configuration with the best results is based on a single hidden layer. Moreover, while the authors in [13] use AEs, the configuration of the testbed is not provided. Furthermore, a few works employ variations of AEs. The concept of *stacked AE* is represented as several layers of AEs, where each layer's output is fed as input to the next layer. This AE scheme is considered in [35], where the proposed architecture exploits the AWID dataset; therefore, the input layer has a number of neurons equal to the number of attributes in the dataset (74), and four neurons in the output layer correspond to the equal number of classes in AWID. Furthermore, [85] proposes the use of stacked AE to detect attacks in industrial environments, although the authors do not provide details about the evaluation carried out. On the other hand, [229] proposes the use of **variational AEs** (**VAE**) for anomaly detection in different datasets, including NSL-KDD. A VAE is an extension of the traditional AE that includes a probabilistic model to allow the generation of new data.

Furthermore, a recent trend is based on the use of **Generative Adversarial Networks** (**GANs**) [69]. In this direction, [208] uses a GAN to generate adversarial samples by considering a CNN architecture for generator and discriminator functions. The authors propose two different algorithms to mitigate the impact of such adversarial samples during the training process. The proposed approach in [95] relies on DBN. In particular, the authors use **Gaussian Binary Restricted Boltzmann Machine** (**GBRM**) [141] to construct a **Deep Belief Network** (**DBN**), which is deployed on different gateways acting as FL clients. Also based on DBN, [232] introduces an attention mechanism for attack detection in smart grid scenarios. Moreover, [139] contributes a rather preliminary work on the possibility of implementing **Self-Organizing Maps** (**SOMs**) in a federated architecture. Another primal work is given in [227], which briefly compares isolation forest [71] and elliptic envelope [25] techniques on a subset of the CIC-IDS2017 dataset.

4.3 Semi-Supervised Learning

Semi-supervised learning is based on the use of both labeled and unlabeled data to build a certain model. According to our analysis, the use of semi-supervised techniques is scarcely considered in the development of FL-enabled IDS. The work in [21] combines a 3-hidden-layer AE model and a simple NN. In particular, FL nodes (represented by IoT gateways) use an AE on unlabeled

data, and the aggregator combines these models by adding layers to the model using labeled data. The same authors proposed a similar approach in [20], which is evaluated considering an industrial environment. Based on the integration of AEs and NNs, [172] employs an approach called ONLAD, which is based on **Online Sequential Extreme Learning Machine** (**OS-ELM**) [119]. They evaluated their approach through the NSL-KDD dataset.

Moreover, [252] propose a semi-supervised approach based on a CNN model. It is assumed that each node initially is trained with labeled data, so that the traffic information obtained from other unlabeled data is classified using a discriminator considering the previous training. An additional approach is proposed by [198], which makes use of a lightweight CNN architecture and a GAN; the latter is used to generate unobserved data for training purposes. Also based on semi-supervised learning, [164] capitalizes on a TL-based approach, where a cloud model is trained with labeled data and this information is used by the end nodes for attack classification on unlabeled data. Particularly, the authors use a graph-based representation called **subgraph aggregated capsule network** (**SACN**) to produce a more precise representation of complex attacks.

4.4 Reinforcement Learning

RL [89] represents the branch of ML in which an agent learns to maximize a certain numerical reward by interacting with the environment. The approach is based on trial and error so that the agent learns which actions lead to greater reward over time. RL algorithms are mainly classified into: *value-based* techniques, in which a certain value function indicates the quality of an action for a specific state, *policy-based* techniques, in which a policy determines the best action in each state, and *model-based* techniques, where the agent uses a model of the specific environment for the learning process.

Although RL has been widely used in different scenarios, such as the development of robots and control systems, its application in the development of IDS is not so widely considered [128]. In the case of FL, our analysis reveals that only a few works are based on RL techniques. In particular, [159] uses a value-based approach using Q-learning [44]. The authors also compare their approach with SVM on the CIC-IDS2017 dataset. Furthermore, [226] uses a deep RL technique called **Deep Q-Network** (**DQN**) [145]. The authors deploy this approach in a 5G network architecture where edge nodes cooperate with a cloud architecture to achieve a federated attack detection scheme. The work in [155] presents a monitoring framework called DeepMonitor, which includes intrusion detection capabilities for DDoS attacks based on **Double Deep Q-Network** (**DDQN**) [218].

4.5 Self-Supervised Learning

Self-supervised learning [68] has recently gained significant traction in various domains such as computer vision, NLP, and time-series analysis, due to its ability to learn robust feature representations without relying on large labeled datasets. In self-supervised learning, models automatically generate pseudo-labels or auxiliary tasks (e.g., predicting masked parts of the input) based on inherent structures within the data, eliminating the need for explicit manual annotation. In the context of IDS, self-supervised approaches can be particularly beneficial because high-quality labeled data are often scarce or expensive to obtain, and network traffic behaviors may frequently evolve. Therefore, models are able to learn intrinsic characteristics of benign and malicious patterns from unlabeled or partially labeled data. This way, self-supervised learning could help reduce annotation overhead and better adapt to rapidly changing threat landscapes. Moreover, the integration of self-supervised learning with FL could further preserve privacy by keeping raw data local while still sharing intermediate representations or model parameters that capture global insights. Despite such benefits, we found that few FL-enabled IDS works in the current literature explicitly employ self-supervised learning methods. Nonetheless, we anticipate a growing interest in leveraging

self-supervised frameworks—such as contrastive learning or masked autoencoding—for intrusion detection tasks in the near future, as these methods can produce richer representations from large-scale unlabeled network traffic data. The resulting models have the potential to improve detection accuracy and resilience to unseen attack patterns, especially when combined with the decentralized and privacy-preserving nature of FL.

4.6 Analysis

Based on the previous description, Table 6 provides an analysis on the use of ML techniques in existing FL-enabled IDS approaches. Furthermore, a more summarized perspective can be shown in Figure 5. According to our analysis, 78 out of the 104 analyzed works (75%) employ a supervised learning approach. While most of these works provide high performance in detecting different cyberattacks, they are characterized by the need for labeled data, which represents a significant limitation in real-life scenarios. Indeed, considering the dynamism, scale, and heterogeneity of current deployments, this could be impractical in many settings (e.g., IoT) where a large number of devices would need to be equipped with this information to detect potential threats. Furthermore, supervised learning techniques cannot be used to detect previously unknown attacks. This represents yet another significant limitation that is, however, inherent in most FL-enabled IDS approaches.

Regarding the considered techniques, CNN represents the most widely used approach (33 out of 78) either uniquely or in combination with other techniques, such as MLP [125] or GRU [112]. An additional important aspect to highlight is that a significant portion of the analyzed works describes the use of NNs (16) but not the specific type, even though some of them specify the number of layers and activation functions employed. Another notable observation is that while a substantial number of articles employ datasets related to IoT, most works use models without considering the inherent constraints of IoT devices and networks. Indeed, only one work [170] uses BNNs, which could be considered a promising approach for IoT scenarios. Additionally, we found that other promising options in terms of lightweightness and low complexity, such as RaNN [151], are not considered in the analyzed works.

Furthermore, while unsupervised and semi-supervised schemes mitigate some of the limitations of supervised learning, we found that only a small portion of the surveyed works considers these techniques. Specifically, FL-enabled IDS using unsupervised techniques represent 16% (17 out of 104), while only a 5.8% of them consider semi-supervised approaches (6 out of 104).

In the case of unsupervised techniques, the most widely used approach is AE (9). Although AE is a widely known technique, it is worth noting that most of the surveyed works use simple or stacked AEs, and only one exploits VAEs [229], which are considered a promising approach for intrusion detection. Furthermore, we noticed that the use of GANs has been barely considered so far for generating attack data that could help improve the effectiveness of developed approaches. Due to the recent advances in generative models, it is very likely that the use of these techniques will expand in the coming years for the development of IDS.

Finally, only 2.9% (3 out of 104) of the analyzed works employ an RL-based approach. Despite the potential advantages of RL in terms of adapting to possible environmental changes for attack detection, one of the commonly accepted challenges is the definition of a suitable reward function in the context of IDS. In this regard, some recent works [128] propose replacing the environment with a sampling function of recorded training intrusions. We believe that the use of RL approaches needs to be further investigated in the field of IDS, as well as variants based on **Inverse Reinforcement Learning** (**IRL**) [23] that can aid in better understanding the cybersecurity behavior of devices and their users. As it is widely known, in many real-world scenarios, the combination of different approaches and techniques is likely to enhance the effectiveness of intrusion detection. One noteworthy aspect of our analysis of FL-enabled IDS development is the lack of schemes that

consider different models to be used at each FL client. Although this may be feasible in small-scale controlled settings, enforcing the use of the same model for all FL clients is not a realistic approach for environments where different organizations, such as companies or even **Security Operation Centers** (**SOCs**) could be acting as FL clients during the training process.

5 Aggregation Methods

As already described in Section 2, aggregation methods are a core part of FL. They determine how the local model updates provided by FL clients are combined, resulting in a new aggregated model in each training round. Appendix E details the main aggregation functions used in current literature, indicating the advantages and disadvantages of each approach, and the use of such techniques in existing FL-enabled IDS approaches. In particular, Table 7 in Appendix E provides a concise description of the key characteristics associated to the most relevant approaches. Below, we provide an analysis on the use of such techniques in existing FL-enabled IDS approaches. It should be noted that a summarized overview of such analysis is provided in Figure 6 in Appendix E.

Analysis: as previously described, the choice of the FL aggregation approach is crucial for the development of a robust and secure FL-enabled IDS. Although the initially proposed aggregation approach (i.e., FedAvg) relies on averaging client updates in each training round, subsequent studies have demonstrated the limitations of this approach in scenarios with a high degree of data and device heterogeneity, as well as in terms of security and efficiency [223]. As a result, additional aggregation approaches were proposed to address (at least, partially) these limitations. For example, approaches based on FedCD, **Communication-Mitigated Federated Learning** (**CMFL**), FedMA, FedProx, and Fed+ deal with data and device heterogeneity for mitigating the impact derived from non-IID data distributions and stragglers. While FedMA seems to be oriented toward specific ML models, Fed+ offers a significantly flexible approach that can be considered in different aggregation functions, such as mean or median. Moreover, security aspects are addressed by approaches like Turbo-Aggregate, SecureBoost, and Krum, given in subsections E.6.1, E.6.6, and E.6.3, respectively. While the first two are more related to privacy aspects, Krum presents an approach that may be of interest for designing more robust FL systems, where some updates are discarded and therefore not aggregated. Additionally, depending on the number of clients involved, as well as the number of training rounds and their frequency, the computation and communication overhead of the FL aggregation process can be unaffordable under certain circumstances. In this regard, some of the described approaches, including FedPAQ or SAFA, are intended to improve the performance of the training process.

Beyond the general analysis of aggregation approaches, we examine the use of different methods in the context of FL-enabled IDS. To this end, Figure 6 offers a summarized overview of the usage of each aggregation function in existing literature. Based on our literature review, we found that the vast majority of approaches (87/104 or 83.7%) use FedAvg or they are mostly based on it as the aggregation mechanism. One notable observation is that some works do not explicitly mention which aggregation function is employed, indicating that this aspect may currently be sufficiently considered in the current literature. In cases where it is not explicitly indicated, we assume that FedAvg is the employed method.

The convergence issues stemming from the use of FedAvg have been partially addressed by some approaches using alternative methods. In this regard, three works (2.88%) consider FedProx-based approaches. In particular, [202] proposes a FedProx-based aggregation mechanism where nodes are grouped such that, in each training round, certain nodes are selected according to the descending gradient order. Additionally, [196] and [195] propose a robust version of FedProx where nodes suspected of producing fake updates are removed from the process. The use of Fed+ is also considered by three works (2.88%); [167] compares the Fed+ approach with other commonly used aggregation functions, such as FedAvg and **Coordinate Median** (**CM**). In the case

of [32], the authors provide an evaluation of the impact of Fed+ compared to FedAvg, considering a non-IID scenario. Similarly, [181] demonstrates that the use of Fed+ substantially improves FedAvg in a scenario with non-IID data distribution where DP techniques are also employed. Furthermore, the Krum approach is used by [208] to evaluate the impact of different poisoning attacks, considering various aggregation functions. Particularly, the authors use Krum to assess the robustness of a malware detection system considering a GAN that generates adversarial samples. In the same direction, [176] analyzes the impact of different aggregation functions, considering data and model poisoning attacks in their approach. These functions include versions of mean and median (coordinate-wise and trimmed mean [238]) specifically designed to discard potential outliers. Furthermore, the work in [158] uses SecureBoost as an aggregation approach to provide additional privacy features, considering local models based on GBDT and vertical FL scenarios.

No less important, as shown in Figure 6, a few works make use of "other" aggregation approaches. This category includes works proposing an aggregation scheme as part of their contributions. Among them, [41] proposes FedAGRU as an improvement to FedAvg to reduce communication rounds while ensuring convergence. Inspired by the previous approach, [135] proposes an aggregation function (FedACNN) where the updates from each client are given different weights depending on their contribution to the global model convergence. In the case of [113], the authors design an aggregation function based on FedAvg where certain parameters are dynamically adjusted. The work in [247] introduces a hierarchical aggregation approach based on k-means to alleviate the data imbalance problem. Moreover, [9] proposes an aggregation function (i.e., a temporal weighted averaging) that takes into account the heterogeneity of clients participating in the FL training process. Specifically, it considers the time required by clients to perform local training. The authors in [125] propose FedBatch, an adaptive aggregation approach where the aggregator does not need to wait to receive the aggregation from all clients in each training round. This proposal intends to address the straggler problem, especially in scenarios with interconnection issues between FL clients and the aggregator. Finally, [252] proposes an aggregation function based on the concepts of **Federated Distillation** (**FD**) to reduce communication overhead and improve performance in non-IID data scenarios. In spite of such efforts, we note that the existing literature around FL-enabled IDS lacks of suitable analysis about the implications associated to data/device heterogeneity and robustness in the development and deployment of such systems.

6 Challenges

The analysis of FL-enabled IDS literature highlights key challenges for future research. This section aims to provide an overview of such challenges, as well as current solutions and future paths to address them. This section outlines these challenges, existing solutions, and future directions based on surveyed works, FL literature [8, 14, 116], and the authors' experience [32, 137, 181]. In particular, we attempt to answer three main questions: (1) What is the nature of the challenge and what is its impact in the context of FL-enabled IDS?; (2) What solutions have been proposed in the context of FL-enabled IDS to address this challenge?; (3) What potential approaches should be considered in the future to address this challenge, and how can they be used in the context of FL-enabled IDS?. Furthermore, Table 8 in Appendix G provides a summary of the challenges considered by the FL-enabled IDS approaches analyzed in this study, as well as some of the key considerations to be addressed by future research. It should be noted that the same reference is repeated when it addresses more than one challenge.

6.1 Security

The security aspects of FL settings have attracted a significant interest in recent years [31, 148]. Unlike centralized approaches, security in an FL scenario is typically compromised by *model*

poisoning attacks, where the weights/gradients sent by clients in each training round are deliberately/maliciously modified. These attacks, combined with *data poisoning* techniques—such as *dirty-label* attacks that produce incorrect weight updates—pose a significant threat in FL because each client operates independently without data sharing. This independence allows attackers to introduce inconsistencies that affect the entire model's integrity. In addition to these widely studied attacks, other security threats specific to FL can arise, including *backdoor* attacks, *free-riding* attacks (where clients participate solely to obtain the trained model without contributing meaningful data), *availability* attacks (where malicious entities aim to exclude legitimate clients from the FL process), and typical *eavesdropping* attacks [148]. These attacks have unique impacts in the IDS context, as they can severely degrade an IDS's ability to detect cyberattacks accurately, potentially leading to false alarms or undetected threats depending on the deployment environment.

The main impact in the context of IDS is represented by reduction in performance for detecting cyberattacks, which can have serious consequences depending on the environment where the IDS is deployed. According to the analysis of the literature, 11/104 works (that is, 10.6%) have proposed a security approach to address some of the mentioned issues in the context of FL-enabled IDS. On one hand, some proposals have defined alternative aggregation functions to FedAvg (see Section 5) to provide a certain degree of resilience against malicious clients. For example, [196] proposes a robust approach derived from FedProx as a defense mechanism against different malicious devices. Additionally, [176] analyzes the impact of different robust aggregation functions against data and model poisoning attacks. In particular, some authors evaluate the use of well-known functions, such as trimmed mean [238], against attacks related to label-flipping and gradient modification. Beyond aggregation functions, some studies propose using GANs to detect malicious nodes by generating synthetic weights and gradients for comparison [198]. Although GANs can be considered a double-edged sword, their potential to simulate attack behaviors allows for proactive model robustness improvement. Trust and reputation-based approaches also play a critical role, as demonstrated in [108], where client updates are evaluated based on historical behavior to prevent potential model poisoning.

Recent works, such as [31], emphasize the importance of incorporating additional robust aggregation functions, such as Krum, Multi-Krum [30], and Bulyan [67], in FL-enabled IDS. Although these methods improve resilience against certain attacks, there is a notable gap in understanding their computational complexity and potential impact on IDS performance, especially given IDS's real-time detection requirements. Additionally, the lack of a standardized taxonomy of attacks in FL environments complicates comparisons across techniques, as each study may use varying definitions and assumptions regarding attack capabilities and attacker knowledge, a topic further examined in [192]. Beyond the definition of robust aggregation techniques, as mentioned by [31], the use of statistical approaches can be essential to identify nodes that are sending forged updates. Likewise, the application of cryptographic approaches and tools like TEE is not widely considered.

6.2 Privacy

Despite the well-known advantages of FL in terms of privacy, as extensively discussed in the literature, the exchange of weights/gradients during the training process raises serious privacy concerns, as information derived from the local dataset can be still inferred [8, 148]. As described in [223], privacy threats can involve diverse types of actors, such as *honest-but-curious aggregators*, which attempt to infer information from each client's dataset using the shared updates in each training round. Likewise, clients may also be able to infer information from the gradients of other clients using the information contained in the global models sent by the aggregator. Additionally, other attacks can be considered related to the inference of the participation of certain nodes in the training process [223].

The impact of privacy concerns on FL-enabled IDS depends largely on the nature of the data being used to detect attacks. For example, in the case of network data in a medical environment, the inference of health-related data of individuals can have a significant impact, especially regarding compliance with current data protection regulations, say, GDPR. According to our analysis, 9 out of 104 papers analyzed (8.7%) propose the use of different techniques to address privacy concerns in FL-enabled IDS environments. Most of the presented works are based on the application of cryptographic techniques for privacy preservation [65], such as DP, SMPC, and HE. DP involves adding noise to the weights exchanged in each training round and is used by [181], considering different DP techniques following Gaussian and Laplacian distributions in the context of attack detection in IoT. Other works based on DP include [81] and [104], as well as [70], which also integrates HE. This cryptographic approach allows computations to be performed on encrypted data. Moreover, SMPC refers to a cryptographic protocol where multiple parties perform a certain function without revealing their inputs. In this way, in the context of FL, the aggregator would not have access to the updates generated by each client [76].

While these cryptographic techniques provide some privacy assurances, they are often assessed without considering operational challenges unique to IDS, such as communication efficiency, heterogeneity, and system overhead. These aspects can influence the practical performance of FL-enabled IDS, potentially limiting detection accuracy due to resource constraints. Additionally, the literature lacks comprehensive studies evaluating how parameters like client count, training rounds, and data types affect privacy leakage in federated IDS environments [223]. Given the diversity of data across different IDS environments, further research is needed to develop techniques that balance privacy and efficacy in detecting attacks. In fact, these aspects should be considered along with the type of data and the environment where the IDS is deployed, as they can have a crucial impact on finding tradeoffs between privacy and system effectiveness in detecting attacks. Additionally, the described analyses do not encompass practical considerations, such as communication efficiency or overhead, as well as system heterogeneity. These aspects can pose a challenge for the deployed IDS to achieve adequate performance. Moreover, the use of GANs to generate synthetic data could be considered to avoid potential inference of real client data. However, the challenge lies in generating data that accurately represents reality while still preserving client privacy. Furthermore, there is a lack of approaches assessing the level of privacy offered by FL environments in compliance with legal frameworks such as GDPR.

6.3 Aggregator as a Bottleneck

Most current FL deployments are carried out in a centralized mode where the aggregator can become a *bottleneck*. Indeed, previous security and privacy issues are exacerbated in FL environments where a single entity is responsible for performing the entire aggregation process in each training round; this entity also becomes a *single point of failure*. Additionally, deploying a single aggregator has a clear impact on the scalability of the environment, as a large number of clients can cause an overwhelming computational and communication overhead. This problem can be further aggravated in synchronous FL approaches where the central entity performs the aggregation process only after all clients have sent their updates. Indeed, with reference to Section 6.5, clients' heterogeneity can lead to convergence problems due to the presence of clients with low processing capabilities, a.k.a *stragglers* [8].

In the context of FL-enabled IDS, the consequences of the mentioned problems can limit the applicability and effectiveness of the developed IDS. For example, in an IoT environment like a smart city with a large number of deployed devices, the FL training process can be significantly affected. According to our analysis, only 6/104 (5.8%) works address this problem in FL-enabled IDS approaches. The main solution is based on blockchain technology, which represents an immutable

ledger managed by a set of nodes based on different *smart contracts* and a certain *consensus* mechanism. We note that most proposed approaches utilize blockchain to maintain model updates throughout the training process (e.g., [15, 72, 168]). However, these proposed approaches do not provide some key specific details on how the system is designed and lack a comprehensive evaluation demonstrating the viability of the solution. Furthermore, although some approaches propose a distributed aggregation process [123], they do not provide details of the implemented smart contracts or evaluation of different consensus mechanisms.

We believe that additional research efforts are needed to provide evidence of the applicability of blockchain in FL to mitigate the problems associated with the aggregator as a centralized entity. Particularly, based on the experience of some authors in blockchain [74], we advocate for comprehensive evaluations, considering the communication and computation requirements that can be crucial in IDS systems. Future efforts should harness the potential of designing robust aggregation functions using smart contracts. Beyond blockchain-based approaches, recent decentralized FL approaches also need to be considered in the future. In this scenario, clients themselves are responsible for performing the aggregation process based on the updates received from other participants. Although the deployment of decentralized FL environments has recently gained great interest [241], additional efforts are still required to evaluate their applicability in specific contexts, especially considering the potential limitations of FL clients. In addition to this approach, other trends should consider asynchronous approaches in the aggregation process, where aggregation is performed as updates arrive [116], aiming to alleviate the computational and communication overhead of traditional synchronous approaches.

6.4 Data Heterogeneity

One of the main inherent characteristics of FL is represented by non-IID data distributions, which are common in real-world scenarios. This data heterogeneity (or statistical heterogeneity [8]) is characterized by the presence of devices with different size and class distribution, so the data from a single device are not representative of the entire dataset [223]. Although these aspects also affect centralized ML environments, they are aggravated in FL settings. As demonstrated in previous works [118], statistical heterogeneity has a key impact on the convergence of the federated training process, especially when FedAvg is employed. Indeed, some of the main aggregation functions described in Section 5, such as FedProx or Fed+, were mainly proposed to address the limitations of FedAvg in non-IID settings.

In the context of IDS, non-IID data distributions represent situations where devices may have a large number of samples of a certain type of attack. This is quite common in real-world scenarios where certain devices may be more vulnerable and serve as entry points to the system. According to our analysis, data heterogeneity is widely considered in the proposed FL-enabled IDS approaches (26/104, i.e., 25%). Some of these works (e.g., [59, 242, 256]) evaluate their approach in configurations with non-IID data distributions, although they do not propose any specific solution. As mentioned earlier, while the aggregation function is directly related to the system's convergence, according to Section 5, we note that most proposed schemes use approaches based on FedAvg, which has convergence issues [118]. Actually, only a few approaches use alternative aggregation functions, such as Fed+ ([181]) or FedProx-based schemes ([202]), while other authors [125] propose an alternative aggregation approach (FedBatch). Some works advocate the use of TL as an approach to address data heterogeneity. For example, TL has been used in scenarios where certain devices have access to a restricted set of training data [254], or a model built from labeled data is used to label other data [164]. Another common practice to address data heterogeneity is the use of oversampling and undersampling techniques (e.g., based on SMOTE-TOMEK [137]). Furthermore, a current trend is represented by generative approaches based on GANs [206].

While data heterogeneity has been widely considered, most proposed approaches do not address the heterogeneity associated with the representation of data considering different feature spaces (i.e., vertical FL). In fact, most solutions are based on dividing a dataset into a certain number of parts acting as FL clients. Specifically, according to our analysis, only three works ([76, 158, 255]) consider vertical FL scenarios. In real-world scenarios, the need for approaches addressing this level of heterogeneity will be required for the development of FL-enabled IDS. Otherwise, different organizations would be forced to represent their data using the same feature space, which may be unfeasible. Related to this issue, future approaches need to consider the heterogeneity of the model used for local training. On the opposite side, forcing all parties in a federated environment to use the same training model may limit its applicability in any context, particularly in the field of IDS. In this context, the potential of TL can be exploited as a potential approach. Additionally, a comprehensive theoretical and practical analysis of the impact of non-IID data distributions considering different aggregation functions and IDS datasets is needed. As mentioned, even though many works consider non-IID configurations, the reality is that most approaches still rely on using FedAvg as the aggregation approach. Furthermore, as mentioned in [223], the conducted evaluations need to consider the tuning of FL hyperparameters to favor the convergence of the developed systems even in scenarios with a high degree of data heterogeneity.

6.5 Device Heterogeneity

FL scenarios are often characterized by the presence of diverse types of devices with varying processing capabilities, network connectivity, and battery life [116]. Additionally, depending on their local datasets, these devices may either benefit or hinder the convergence of the whole FL system. Furthermore, depending on the considered scenario, certain clients may enter/leave the FL training process due to poor network capabilities, failures, or mobility factors. Moreover, certain devices may voluntarily choose not to participate in the training process due to critical tasks they need to perform at a given time. This heterogeneity of FL clients significantly impacts the effectiveness and efficiency of FL systems, so that effective client selection approaches are essential [43]. As described in [8], different processing capabilities can favor the presence of stragglers. With reference to Section 6.3, this circumstance exacerbates the problem of the *aggregator as a bottleneck* and requires asynchronous communication approaches to mitigate the impact in terms of processing time [116].

In the context of IDS, only 8/104 of the analyzed works (7.7%) tackle this issue. Most of them are based on client selection mechanisms considering different aspects, such as client contribution to the global model [150, 204], attack type [27], as well as communication capabilities and computational power [232]. Communication aspects are also addressed by [9], which considers an aggregation function to determine the waiting time of the aggregator before initiating a new training round. A common feature of the proposed approaches is that they do not consider the changing conditions of the environment and the devices themselves. Additionally, they do not offer a comprehensive analysis of the system's complexity and required delay, which could significantly affect the performance of IDS. Furthermore, as detailed in Section 6.4, client selection approaches should also consider data heterogeneity. Such approaches should rely on solutions that do not require accessing the data or related information, e.g., labels. In the context of IDS, this could provide information about which devices are being targeted and by what types of attacks.

Although device heterogeneity has been partially addressed in the context of IDS, further research efforts are required to limit its effects. Firstly, a more comprehensive analysis is needed regarding which features of devices can affect the effectiveness and efficiency of the FL training process. While it is commonly accepted that computing and network capabilities have a significant impact, the extent to which each of these characteristics can affect the training process is unclear. As described

by [120], a potential approach is the deployment of edge nodes that homogenize the resulting system. However, the addition of edge nodes can render management more complex and introduce communication delays. Secondly, device heterogeneity should be addressed by considering the dynamism of the environment and the devices themselves throughout their lifecycle. In the context of IDS, we observe that only very few works (e.g., [171]) address dynamism during the FL training process. In fact, almost all the analyzed works consider static scenarios where devices already have their entire local dataset before the training process begins. In real-world scenarios, certain devices may receive new data while participating in FL training. Therefore, the design and implementation of resource management approaches are essential in the near future [120, 223]. Specifically, RL-based solutions [128] could help capture device properties through interaction with the environment to select the most appropriate clients. Additionally, the use of SDNs could facilitate such resource management by allowing clients and the underlying network to adapt to changing environmental conditions.

6.6 Deployment in Constrained Scenarios

Related to the previous challenge, an additional aspect is represented by the deployment of FL systems in scenarios with certain constraints. In fact, this aspect is inherent to FL since, unlike centralized ML approaches, end devices must bear a higher processing load. Firstly, depending on the scenario, certain devices may be unable to execute certain ML/DL algorithms and techniques. Even if they can be executed, a training process that requires a significant number of epochs or rounds may be infeasible depending on the ML model being considered. Besides, it has been shown that a key factor is the overhead required for model exchange between the aggregator and clients in each training round [131]. This aspect also impacts the overhead required during the training process, which can exacerbate the bottleneck problem of the aggregator.

According to our analysis, only 7 out of 104 works address the problems associated with deploying FL-enabled IDS in resource-constrained environments. Namely, our analysis reveals that while many works make use of datasets related to IoT, the practical deployment aspects are not truly addressed. In particular, only a few works consider lightweight ML approaches based on NNs [78, 135, 243], such as BNNs [170] for reducing the memory and computation overhead. Other works are more focused on lessening the communication overhead during the training process through the application of encoding techniques [195] and FD [252].

This limited sample of works highlights the need for additional approaches in the coming years that offer more comprehensive analysis of the limits in applying FL-based approaches in collaborative IDS development. In particular, a potential research direction is determined by the use of lightweight ML techniques, such as BNNs or Random NNs, which currently are not considered widely. Additionally, the deployment of systems based on TinyML [184] frameworks needs to be considered in order to accommodate the performance requirements of FL systems in constrained devices and environments. These frameworks offer implementations of pruning and quantization techniques to reduce the size of models, which can be considered in each FL training round. Specifically, pruning is a technique to reduce models by eliminating redundant and unnecessary connections or parameters. In the case of quantization, it involves reducing the numerical precision of a model's parameters. However, these techniques come with a cost in the model's performance in terms of inference precision. Therefore, additional efforts are required to find tradeoffs between efficiency and effectiveness in FL scenarios.

7 Conclusions

During the last few years, FL has proliferated throughout the IDS domain thanks to its decentralized, collaborative, and privacy-preserving intrinsic traits. Characteristically, with reference to Figure 2,

the number of peer-review publications in FL-powered IDS has mushroomed from less than 5 in 2018 to more than 150 in 2022. Nevertheless, as explained in Section 1, so far, no survey work provides a full-fledged, systematic overview of this rapidly evolving topic. The article at hand aspires to address this noticeable research gap by offering a pluralistic understanding of this composite ecosystem spanning several key axes. That is, apart from providing a contemporary taxonomy of the FL-enabled IDS approaches in the literature from 2018 to 2022, we detail the major ML models, datasets, aggregation functions, and implementation libraries, which are used in the context of such works. Additionally, we offer a thorough view of the current state of affairs on this topic, and identify the main challenges and future directions. Overall, due to its holistic orientation, this work is envisioned to not only shed more light upon this interesting research branch but also serve as a source of reference for future work.

References

[1] 2018. Contagio dataset. Retrieved January 30, 2025 from https://www.impactcybertrust.org/dataset_view?idDataset=1273

[2] 2020. Nrel National Renewable Energy Laboratory. Retrieved January 30, 2025 from https://www.nrel.gov/solar/

[3] 2016. pytorch. Retrieved January 30, 2025 from https://pytorch.org/

[4] 1999. KDDCup99 dataset. Retrieved January 30, 2025 from http://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html

[5] 2019. DDoS Botnet Attack on IOT Devices. Retrieved January 30, 2025 from https://www.kaggle.com/datasets/siddharthm1698/ddos-botnet-attack-on-iot-devices

[6] 2020. STIN-data-set. Retrieved January 30, 2025 from https://github.com/kun9717/STIN-data-set/

[7] Mohamed Abdel-Basset, Nour Moustafa, Hossam Hawash, Imran Razzak, Karam M. Sallam, and Osama M. Elkomy. 2021. Federated intrusion detection in blockchain-based smart transportation systems. *IEEE Transactions on Intelligent Transportation Systems* 23, 3 (2021), 2523–2537.

[8] Sawsan AbdulRahman, Hanine Tout, Hakima Ould-Slimane, Azzam Mourad, Chamseddine Talhi, and Mohsen Guizani. 2021. A survey on federated learning: The journey from centralized to distributed on-site learning and beyond. *IEEE Internet of Things Journal* 8, 7 (2021), 5476–5497.

[9] Shaashwat Agrawal, Aditi Chowdhuri, Sagnik Sarkar, Ramani Selvanambi, and Thippa Reddy Gadekallu. 2021. Temporal weighted averaging for asynchronous federated intrusion detection systems. *Computational Intelligence and Neuroscience* 2021, 1 (2021), 5844728.

[10] Shaashwat Agrawal, Sagnik Sarkar, Ons Aouedi, Gokul Yenduri, Kandaraj Piamrat, Sweta Bhattacharya, Praveen Kumar Reddy Maddikunta, and Thippa Reddy Gadekallu. 2022. Federated learning for intrusion detection system: Concepts, challenges and future directions. *Computer Communications* 195 (2022), 346–361.

[11] Muna Al-Hawawreh, Elena Sitnikova, and Neda Aboutorab. 2021. X-IIoTID: A connectivity-and device-agnostic intrusion dataset for industrial internet of things. *IEEE Internet of Things Journal* (2021).

[12] Noor Ali Al-Athba Al-Marri, Bekir S. Ciftler, and Mohamed M. Abdallah. 2020. Federated mimic learning for privacy preserving intrusion detection. In *Proceedings of the 2020 IEEE International Black Sea Conference on Communications and Networking (BlackSeaCom)*. IEEE, 1–6.

[13] Malik Bader Alazzam, Fawaz Alassery, and Ahmed Almulihi. 2022. Federated deep learning approaches for the privacy and security of Iot systems. *Wireless Communications and Mobile Computing* 1, 2022 (2022).

[14] Mohammed Aledhari, Rehma Razzak, Reza M. Parizi, and Fahad Saeed. 2020. Federated learning: A survey on enabling technologies, protocols, and applications. *IEEE Access* 8 (2020), 140699–140725.

[15] Ibrahim Aliyu, Marco Carlo Feliciano, Sélinde Van Engelenburg, Dong Ok Kim, and Chang Gyoon Lim. 2021. A Blockchain-based federated forest for SDN-enabled in-vehicle network intrusion detection system. *IEEE Access* 9 (2021), 102593–102608.

[16] Kevin Allix, Tegawendé F. Bissyandé, Jacques Klein, and Yves Le Traon. 2016. Androzoo: Collecting millions of android apps for the research community. In *Proceedings of the 2016 IEEE/ACM 13th Working Conference on Mining Software Repositories (MSR)*. IEEE, 468–471.

[17] Josue Genaro Almaraz-Rivera, Jesus Arturo Perez-Diaz, Jose Antonio Cantoral-Ceballos, Juan Felipe Botero, and Luis A. Trejo. 2022. Toward the protection of IoT networks: Introducing the LATAM-DDoS-IoT dataset. *IEEE Access* 10 (2022), 106909–106920.

[18] Iman Almomani, Bassam Al-Kasasbeh, and Mousa Al-Akhras. 2016. WSN-DS: A dataset for intrusion detection systems in wireless sensor networks. *Journal of Sensors* 2016, 1 (2016), 4731953.

[19] Abdullah Alsaedi, Nour Moustafa, Zahir Tari, Abdun Mahmood, and Adnan Anwar. 2020. TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems. *IEEE Access* 8 (2020), 165130–165150.

[20] Ons Aouedi, Kandaraj Piamrat, Guillaume Muller, and Kamal Singh. 2022. Federated semi-supervised learning for attack detection in industrial internet of things. *IEEE Transactions on Industrial Informatics* (2022).

[21] Ons Aouedi, Kandaraj Piamrat, Guillaume Muller, and Kamal Singh. 2022. FLUIDS: Federated learning with semi-supervised approach for intrusion detection system. In *Proceedings of the 2022 IEEE 19th Annual Consumer Communications & Networking Conference (CCNC)*. IEEE, 523–524.

[22] Sarhad Arisdakessian, Omar Abdel Wahab, Azzam Mourad, Hadi Otrok, and Mohsen Guizani. 2022. A survey on IoT intrusion detection: Federated learning, game theory, social psychology, and explainable AI as future directions. *IEEE Internet of Things Journal* 10, 5 (2022), 4059–4092.

[23] Saurabh Arora and Prashant Doshi. 2021. A survey of inverse reinforcement learning: Challenges, methods and progress. *Artificial Intelligence* 297 (2021), 103500.

[24] Daniel Arp, Michael Spreitzenbarth, Malte Hubner, Hugo Gascon, Konrad Rieck, and CERT Siemens. 2014. Drebin: Effective and explainable detection of android malware in your pocket. In *Proceedings of the 2014 Network and Distributed System Security Symposium*. Vol. 14, 23–26.

[25] Mohammad Ashrafuzzaman, Saikat Das, Ananth A. Jillepalli, Yacine Chakhchoukh, and Frederick T. Sheldon. 2020. Elliptic envelope based detection of stealthy false data injection attacks in smart grid control systems. In *Proceedings of the 2020 IEEE Symposium Series on Computational Intelligence (SSCI)*. IEEE, 1131–1137.

[26] Dinesh Chowdary Attota, Viraaji Mothukuri, Reza M. Parizi, and Seyedamin Pouriyeh. 2021. An ensemble multi-view federated learning intrusion detection for IoT. *IEEE Access* 9 (2021), 117734–117745.

[27] Mohamed Ali Ayed and Chamseddine Talhi. 2021. Federated learning for anomaly-based intrusion detection. In *Proceedings of the 2021 International Symposium on Networks, Computers and Communications (ISNCC)*. IEEE, 1–8.

[28] Elaheh Biglar Beigi, Hossein Hadian Jazi, Natalia Stakhanova, and Ali A. Ghorbani. 2014. Towards effective feature selection in machine learning-based botnet detection approaches. In *Proceedings of the 2014 IEEE Conference on Communications and Network Security*. IEEE, 247–255.

[29] Aitor Belenguer, Jose A. Pascual, and Javier Navaridas. 2025. *A Review of Federated Learning Applications in Intrusion Detection Systems*. Computer Networks, Article 111023.

[30] Peva Blanchard, El Mahdi El Mhamdi, Rachid Guerraoui, and Julien Stainer. 2017. Machine learning with adversaries: Byzantine tolerant gradient descent. In *Proceedings of the 31st International Conference on Neural Information Processing Systems* .

[31] Alberto Blanco-Justicia, Josep Domingo-Ferrer, Sergio Martínez, David Sánchez, Adrian Flanagan, and Kuan Eeik Tan. 2021. Achieving security and privacy in federated learning systems: Survey, research challenges and future directions. *Engineering Applications of Artificial Intelligence* 106 (2021), 104468.

[32] Enrique Mármol Campos, Pablo Fernández Saura, Aurora González-Vidal, José L. Hernández-Ramos, Jorge Bernal Bernabe, Gianmarco Baldini, and Antonio Skarmeta. 2021. Evaluating federated learning for intrusion detection in internet of things: Review and challenges. *Computer Networks* 203 (2021), 108661.

[33] Fernando E. Casado, Dylan Lema, Marcos F. Criado, Roberto Iglesias, Carlos V. Regueiro, and Senén Barro. 2022. Concept drift detection and adaptation for federated and continual learning. *Multimedia Tools and Applications* 81, 3 (2022), 3397–3419.

[34] M. Emre Celebi and Kemal Aydin. 2016. *Unsupervised Learning Algorithms*. Vol. 9. Springer.

[35] Burak Cetin, Alina Lazar, Jinoh Kim, Alex Sim, and Kesheng Wu. 2019. Federated wireless network intrusion detection. In *Proceedings of the 2019 IEEE International Conference on Big Data (Big Data)*. IEEE, 6004–6006.

[36] Efstratios Chatzoglou, Georgios Kambourakis, and Constantinos Kolias. 2021. Empirical evaluation of attacks against IEEE 802.11 enterprise networks: The AWID3 dataset. *IEEE Access* 9 (2021), 34188–34205.

[37] Efstratios Chatzoglou, Vasileios Kouliaridis, Georgios Kambourakis, Georgios Karopoulos, and Stefanos Gritzalis. 2023. A hands-on gaze on HTTP/3 security through the lens of HTTP/2 and a public dataset. *Computers & Security* 125 (2023), 103051. DOI: 10.1016/j.cose.2022.103051

[38] Ayushi Chaudhuri, Arijit Nandi, and Buddhadeb Pradhan. 2023. A dynamic weighted federated learning for android malware classification. In *Soft Computing: Theories and Applications: Proceedings of SoCTA 2022*. Springer, 147–159.

[39] Naiyue Chen, Yi Jin, Yinglong Li, and Luxin Cai. 2021. Trust-based federated learning for network anomaly detection. In *Web Intelligence*. SAGE Publications, 1–11.

[40] Yang Chen, Junzhe Zhang, and Chai Kiat Yeo. 2022. Privacy-preserving knowledge transfer for intrusion detection with federated deep autoencoding Gaussian mixture model. *Information Sciences* 609 (2022), 1204–1220.

[41] Zhuo Chen, Na Lv, Pengfei Liu, Yu Fang, Kun Chen, and Wu Pan. 2020. Intrusion detection for wireless edge networks based on federated learning. *IEEE Access* 8 (2020), 217463–217472.

[42] Kewei Cheng, Tao Fan, Yilun Jin, Yang Liu, Tianjian Chen, Dimitrios Papadopoulos, and Qiang Yang. 2021. Secureboost: A lossless federated learning framework. *IEEE Intelligent Systems* 36, 6 (2021), 87–98.

[43] Yanyu Cheng, Jianyuan Lu, Dusit Niyato, Biao Lyu, Jiawen Kang, and Shunmin Zhu. 2022. Federated transfer learning with client selection for intrusion detection in mobile edge computing. *IEEE Communications Letters* 26, 3 (2022), 552–556.

[44] Jesse Clifton and Eric Laber. 2020. Q-learning: Theory and applications. *Annual Review of Statistics and Its Application* 7 (2020), 279–301.

[45] Sajjad Dadkhah, Hassan Mahdikhani, Priscilla Kyei Danso, Alireza Zohourian, Kevin Anh Truong, and Ali A. Ghorbani. 2022. Towards the development of a realistic multidimensional IoT profiling dataset. In *Proceedings of the 2022 19th Annual International Conference on Privacy, Security & Trust (PST)*. IEEE, 1–11.

[46] Gustavo de Carvalho Bertoli, Lourenço Alves Pereira Júnior, and Osamu Saotome. 2022. Improving detection of scanning attacks on heterogeneous networks with Federated Learning. *ACM SIGMETRICS Performance Evaluation Review* 49, 4 (2022), 118–123.

[47] Mostafa Dehghani, Yi Tay, Alexey A. Gritsenko, Zhe Zhao, Neil Houlsby, Fernando Diaz, Donald Metzler, and Oriol Vinyals. 2021. The benchmark lottery. arXiv:2107.07002. Retrieved from https://arxiv.org/abs/2107.07002

[48] Tian Dong, Han Qiu, Jialiang Lu, Meikang Qiu, and Chun Fan. 2021. Towards fast network intrusion detection based on efficiency-preserving federated learning. In *Proceedings of the 2021 IEEE International Conference on Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social Computing & Networking (ISPA/BDCloud/SocialCom/SustainCom)*. IEEE, 468–475.

[49] Gerard Draper-Gil, Arash Habibi Lashkari, Mohammad Saiful Islam Mamun, and Ali A. Ghorbani. 2016. Characterization of encrypted and vpn traffic using time-related. In *Proceedings of the 2nd International Conference on Information Systems Security and Privacy (ICISSP)*. 407–414.

[50] Maha Driss, Iman Almomani, Jawad Ahmad, et al. 2022. A federated learning framework for cyberattack detection in vehicular sensor networks. *Complex & Intelligent Systems* 8, 5 (2022), 4221–4235.

[51] Phan The Duy, Tran Van Hung, Nguyen Hong Ha, Hien Do Hoang, and Van-Hau Pham. 2021. Federated learning-based intrusion detection in SDN-enabled IIoT networks. In *Proceedings of the 2021 8th NAFOSTED Conference on Information and Computer Science (NICS)*. IEEE, 424–429.

[52] Mahmoud Said Elsayed, Nhien-An Le-Khac, and Anca D. Jurcut. 2020. InSDN: A novel SDN intrusion dataset. *IEEE Access* 8 (2020), 165263–165284.

[53] European Commission. 2021. Proposal for a regulation of the European Parliament and the council laying down harmonised rules on artificial intelligence (artificial intelligence act) and amending certain union legislative acts. com/2021/206 final. *EUR-Lex-52021PC0206* (April 2021).

[54] Yulin Fan, Yang Li, Mengqi Zhan, Huajun Cui, and Yan Zhang. 2020. IoTDefender: A federated transfer learning intrusion detection framework for 5G IoT. In *Proceedings of the 2020 IEEE 14th International Conference on Big Data Science and Engineering (BigDataSE)*. IEEE, 88–95.

[55] Elena Fedorchenko, Evgenia Novikova, and Anton Shulepov. 2022. Comparative review of the intrusion detection systems based on federated learning: Advantages and open challenges. *Algorithms* 15, 7 (2022), 247.

[56] Mohamed Amine Ferrag, Othmane Friha, Djallel Hamouda, Leandros Maglaras, and Helge Janicke. 2022. Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning. *IEEE Access* (2022).

[57] Mohamed Amine Ferrag, Othmane Friha, Leandros Maglaras, Helge Janicke, and Lei Shu. 2021. Federated deep learning for cyber security in the internet of things: Concepts, applications, and experimental analysis. *IEEE Access* 9 (2021), 138509–138542.

[58] Ivo Frazão, Pedro Henriques Abreu, Tiago Cruz, Hélder Araújo, and Paulo Simões. 2018. Denial of service attacks: Detecting the frailties of machine learning algorithms in the classification process. In *Proceedings of the International Conference on Critical Information Infrastructures Security*. Springer, 230–235.

[59] Othmane Friha, Mohamed Amine Ferrag, Lei Shu, Leandros Maglaras, Kim-Kwang Raymond Choo, and Mehdi Nafaa. 2022. FELIDS: Federated learning-based intrusion detection system for agricultural Internet of Things. *Journal of Parallel and Distributed Computing* 165 (2022), 17–31.

[60] Sunanda Gamage and Jagath Samarabandu. 2020. Deep learning methods in network intrusion detection: A survey and an objective comparison. *Journal of Network and Computer Applications* 169 (2020), 102767.

[61] Sebastian Garcia, Martin Grill, Jan Stiborek, and Alejandro Zunino. 2014. An empirical comparison of botnet detection methods. *Computers & Security* 45 (2014), 100–123.

[62] Federated Deep Autoencoding Gaussian. 2020. Network anomaly detection using federated deep autoencoding Gaussian mixture model. In *Proceedings of the 2nd IFIP TC 6 International Conference on Machine Learning for Networking, MLN 2019, Revised Selected Papers*. Vol. 12081, Springer Nature, 1.

[63] Bimal Ghimire and Danda B. Rawat. 2022. Recent advances on federated learning for cybersecurity and cybersecurity for federated learning for internet of things. *IEEE Internet of Things Journal* (2022).

[64] Jonathan Goh, Sridhar Adepu, Khurum Nazir Junejo, and Aditya Mathur. 2016. A dataset to support research in the design of secure water treatment systems. In *Proceedings of the International Conference on Critical Information Infrastructures Security*. Springer, 88–99.

[65] Slawomir Goryczka and Li Xiong. 2015. A comprehensive comparison of multiparty secure additions with differential privacy. *IEEE Transactions on Dependable and Secure Computing* 14, 5 (2015), 463–477.
[66] Alejandro Guerra-Manzanares, Jorge Medina-Galindo, Hayretdin Bahsi, and Sven Nõmm. 2020. MedBIoT: Generation of an IoT botnet dataset in a medium-sized IoT network. In *Proceedings of the International Conference on Information Systems Security and Privacy*. 207–218.
[67] Rachid Guerraoui, Sébastien Rouault, et al. 2018. The hidden vulnerability of distributed learning in byzantium. In *Proceedings of the International Conference on Machine Learning*. PMLR, 3521–3530.
[68] Jie Gui, Tuo Chen, Jing Zhang, Qiong Cao, Zhenan Sun, Hao Luo, and Dacheng Tao. 2024. A survey on self-supervised learning: Algorithms, applications, and future trends. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2024).
[69] Jie Gui, Zhenan Sun, Yonggang Wen, Dacheng Tao, and Jieping Ye. 2021. A review on generative adversarial networks: Algorithms, theory, and applications. *IEEE Transactions on Knowledge and Data Engineering* (2021).
[70] Huynh Nhat Hao, Huynh Minh Chu, Van-Hau Pham, et al. 2021. A secure and privacy preserving federated learning approach for IoT intrusion detection system. In *Proceedings of the International Conference on Network and System Security*. Springer, 353–368.
[71] Sahand Hariri, Matias Carrasco Kind, and Robert J. Brunner. 2019. Extended isolation forest. *IEEE Transactions on Knowledge and Data Engineering* 33, 4 (2019), 1479–1489.
[72] Xinhong Hei, Xinyue Yin, Yichuan Wang, Ju Ren, and Lei Zhu. 2020. A trusted feature aggregator federated learning for distributed malicious attack detection. *Computers & Security* 99 (2020), 102033.
[73] Rens W. Heijden, Thomas Lukaseder, and Frank Kargl. 2018. VeReMi: A dataset for comparable evaluation of misbehavior detection in VANETs. In *Proceedings of the International Conference on Security and Privacy in Communication Systems*. Springer, 318–337.
[74] José L. Hernández-Ramos, Georgios Karopoulos, Dimitris Geneiatakis, Tania Martin, Georgios Kambourakis, and Igor Nai Fovino. 2021. Sharing pandemic vaccination certificates through blockchain: Case study and performance evaluation. *Wireless Communications and Mobile Computing* 2021 (2021), 1–12.
[75] Hanan Hindy, Christos Tachtatzis, Robert Atkinson, Ethan Bayne, and Xavier Bellekens. 2020. MQTT-IoT-IDS2020: MQTT Internet of Things Intrusion Detection Dataset. *IEEE Dataport*. DOI: 10.21227/bhxy-ep04. Retrieved January 30, 2025 from https://ieee-dataport.org/open-access/mqtt-iot-ids2020-mqtt-internet-things-intrusion-detection-dataset
[76] Ruei-Hau Hsu, Yi-Cheng Wang, Chun-I Fan, Bo Sun, Tao Ban, Takeshi Takahashi, Ting-Wei Wu, and Shang-Wei Kao. 2020. A privacy-preserving federated learning system for android malware detection based on edge computing. In *Proceedings of the 2020 15th Asia Joint Conference on Information Security (AsiaJCIS)*. IEEE, 128–136.
[77] Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. 2016. Binarized neural networks. In *Proceedings of the 30th International Conference on Neural Information Processing Systems* .
[78] Truong Huong, Phuong Bac Ta, Dao Long, Bui Thang, Nguyen Binh, Tran Luong, and Kim Phuc TRAN. 2021. LocKedge: Low-complexity cyberattack detection in IoT edge computing. *IEEE Access* PP (2021).
[79] Faisal Hussain, Syed Ghazanfar Abbas, Ghalib A. Shah, Ivan Miguel Pires, Ubaid U. Fayyaz, Farrukh Shahzad, Nuno M. Garcia, and Eftim Zdravevski. 2021. IoT Healthcare Security Dataset. *IEEE Dataport*. DOI: 10.21227/9w13-2t13. Retrieved January 30, 2025 from https://ieee-dataport.org/documents/iot-healthcare-security-dataset
[80] Faisal Hussain, Syed Ghazanfar Abbas, Muhammad Husnain, Ubaid U. Fayyaz, Farrukh Shahzad, and Ghalib A. Shah. 2021. IoT DoS and DDoS Attack Dataset. *IEEE Dataport*. DOI: 10.21227/0s0p-s959. Retrieved January 30, 2025 from https://ieee-dataport.org/documents/iot-dos-and-ddos-attack-dataset
[81] Olakunle Ibitoye, M. Omair Shafiq, and Ashraf Matrawy. 2022. Differentially private self-normalizing neural networks for adversarial robustness in federated learning. *Computers & Security* 116 (2022), 102631.
[82] Obinna Igbe, Ihab Darwish, and Tarek Saadawi. 2017. Deterministic dendritic cell algorithm application to smart grid cyber-attack detection. In *Proceedings of the 2017 IEEE 4th International Conference on Cyber Security and Cloud Computing (CSCloud)*. IEEE, 199–204.
[83] Ahmed Imteaj, Urmish Thakker, Shiqiang Wang, Jian Li, and M. Hadi Amini. 2021. A survey on federated learning for resource-constrained IoT devices. *IEEE Internet of Things Journal* (2021).
[84] Wael Issa, Nour Moustafa, Benjamin Turnbull, Nasrin Sohrabi, and Zahir Tari. 2023. Blockchain-based federated learning for securing internet of things: A comprehensive survey. *ACM Computing Surveys* 55, 9 (2023), 1–43.
[85] Amir Namavar Jahromi, Hadis Karimipour Schulich, and Ali Dehghantanha. 2021. Deep federated learning-based cyber-attack detection in industrial control systems. In *Proceedings of the 2021 18th International Conference on Privacy, Security and Trust (PST)*. IEEE, 1–6.
[86] Abdul Rehman Javed, Mirza Omer Beg, Muhammad Asim, Thar Baker, and Ali Hilal Al-Bayatti. 2020. Alphalogger: Detecting motion-based side-channel attack using smartphone keystrokes. *Journal of Ambient Intelligence and Humanized Computing* 14 (2020), 4869–4882.

[87] Hossein Hadian Jazi, Hugo Gonzalez, Natalia Stakhanova, and Ali A. Ghorbani. 2017. Detecting HTTP-based application layer DoS attacks on web servers in the presence of sampling. *Computer Networks* 121 (2017), 25–36.

[88] Michael I. Jordan and Tom M. Mitchell. 2015. Machine learning: Trends, perspectives, and prospects. *Science* 349, 6245 (2015), 255–260.

[89] Leslie Pack Kaelbling, Michael L. Littman, and Andrew W. Moore. 1996. Reinforcement learning: A survey. *Journal of Artificial Intelligence Research* 4 (1996), 237–285.

[90] Hyunjae Kang, Dong Hyun Ahn, Gyung Min Lee, Jeong Do Yoo, Kyung Ho Park, and Huy Kang Kim. 2019. IoT network intrusion dataset. DOI: 10.21227/q70p-q449. Retrieved January 30, 2025 from https://ieee-dataport.org/open-access/iot-network-intrusion-dataset

[91] Hyunjae Kang, Byung Il Kwak, Young Hun Lee, Haneol Lee, Hwejae Lee, and Huy Kang Kim. 2021. Car hacking and defense competition on in-vehicle network. In *Proceedings of the Workshop on Automotive and Autonomous Vehicle Security (AutoSec)*. Vol. 2021, 25.

[92] Hyunjae Kang, Byung Il Kwak, Young Hun Lee, Haneol Lee, Hwejae Lee, and Huy Kang Kim. 2021. Car Hacking: Attack & Defense Challenge 2020 Dataset. DOI: 10.21227/qvr7-n418. Retrieved January 30, 2025 from https://ieee-dataport.org/open-access/car-hacking-attack-defense-challenge-2020-dataset

[93] Vasiliki Kelli, Vasileios Argyriou, Thomas Lagkas, George Fragulis, Elisavet Grigoriou, and Panagiotis Sarigiannidis. 2021. IDS for industrial applications: A federated learning approach with active personalization. *Sensors* 21, 20 (2021), 6743.

[94] Latif U. Khan, Walid Saad, Zhu Han, Ekram Hossain, and Choong Seon Hong. 2021. Federated learning for internet of things: Recent advances, taxonomy, and open challenges. *IEEE Communications Surveys & Tutorials* (2021).

[95] Tran Viet Khoa, Yuris Mulya Saputra, Dinh Thai Hoang, Nguyen Linh Trung, Diep Nguyen, Nguyen Viet Ha, and Eryk Dutkiewicz. 2020. Collaborative learning model for cyberattack detection systems in IoT Industry 4.0. In *Proceedings of the 2020 IEEE Wireless Communications and Networking Conference (WCNC)*. IEEE, 1–6.

[96] Ivan Kholod, Evgeny Yanaki, Dmitry Fomichev, Evgeniy Shalugin, Evgenia Novikova, Evgeny Filippov, and Mats Nordlund. 2020. Open-source federated learning frameworks for IoT: A comparative review and analysis. *Sensors* 21, 1 (2020), 167.

[97] Ansam Khraisat, Ammar Alazab, Sarabjot Singh, Tony Jan, and Alfredo Jr. Gomez. 2024. Survey on federated learning for intrusion detection system: Concept, architectures, aggregation strategies, challenges, and future directions. *ACM Computing Surveys* 57, 1 (2024), 1–38.

[98] Ansam Khraisat, Iqbal Gondal, Peter Vamplew, and Joarder Kamruzzaman. 2019. Survey of intrusion detection systems: Techniques, datasets and challenges. *Cybersecurity* 2, 1 (2019), 1–22.

[99] Seongwoo Kim, He Cai, Cunqing Hua, Pengwenlong Gu, Wenchao Xu, and Jeonghyeok Park. 2020. Collaborative anomaly detection for internet of things based on federated learning. In *Proceedings of the 2020 IEEE/CIC International Conference on Communications in China (ICCC)*. IEEE, 623–628.

[100] Constantinos Kolias, Vassilis Kolias, and Georgios Kambourakis. 2017. TermID: a distributed swarm intelligence-based approach for wireless intrusion detection. *International Journal of Information Security* 16, 4 (2017), 401–416. DOI: 10.1007/s10207-016-0335-z

[101] Kavya Kopparapu, Eric Lin, and Jessica Zhao. 2020. FedCD: Improving performance in non-IID federated learning. arXiv:2006.09637. Retrieved from https://arxiv.org/abs/2006.09637

[102] Nickolaos Koroniotis, Nour Moustafa, Elena Sitnikova, and Benjamin Turnbull. 2019. Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: Bot-IoT dataset. *Future Generation Computer Systems* 100 (2019), 779–796.

[103] Oliver Kramer and Oliver Kramer. 2016. Scikit-learn. *Machine Learning for Evolution Strategies* 20 (2016), 45–53.

[104] K. P. Sanal Kumar, S. Anu H. Nair, Deepsubhra Guha Roy, B. Rajalingam, and R. Santhosh Kumar. 2021. Security and privacy-aware artificial intrusion detection system using federated machine learning. *Computers & Electrical Engineering* 96 (2021), 107440.

[105] Prabhat Kumar, Govind P. Gupta, and Rakesh Tripathi. 2021. PEFL: Deep Privacy-encoding-based federated learning framework for smart agriculture. *IEEE Micro* 42, 1 (2021), 33–40.

[106] Junhyung Kwon, Byeonggil Jung, Hyungil Lee, and Sangkyun Lee. 2022. Anomaly detection in Multi-host environment based on federated hypersphere classifier. *Electronics* 11, 10 (2022), 1529.

[107] Wassila Lalouani and Mohamed Younis. 2021. Robust distributed intrusion detection system for edge of things. In *Proceedings of the 2021 IEEE Global Communications Conference (GLOBECOM)*. IEEE, 01–06.

[108] Wassila Lalouani and Mohamed Younis. 2022. A robust distributed intrusion detection system for collusive attacks on edge of things. In *Proceedings of the 2022 IEEE Wireless Communications and Networking Conference (WCNC)*. IEEE, 1004–1009.

[109] Arash Habibi Lashkari, Gerard Draper-Gil, Mohammad Saiful Islam Mamun, Ali A. Ghorbani, et al. 2017. Characterization of tor traffic using time based features. In *Proceedings of the International Conference on Information Systems Security and Privacy*. 253–262.

[110] Léo Lavaur, Marc-Oliver Pahl, Yann Busnel, and Fabien Autrel. 2022. The evolution of federated learning-based intrusion detection and mitigation: A survey. *IEEE Transactions on Network and Service Management* (2022).

[111] Hyunsung Lee, Seong Hoon Jeong, and Huy Kang Kim. 2017. OTIDS: A novel intrusion detection system for in-vehicle network by using remote frame. In *Proceedings of the 2017 15th Annual Conference on Privacy, Security and Trust (PST)*. IEEE, 57–5709.

[112] Beibei Li, Yuhao Wu, Jiarui Song, Rongxing Lu, Tao Li, and Liang Zhao. 2020. DeepFed: Federated deep learning for intrusion detection in industrial cyber–physical systems. *IEEE Transactions on Industrial Informatics* 17, 8 (2020), 5615–5624.

[113] Jingyi Li, Zikai Zhang, Yidong Li, Xinyue Guo, and Huifang Li. 2021. FIDS: Detecting DDoS through federated learning based method. In *Proceedings of the 2021 IEEE 20th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. IEEE, 856–862.

[114] Kun Li, Huachun Zhou, Zhe Tu, Weilin Wang, and Hongke Zhang. 2020. Distributed network intrusion detection system in satellite-terrestrial integrated networks using federated learning. *IEEE Access* 8 (2020), 214852–214865.

[115] Li Li, Yuxi Fan, Mike Tse, and Kuo-Yi Lin. 2020. A review of applications in federated learning. *Computers & Industrial Engineering* 149 (2020), 106854.

[116] Tian Li, Anit Kumar Sahu, Ameet Talwalkar, and Virginia Smith. 2020. Federated learning: Challenges, methods, and future directions. *IEEE Signal Processing Magazine* 37, 3 (2020), 50–60.

[117] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. 2020. Federated optimization in heterogeneous networks. *Proceedings of Machine Learning and Systems* 2 (2020), 429–450.

[118] Xiang Li, Kaixuan Huang, Wenhao Yang, Shusen Wang, and Zhihua Zhang. 2019. On the convergence of FedAvg on non-IID data. arXiv:1907.02189. Retrieved from https://arxiv.org/abs/1907.02189

[119] Nan-Ying Liang, Guang-Bin Huang, Paramasivan Saratchandran, and Narasimhan Sundararajan. 2006. A fast and accurate online sequential learning algorithm for feedforward networks. *IEEE Transactions on Neural Networks* 17, 6 (2006), 1411–1423.

[120] Wei Yang Bryan Lim, Nguyen Cong Luong, Dinh Thai Hoang, Yutao Jiao, Ying-Chang Liang, Qiang Yang, Dusit Niyato, and Chunyan Miao. 2020. Federated learning in mobile edge networks: A comprehensive survey. *IEEE Communications Surveys & Tutorials* 22, 3 (2020), 2031–2063.

[121] Tianyang Lin, Yuxin Wang, Xiangyang Liu, and Xipeng Qiu. 2022. A survey of transformers. *AI Open* (2022).

[122] Hongyu Liu and Bo Lang. 2019. Machine learning and deep learning methods for intrusion detection systems: A survey. *Applied Sciences* 9, 20 (2019), 4396.

[123] Hong Liu, Shuaipeng Zhang, Pengfei Zhang, Xinqiang Zhou, Xuebin Shao, Geguang Pu, and Yan Zhang. 2021. Blockchain and federated learning for collaborative intrusion detection in vehicular edge computing. *IEEE Transactions on Vehicular Technology* (2021).

[124] Ji Liu, Jizhou Huang, Yang Zhou, Xuhong Li, Shilei Ji, Haoyi Xiong, and Dejing Dou. 2022. From distributed machine learning to federated learning: A survey. *Knowledge and Information Systems* 64, 4 (2022), 885–917.

[125] Wentao Liu, Xiaolong Xu, Lianxiang Wu, Lianyong Qi, Alireza Jolfaei, Weiping Ding, and Mohammad R. Khosravi. 2022. Intrusion detection for maritime transportation systems with batch federated aggregation. *IEEE Transactions on Intelligent Transportation Systems* (2022).

[126] Xiaoyuan Liu, Tianneng Shi, Chulin Xie, Qinbin Li, Kangping Hu, Haoyu Kim, Xiaojun Xu, Bo Li, and Dawn Song. 2022. UniFed: A benchmark for federated learning frameworks. arXiv:2207.10308. Retrieved from https://arxiv.org/abs/2207.10308

[127] Zhipeng Liu, Niraj Thapa, Addison Shaver, Kaushik Roy, Madhuri Siddula, Xiaohong Yuan, and Anna Yu. 2021. Using embedded feature selection and CNN for classification on CCD-INID-V1—A new IoT dataset. *Sensors* 21, 14 (2021), 4834.

[128] Manuel Lopez-Martin, Belen Carro, and Antonio Sanchez-Esguevillas. 2020. Application of deep reinforcement learning to intrusion detection for supervised problems. *Expert Systems with Applications* 141 (2020), 112963.

[129] Yunlong Lu, Xiaohong Huang, Ke Zhang, Sabita Maharjan, and Yan Zhang. 2020. Low-latency federated learning and blockchain for edge association in digital twin empowered 6G networks. *IEEE Transactions on Industrial Informatics* 17, 7 (2020), 5098–5107.

[130] Jiechen Luo, Xuelan Yang, and Muamer N. Mohammed. 2022. Federation learning for intrusion detection methods by parse convolutional neural network. In *Proceedings of the 2022 2nd International Conference on Advances in Electrical, Computing, Communication and Sustainable Technologies (ICAECT)*. IEEE, 1–7.

[131] Luping Wang, Wei Wang, and Bo Li. 2019. CMFL: Mitigating communication overhead for federated learning. In *Proceedings of the 2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 954–964.

[132] Dingyang Lv, Xinyu Cheng, Jinghui Zhang, Wei Zhang, Wei Zhao, and He Xu. 2022. DDoS attack detection based on CNN and federated learning. In *Proceedings of the 2021 9th International Conference on Advanced Cloud and Big Data (CBD)*. IEEE, 236–241.

[133] Samaneh Mahdavifar, Andi Fitriah Abdul Kadir, Rasool Fatemi, Dima Alhadidi, and Ali A. Ghorbani. 2020. Dynamic android malware category classification using semi-supervised deep learning. In *Proceedings of the 2020 IEEE International Conference on Dependable, Autonomic and Secure Computing, International Conference on Pervasive Intelligence and Computing, International Conference on Cloud and Big Data Computing, International Conference on Cyber Science and Technology Congress (DASC/PiCom/CBDCom/CyberSciTech)*. IEEE, 515–522.

[134] Batta Mahesh. 2020. Machine learning algorithms-A review. *International Journal of Science and Research (IJSR).[Internet]* 9, 1 (2020), 381–386.

[135] Dapeng Man, Fanyi Zeng, Wu Yang, Miao Yu, Jiguang Lv, and Yijing Wang. 2021. Intelligent intrusion detection based on federated learning for edge-assisted internet of things. *Security and Communication Networks* 2021, 1 (2021), 9361348.

[136] Tijana Markovic, Miguel Leon, David Buffoni, and Sasikumar Punnekkat. 2022. Random forest based on federated learning for intrusion detection. In *Proceedings of the IFIP International Conference on Artificial Intelligence Applications and Innovations*. Springer, 132–144.

[137] Sara N. Matheu, Enrique Mármol, José L. Hernández-Ramos, Antonio Skarmeta, and Gianmarco Baldini. 2022. Federated cyberattack detection for internet of things-enabled smart cities. *Computer* 55, 12 (2022), 65–73.

[138] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. 2017. Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the International Conference on Artificial Intelligence and Statistics*. PMLR, 1273–1282.

[139] Caitlin L. McOsker, Michael Steven Handlin, Lei Li, Hossain Shahriar, and Liang Zhao. 2021. An architecture for federated learning enabled collaborative intrusion detection system. In *Proceedings of the 27th Americas Conference on Information Systems (AMCIS 2021)*. Article 26.

[140] Yair Meidan, Michael Bohadana, Yael Mathov, Yisroel Mirsky, Asaf Shabtai, Dominik Breitenbacher, and Yuval Elovici. 2018. N-BaIoT—network-based detection of IoT botnet attacks using deep autoencoders. *IEEE Pervasive Computing* 17, 3 (2018), 12–22.

[141] Jan Melchior, Nan Wang, and Laurenz Wiskott. 2017. Gaussian-binary restricted Boltzmann machines for modeling natural image statistics. *PloS One* 12, 2 (2017), e0171015.

[142] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. 2018. Kitsune: An ensemble of autoencoders for online network intrusion detection. In *Proceedings of the 2018 Network and Distributed System Security Symposium (NDSS 2018)*. Internet Society. DOI: 10.14722/ndss.2018.23204

[143] Parya Haji Mirzaee, Mohammad Shojafar, Zahra Pooranian, Pedram Asefy, Haitham Cruickshank, and Rahim Tafazolli. 2021. FIDS: A federated intrusion detection system for 5G smart metering network. In *Proceedings of the 2021 17th International Conference on Mobility, Sensing and Networking (MSN)*. IEEE, 215–222.

[144] Tom Mitchell, William Cohen, Estevam Hruschka, Partha Talukdar, Bishan Yang, Justin Betteridge, Andrew Carlson, Bhavana Dalvi, Matt Gardner, Bryan Kisiel, et al. 2018. Never-ending learning. *Communications of the ACM* 61, 5 (2018), 103–115.

[145] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. 2015. Human-level control through deep reinforcement learning. *Nature* 518, 7540 (2015), 529–533.

[146] Thomas Morris and Wei Gao. 2014. Industrial control system traffic data sets for intrusion detection research. In *Proceedings of the International Conference on Critical Infrastructure Protection*. Springer, 65–78.

[147] Viraaji Mothukuri, Prachi Khare, Reza M. Parizi, Seyedamin Pouriyeh, Ali Dehghantanha, and Gautam Srivastava. 2021. Federated learning-based anomaly detection for IoT security attacks. *IEEE Internet of Things Journal* (2021).

[148] Viraaji Mothukuri, Reza M. Parizi, Seyedamin Pouriyeh, Yan Huang, Ali Dehghantanha, and Gautam Srivastava. 2021. A survey on security and privacy of federated learning. *Future Generation Computer Systems* 115 (2021), 619–640.

[149] Nour Moustafa and Jill Slay. 2015. UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In *Proceedings of the 2015 Military Communications and Information Systems Conference (MilCIS)*. IEEE, 1–6.

[150] Nishat I. Mowla, Nguyen H. Tran, Inshil Doh, and Kijoon Chae. 2019. Federated learning-based cognitive detection of jamming attack in flying ad-hoc network. *IEEE Access* 8 (2019), 4338–4350.

[151] Mert Nakip and Erol Gelenbe. 2021. MIRAI botnet attack detection with auto-associative dense random neural network. In *Proceedings of the 2021 IEEE Global Communications Conference (GLOBECOM)*. IEEE, 01–06.

[152] Dinh C. Nguyen, Ming Ding, Pubudu N. Pathirana, Aruna Seneviratne, Jun Li, and H. Vincent Poor. 2021. Federated learning for internet of things: A comprehensive survey. *IEEE Communications Surveys & Tutorials* (2021).

[153] Dinh C. Nguyen, Quoc-Viet Pham, Pubudu N. Pathirana, Ming Ding, Aruna Seneviratne, Zihuai Lin, Octavia Dobre, and Won-Joo Hwang. 2022. Federated learning for smart healthcare: A survey. *ACM Computing Surveys (CSUR)* 55, 3 (2022), 1–37.

[154] Thien Duc Nguyen, Samuel Marchal, Markus Miettinen, Hossein Fereidooni, N. Asokan, and Ahmad-Reza Sadeghi. 2019. DÏoT: A federated self-learning anomaly detection system for IoT. In *Proceedings of the 2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 756–767.

[155] Tri Gia Nguyen, Trung V. Phan, Dinh Thai Hoang, Tu N. Nguyen, and Chakchai So-In. 2021. Federated deep reinforcement learning for traffic monitoring in SDN-based IoT networks. *IEEE Transactions on Cognitive Communications and Networking* 7, 4 (2021), 1048–1065.

[156] Antonia Nisioti, Alexios Mylonas, Paul D. Yoo, and Vasilios Katos. 2018. From intrusion detection to attacker attribution: A comprehensive survey of unsupervised methods. *IEEE Communications Surveys & Tutorials* 20, 4 (2018), 3369–3388.

[157] Karen Scarfone and Peter Mell. 2007. Guide to intrusion detection and prevention systems (IDPS). NIST special publication 800-94. National Institute of Standards and Technology.

[158] Evgenia Novikova, Elena Doynikova, and Sergey Golubev. 2022. Federated learning for intrusion detection in the critical infrastructures: Vertically partitioned data use case. *Algorithms* 15, 4 (2022), 104.

[159] Safa Otoum, Nadra Guizani, and Hussein Mouftah. 2021. Federated reinforcement learning-supported IDS for IoT-steered healthcare systems. In *ICC 2021-Proceedings of the IEEE International Conference on Communications*. IEEE, 1–6.

[160] Safa Otoum, Nadra Guizani, and Hussein Mouftah. 2022. On the feasibility of split learning, transfer Learning and federated learning for preserving security in ITS systems. *IEEE Transactions on Intelligent Transportation Systems* (2022).

[161] Yazan Otoum, Yue Wan, and Amiya Nayak. 2021. Federated transfer learning-based IDS for the internet of medical things (IoMT). In *Proceedings of the 2021 IEEE Globecom Workshops (GC Wkshps)*. IEEE, 1–6.

[162] Pascal Paillier. 1999. Public-key cryptosystems based on composite degree residuosity classes. In *Proceedings of the International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 223–238.

[163] Amirmohammad Pasdar, Young Choon Lee, Tongliang Liu, and Seok-Hee Hong. 2022. Train me to fight: Machine-Learning based on-device malware detection for mobile devices. In *Proceedings of the 2022 22nd IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. IEEE, 239–248.

[164] Xinjun Pei, Xiaoheng Deng, Shengwei Tian, Lan Zhang, and Kaiping Xue. 2022. A knowledge transfer-based semi-supervised federated learning for IoT malware detection. *IEEE Transactions on Dependable and Secure Computing* (2022).

[165] Peng Peng, Limin Yang, Linhai Song, and Gang Wang. 2019. Opening the blackbox of virustotal: Analyzing online phishing scan engines. In *Proceedings of the Internet Measurement Conference*. 478–485.

[166] Segun I. Popoola, Ruth Ande, Bamidele Adebisi, Guan Gui, Mohammad Hammoudeh, and Olamide Jogunola. 2021. Federated deep learning for zero-day botnet attack detection in IoT edge devices. *IEEE Internet of Things Journal* (2021).

[167] Segun I. Popoola, Guan Gui, Bamidele Adebisi, Mohammad Hammoudeh, and Haris Gacanin. 2021. Federated deep learning for collaborative intrusion detection in heterogeneous networks. In *Proceedings of the 2021 IEEE 94th Vehicular Technology Conference (VTC2021-Fall)*. IEEE, 1–6.

[168] Davy Preuveneers, Vera Rimmer, Ilias Tsingenopoulos, Jan Spooren, Wouter Joosen, and Elisabeth Ilie-Zudor. 2018. Chained anomaly detection models for federated learning: An intrusion detection case study. *Applied Sciences* 8, 12 (2018), 2663.

[169] Oscar Puñal, Carlos Pereira, Ana Aguiar, and James Gross. 2014. The uportorwthaachen/vanetjamming2014 dataset. DOI: 10.15783/C7Q306. Retrieved January 30, 2025 from https://ieee-dataport.org/open-access/crawdad-uportorwthaachenvanetjamming2014

[170] Qiaofeng Qin, Konstantinos Poularakis, Kin K. Leung, and Leandros Tassiulas. 2020. Line-speed and scalable intrusion detection at the network edge via federated learning. In *Proceedings of the 2020 IFIP Networking Conference (Networking)*. IEEE, 352–360.

[171] Tian Qin, Guang Cheng, Wenchao Chen, and Xuan Lei. 2021. FNEL: An evolving intrusion detection system based on federated never-ending learning. In *Proceedings of the 2021 17th International Conference on Mobility, Sensing and Networking (MSN)*. IEEE, 239–246.

[172] Yang Qin and Masaaki Kondo. 2021. Federated learning-based network intrusion detection with a feature selection approach. In *Proceedings of the 2021 International Conference on Electrical, Communication, and Computer Engineering (ICECCE)*. IEEE, 1–6.

[173] Sawsan Abdul Rahman, Hanine Tout, Chamseddine Talhi, and Azzam Mourad. 2020. Internet of things intrusion detection: Centralized, on-device, or federated learning? *IEEE Network* 34, 6 (2020), 310–317.

[174] Abdul Rehman, Imran Razzak, and Guandong Xu. 2022. Federated learning for privacy preservation of healthcare data from smartphone-based side-channel attacks. *IEEE Journal of Biomedical and Health Informatics* (2022).

[175] Amirhossein Reisizadeh, Aryan Mokhtari, Hamed Hassani, Ali Jadbabaie, and Ramtin Pedarsani. 2020. FedPAQ: A communication-efficient federated learning method with periodic averaging and quantization. In *Proceedings of the International Conference on Artificial Intelligence and Statistics*. PMLR, 2021–2031.

[176] Valerian Rey, Pedro Miguel Sánchez Sánchez, Alberto Huertas Celdrán, and Gérôme Bovet. 2022. Federated learning for malware detection in IoT devices. *Computer Networks* 204 (2022), 108693.
[177] Markus Ring, Sarah Wunderlich, Dominik Grüdl, Dieter Landes, and Andreas Hotho. 2017. Creation of flow-based data sets for intrusion detection. *Journal of Information Warfare* 16, 4 (2017), 41–54.
[178] Markus Ring, Sarah Wunderlich, Dominik Grüdl, Dieter Landes, and Andreas Hotho. 2017. Flow-based benchmark data sets for intrusion detection. In *Proceedings of the 16th European Conference on Cyber Warfare and Security. ACPI.* 361–369.
[179] Markus Ring, Sarah Wunderlich, Deniz Scheuring, Dieter Landes, and Andreas Hotho. 2019. A survey of network-based intrusion detection data sets. *Computers & Security* 86 (2019), 147–167.
[180] Royi Ronen, Marian Radu, Corina Feuerstein, Elad Yom-Tov, and Mansour Ahmadi. 2018. Microsoft malware classification challenge. arXiv:1802.10135. Retrieved from https://arxiv.org/abs/1802.10135
[181] Pedro Ruzafa-Alcazar, Pablo Fernandez-Saura, Enrique Marmol-Campos, Aurora Gonzalez-Vidal, Jose Luis Hernandez Ramos, Jorge Bernal, and Antonio F. Skarmeta. 2021. Intrusion detection based on privacy-preserving federated learning for the industrial IoT. *IEEE Transactions on Industrial Informatics* (2021).
[182] Hassan Saadat, Abdulla Aboumadi, Amr Mohamed, Aiman Erbad, and Mohsen Guizani. 2021. Hierarchical federated learning for collaborative IDS in IoT applications. In *Proceedings of the 2021 10th Mediterranean Conference on Embedded Computing (MECO)*. IEEE, 1–6.
[183] Muhammad Sami. 2019. Intrusion Detection in CAN bus. DOI: 10.21227/24m9-a446. Retrieved January 30, 2025 from https://ieee-dataport.org/documents/intrusion-detection-can-bus
[184] Ramon Sanchez-Iborra and Antonio F. Skarmeta. 2020. Tinyml-enabled frugal smart objects: Challenges and opportunities. *IEEE Circuits and Systems Magazine* 20, 3 (2020), 4–18.
[185] Mohanad Sarhan, Siamak Layeghy, Nour Moustafa, and Marius Portmann. 2023. Cyber threat intelligence sharing scheme based on federated learning for network intrusion detection. *Journal of Network and Systems Management* 31, 1 (2023), Article 3.
[186] Matthias Schonlau, William DuMouchel, Wen-Hua Ju, Alan F. Karr, Martin Theus, and Yehuda Vardi. 2001. Computer intrusion: Detecting masquerades. *Statistical Science* 16, 1 (2001), 58–74.
[187] Sebastian Garcia, Agustin Parmisano, and Maria Jose Erquiaga. 2020. IoT-23: A labeled dataset with malicious and benign IoT network traffic (Version 1.0.0) [Data set]. Zenodo. DOI: 10.5281/zenodo.4743746. Retrieved January 30, 2025 from https://zenodo.org/records/4743746
[188] Osama Shahid, Viraaji Mothukuri, Seyedamin Pouriyeh, Reza M. Parizi, and Hossain Shahriar. 2021. Detecting network attacks using federated learning for IoT devices. In *Proceedings of the 2021 IEEE 29th International Conference on Network Protocols (ICNP)*. IEEE, 1–6.
[189] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. 2018. Toward generating a new intrusion detection dataset and intrusion traffic characterization. *ICISSp* 1 (2018), 108–116.
[190] Iman Sharafaldin, Arash Habibi Lashkari, Saqib Hakak, and Ali A. Ghorbani. 2019. Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy. In *Proceedings of the 2019 International Carnahan Conference on Security Technology (ICCST)*. IEEE, 1–8.
[191] Rohini Sharma, R. K. Singla, and Ajay Guleria. 2018. A new labeled flow-based DNS dataset for anomaly detection: PUF dataset. *Procedia Computer Science* 132 (2018), 1458–1466.
[192] Virat Shejwalkar and Amir Houmansadr. 2021. Manipulating the byzantine: Optimizing model poisoning attacks and defenses for federated learning. In *Proceedings of the Network and Distributed Systems Security (NDSS) Symposium.*
[193] Jibo Shi, Bin Ge, Yang Liu, Yu Yan, and Shuang Li. 2021. Data privacy security guaranteed network intrusion detection system based on federated learning. In *IEEE INFOCOM 2021-Proceedings of the IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*. IEEE, 1–6.
[194] Ali Shiravi, Hadi Shiravi, Mahbod Tavallaee, and Ali A. Ghorbani. 2012. Toward developing a systematic approach to generate benchmark datasets for intrusion detection. *Computers & Security* 31, 3 (2012), 357–374.
[195] Sanket Shukla, Gaurav Kolhe, Houman Homayoun, Setareh Rafatirad, and Sai Manoj PD. 2022. RAFeL-robust and Data-aware federated learning-inspired malware detection in internet-of-things (IoT) networks. In *Proceedings of the Great Lakes Symposium on VLSI 2022*. 153–157.
[196] Sanket Shukla, PD Sai Manoj, Gaurav Kolhe, and Setareh Rafatirad. 2021. On-device malware detection using performance-aware and robust collaborative learning. In *Proceedings of the 2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 967–972.
[197] Abhishek Singh, Praneeth Vepakomma, Otkrist Gupta, and Ramesh Raskar. 2019. Detailed comparison of communication efficiency of split learning and federated learning. arXiv:1909.09145. Retrieved from https://arxiv.org/abs/1909.09145
[198] Narendra Singh, Harsh Kasyap, and Somanath Tripathy. 2020. Collaborative learning based effective malware detection system. In *Proceedings of the Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. Springer, 205–219.

[199] Parminder Singh, Gurjot Singh Gaba, Avinash Kaur, Mustapha Hedabou, and Andrei Gurtov. 2022. Dew-cloud-based hierarchical federated learning for intrusion detection in IoMT. *IEEE Journal of Biomedical and Health Informatics* (2022).

[200] Jinhyun So, Başak Güler, and A. Salman Avestimehr. 2021. Turbo-aggregate: Breaking the quadratic aggregation barrier in secure federated learning. *IEEE Journal on Selected Areas in Information Theory* 2, 1 (2021), 479–489.

[201] Hyun Min Song, Jiyoung Woo, and Huy Kang Kim. 2020. In-vehicle network intrusion detection using deep convolutional neural network. *Vehicular Communications* 21 (2020), 100198.

[202] Dingling Su and Zehui Qu. 2022. Detection DDoS of attacks based on federated learning with digital twin network. In *Proceedings of the International Conference on Knowledge Science, Engineering and Management*. Springer, 153–164.

[203] Xin Sun, Zhijun Tang, Mengxuan Du, Chaoping Deng, Wenbin Lin, Jinshan Chen, Qi Qi, and Haifeng Zheng. 2022. A hierarchical federated learning-based intrusion detection system for 5G smart grids. *Electronics* 11, 16 (2022), 2627.

[204] Yuwei Sun, Hiroshi Esaki, and Hideya Ochiai. 2020. Adaptive intrusion detection in the networking of large-scale LANs with segmented federated learning. *IEEE Open Journal of the Communications Society* 2 (2020), 102–112.

[205] Yuwei Sun, Hideya Ochiai, and Hiroshi Esaki. 2020. Intrusion detection with segmented federated learning for large-scale multiple LANs. In *Proceedings of the 2020 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 1–8.

[206] Aliya Tabassum, Aiman Erbad, Wadha Lebda, Amr Mohamed, and Mohsen Guizani. 2022. FEDGAN-IDS: Privacy-preserving IDS using GAN and federated learning. *Computer Communications* 192 (2022), 299–310.

[207] Mariarosaria Taddeo, Tom McCutcheon, and Luciano Floridi. 2019. Trusting artificial intelligence in cybersecurity is a double-edged sword. *Nature Machine Intelligence* 1, 12 (2019), 557–560.

[208] Rahim Taheri, Mohammad Shojafar, Mamoun Alazab, and Rahim Tafazolli. 2020. FED-IIoT: A robust federated malware detection architecture in industrial IoT. *IEEE Transactions on Industrial Informatics* 17, 12 (2020), 8442–8452.

[209] Bushra Tahir, Alireza Jolfaei, and Muhammad Tariq. 2021. Experience driven attack design and federated learning based intrusion detection in industry 4.0. *IEEE Transactions on Industrial Informatics* (2021).

[210] Zhongyun Tang, Haiyang Hu, and Chonghuan Xu. 2022. A federated learning method for network intrusion detection. *Concurrency and Computation: Practice and Experience* 34, 10 (2022), e6812.

[211] Asadullah Tariq, Mohamed Adel Serhani, Farag Sallabi, Tariq Qayyum, Ezedin S. Barka, and Khaled A. Shuaib. 2023. Trustworthy federated learning: A survey. arXiv:2305.11537. Retrieved from https://arxiv.org/abs/2305.11537

[212] Mahbod Tavallaee, Ebrahim Bagheri, Wei Lu, and Ali A. Ghorbani. 2009. A detailed analysis of the KDD CUP 99 data set. In *Proceedings of the 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications*. IEEE, 1–6.

[213] Marcio Andrey Teixeira, Tara Salman, Maede Zolanvari, Raj Jain, Nader Meskin, and Mohammed Samaka. 2018. SCADA system testbed for cybersecurity research using machine learning approach. *Future Internet* 10, 8 (2018), 76.

[214] Melissa J. M. Turcotte, Alexander D. Kent, and Curtis Hash. 2019. Unified host and network data set. In *Data Science for Cyber-Security*. World Scientific, 1–22.

[215] Imtiaz Ullah and Qusay H. Mahmoud. 2020. A scheme for generating a dataset for anomalous activity detection in IoT networks. In *Proceedings of the Canadian Conference on AI*. 508–520.

[216] Aashma Uprety, Danda B. Rawat, and Jiang Li. 2021. Privacy preserving misbehavior detection in IoV using federated machine learning. In *Proceedings of the 2021 IEEE 18th Annual Consumer Communications & Networking Conference (CCNC)*. IEEE, 1–6.

[217] Ivan Vaccari, Giovanni Chiola, Maurizio Aiello, Maurizio Mongelli, and Enrico Cambiaso. 2020. MQTTset, a new dataset for machine learning techniques on MQTT. *Sensors* 20, 22 (2020), 6578.

[218] Hado Van Hasselt, Arthur Guez, and David Silver. 2016. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 30.

[219] Madumitha Venkatasubramanian, Arash Habibi Lashkari, and Saqib Hakak. 2023. IoT malware analysis using federated learning: A comprehensive survey. *IEEE Access* (2023).

[220] Eduardo K. Viegas, Altair O. Santin, and Luiz S. Oliveira. 2017. Toward a reliable anomaly-based intrusion detection in real-world environments. *Computer Networks* 127 (2017), 200–216.

[221] VirusShare. [n.d.]. VirusShare Dataset. Retrieved January 30, 2025 from https://www.impactcybertrust.org/dataset_view?idDataset=1271

[222] Nguyen Chi Vy, Nguyen Huu Quyen, Van-Hau Pham, et al. 2021. Federated learning-based intrusion detection in the context of IIoT networks: Poisoning attack and defense. In *Proceedings of the International Conference on Network and System Security*. Springer, 131–147.

[223] Omar Abdel Wahab, Azzam Mourad, Hadi Otrok, and Tarik Taleb. 2021. Federated machine learning: Survey, multi-level classification, desirable criteria and future directions in communication and networking systems. *IEEE Communications Surveys & Tutorials* 23, 2 (2021), 1342–1397.

[224] Hongyi Wang, Mikhail Yurochkin, Yuekai Sun, Dimitris Papailiopoulos, and Yasaman Khazaeni. 2020. Federated learning with matched averaging. arXiv:2002.06440. Retrieved from https://arxiv.org/abs/2002.06440

[225] Ning Wang, Yimin Chen, Yang Hu, Wenjing Lou, and Y. Thomas Hou. 2022. Feco: Boosting intrusion detection capability in IoT networks via contrastive learning. In *IEEE INFOCOM 2022-Proceedings of the IEEE Conference on Computer Communications*. IEEE, 1409–1418.

[226] Yunkai Wei, Sipei Zhou, Supeng Leng, Sabita Maharjan, and Yan Zhang. 2021. Federated learning empowered end-edge-cloud cooperation for 5G HetNet security. *IEEE Network* 35, 2 (2021), 88–94.

[227] Jens Wettlaufer. 2021. Property Inference-based federated learning groups for collaborative network anomaly detection. *Electronic Communications of the EASST* 80 (2021).

[228] Jason Whelan, Thanigajan Sangarapillai, Omar Minawi, Abdulaziz Almehmadi, and Khalil El-Khatib. 2020. UAV Attack Dataset. *IEEE Dataport*. DOI : 10.21227/00dg-0d12. Retrieved January 30, 2025 from https://ieee-dataport.org/open-access/uav-attack-dataset

[229] Dongmin Wu, Yi Deng, and Mingyong Li. 2022. FL-MGVN: Federated learning for anomaly detection using mixed Gaussian variational self-encoding network. *Information Processing & Management* 59, 2 (2022), 102839.

[230] Wentai Wu, Ligang He, Weiwei Lin, Rui Mao, Carsten Maple, and Stephen Jarvis. 2020. SAFA: A semi-asynchronous protocol for fast federated learning with low overhead. *IEEE Transactions on Computers* 70, 5 (2020), 655–668.

[231] Qi Xia, Winson Ye, Zeyi Tao, Jindi Wu, and Qun Li. 2021. A survey of federated learning for edge computing: Research problems and solutions. *High-Confidence Computing* 1, 1 (2021), 100008.

[232] Zhuoqun Xia, Yaling Chen, Bo Yin, Haolan Liang, Hongmei Zhou, Ke Gu, and Fei Yu. 2022. Fed_ADBN: An efficient intrusion detection framework based on client selection in AMI network. *Expert Systems* (2022).

[233] Bin Xie, Xinyu Dong, and Changguang Wang. 2021. An Improved-means clustering intrusion detection algorithm for wireless networks based on federated learning. *Wireless Communications and Mobile Computing* 2021, 1 (2021), 9322368.

[234] Krishna Yadav and B. B. Gupta. 2021. Clustering based rewarding algorithm to detect adversaries in federated machine learning based IoT environment. In *Proceedings of the 2021 IEEE International Conference on Consumer Electronics (ICCE)*. IEEE, 1–6.

[235] Krishna Yadav, Brij B. Gupta, Ching-Hsein Hsu, and Kwok Tai Chui. 2021. Unsupervised federated learning based IoT intrusion detection. In *Proceedings of the 2021 IEEE 10th Global Conference on Consumer Electronics (GCCE)*. IEEE, 298–301.

[236] Qiang Yang, Yang Liu, Tianjian Chen, and Yongxin Tong. 2019. Federated machine learning: Concept and applications. *ACM Transactions on Intelligent Systems and Technology (TIST)* 10, 2 (2019), 1–19.

[237] Xuelan Yang, Jiechen Luo, and Muamer N. Mohammed. 2022. Federation learning of optimized convolutional neural network structure for intrusion detection. In *Proceedings of the 2022 2nd International Conference on Advances in Electrical, Computing, Communication and Sustainable Technologies (ICAECT)*. IEEE, 1–7.

[238] Dong Yin, Yudong Chen, Ramchandran Kannan, and Peter Bartlett. 2018. Byzantine-robust distributed learning: Towards optimal statistical rates. In *Proceedings of the International Conference on Machine Learning*. PMLR, 5650–5659.

[239] Xuefei Yin, Yanming Zhu, and Jiankun Hu. 2021. A comprehensive survey of privacy-preserving federated learning: A taxonomy, review, and future directions. *ACM Computing Surveys (CSUR)* 54, 6 (2021), 1–36.

[240] Pengqian Yu, Achintya Kundu, Laura Wynter, and Shiau Hong Lim. 2021. Fed+: A unified approach to robust personalized federated learning. arXiv:2009.06303 [cs.LG]. Retrieved from https://arxiv.org/abs/2009.06303

[241] Li Yuan, Zhaoyang Wang, Lifeng Sun, Philip S. Yu, and Christopher G. Brinton. 2024. Decentralized federated learning: A survey and perspective. *IEEE Internet of Things Journal* 11, 21 (2024), 20627–20643. https://doi.org/10.1109/JIOT.2024.3360265

[242] Shuai Yuan, Hongwei Li, Rui Zhang, Meng Hao, Yiran Li, and Rongxing Lu. 2021. Towards lightweight and efficient distributed intrusion detection framework. In *Proceedings of the 2021 IEEE Global Communications Conference (GLOBECOM)*. IEEE, 1–6.

[243] Idris Zakariyya, Harsha Kalutarage, and M. Omar Al-Kadri. 2021. Memory efficient federated deep learning for intrusion detection in IoT networks. CEUR Workshop Proceedings.

[244] Chen Zhang, Yu Xie, Hang Bai, Bin Yu, Weihong Li, and Yuan Gao. 2021. A survey on federated learning. *Knowledge-Based Systems* 216 (2021), 106775.

[245] Chongsheng Zhang, Yuan Zhang, Xianjin Shi, George Almpanidis, Gaojuan Fan, and Xiajiong Shen. 2019. On incremental learning for gradient boosting decision trees. *Neural Processing Letters* 50 (2019), 957–987.

[246] Hao Zhang, Junwei Ye, Wei Huang, Ximeng Liu, and Jason Gu. 2024. Survey of federated learning in intrusion detection. *Journal of Parallel and Distributed Computing* 195 (2024), 104976.

[247] Jiachao Zhang, Peiran Yu, Le Qi, Song Liu, Haiyu Zhang, and Jianzhong Zhang. 2021. FLDDoS: DDoS attack detection model based on federated learning. In *Proceedings of the 2021 IEEE 20th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. IEEE, 635–642.

[248] Yiying Zhang, Yiyang Liu, Nan Zhang, Delong Wang, Suxiang Zhang, and Yannian Wu. 2022. Grid false data intrusion detection method based on edge computing and federated learning. In *3D Imaging—Multidimensional Signal Processing and Deep Learning*, Lakhmi C. Jain, Roumen Kountchev, Yonghang Tai, and Roumiana Kountcheva (Eds.). Springer, 179–188.

[249] Yu Zhang and Qiang Yang. 2021. A survey on multi-task learning. *IEEE Transactions on Knowledge and Data Engineering* 34, 12 (2021), 5586–5609.

[250] Zhao Zhang, Yong Zhang, Da Guo, Lei Yao, and Zhao Li. 2022. SecFedNIDS: Robust defense for poisoning attack against federated learning-based network intrusion detection system. *Future Generation Computer Systems* 134 (2022), 154–169.

[251] Liang Zhao, Jiaming Li, Qi Li, and Fangyu Li. 2021. A federated learning framework for detecting false data injection attacks in solar farms. *IEEE Transactions on Power Electronics* 37, 3 (2021), 2496–2501.

[252] Ruijie Zhao, Yijun Wang, Zhi Xue, Tomoaki Ohtsuki, Bamidele Adebisi, and Guan Gui. 2022. Semi-supervised federated learning based intrusion detection method for internet of things. *IEEE Internet of Things Journal* (2022).

[253] Ruijie Zhao, Yue Yin, Yong Shi, and Zhi Xue. 2020. Intelligent intrusion detection based on federated learning aided long short-term memory. *Physical Communication* 42 (2020), 101157.

[254] Ying Zhao, Junjun Chen, Qianling Guo, Jian Teng, and Di Wu. 2020. Network anomaly detection using federated learning and transfer learning. In *Proceedings of the International Conference on Security and Privacy in Digital Economy*. Springer, 219–231.

[255] Ying Zhao, Junjun Chen, Di Wu, Jian Teng, and Shui Yu. 2019. Multi-task network anomaly detection using federated learning. In *Proceedings of the 10th International Symposium on Information and Communication Technology*. 273–279.

[256] Peng Zhou. 2020. Federated deep payload classification for industrial internet with cloud-edge architecture. In *Proceedings of the 2020 16th International Conference on Mobility, Sensing and Networking (MSN)*. IEEE, 228–235.

[257] Yajin Zhou and Xuxian Jiang. 2012. Dissecting android malware: Characterization and evolution. In *Proceedings of the 2012 IEEE Symposium on Security and Privacy*. IEEE, 95–109.

[258] Hangyu Zhu, Jinjin Xu, Shiqing Liu, and Yaochu Jin. 2021. Federated learning on non-IID data: A survey. *Neurocomputing* 465 (2021), 371–390.

[259] Juncen Zhu, Jiannong Cao, Divya Saxena, Shan Jiang, and Houda Ferradi. 2023. Blockchain-empowered federated learning: Challenges, solutions, and future directions. *ACM Computing Surveys* 55, 11 (2023), 1–31.

[260] Maede Zolanvari, Marcio A. Teixeira, Lav Gupta, Khaled M. Khan, and Raj Jain. 2019. Machine learning-based network vulnerability analysis of industrial internet of things. *IEEE Internet of Things Journal* 6, 4 (2019), 6822–6834.

Appendices

A Supplemental Content for Introduction Section

Table 1. Relationship of the Present Work with Existing Surveys on FL

Reference	Year	Contributions	Relationship with our work	Citations in Scopus (as of 06/02/2025)
[236]	2019	Categorization of FL architectures considering potential applications and related techniques	The proposed definitions and categorization are considered in our taxonomy in subsection 2.3 to classify FL-enabled IDSs	4268
[120]	2020	Analysis of FL around communication, resource allocation and security/privacy, including applications and challenges for multi-access edge computing (MEC) settings	The described challenges are considered in the scope of FL-enabled IDS. Cyberattack detection is regarded as one of the main FL applications that is the central topic of our work	1613
[14]	2020	Analysis of technologies and potential FL applications, including associated challenges	FL implementations are also considered in our analysis in Section F.2 alongside major challenges in the context of FL-enabled IDS	478
[115]	2020	Analysis of FL challenges around security and privacy, and description of potential applications	With reference to Section 6, some of the described challenges are considered in our work, particularly through the prism of FL-enabled IDS	781
[8]	2020	Analysis of FL challenges according to a proposed classification and definition of a taxonomy for FL applications	Some of these challenges are examined in our work, but in the scope of FL-enabled IDS. Moreover, in Section 2.3, the present work covers a number of taxonomy aspects, including aggregation functions and datasets	488
[223]	2021	Description of main FL aspects, including aggregation functions and analysis of existing literature in respect to five main FL challenges	Aggregation functions are also analyzed in our work and used in the proposed taxonomy in Section 2.3. The described challenges are considered in Section 6 from an IDS viewpoint	377
[244]	2021	Analysis of FL literature according to data partitioning, privacy mechanism, ML model, communication architecture, and system heterogeneity	All these aspects have been taken into account for the definition of (a) the proposed taxonomy in Section 2.3 and (b) the challenges associated with FL-enabled IDSs in Section 6	924
[152]	2021	Analysis of the use cases and applications of FL for IoT, as well as the main challenges and research directions	Our work concentrates on the application of FL for IDS, which is considered as a potential use-case. Additionally, in Section 2.3, we reckon with key research challenges and their impact on FL-enabled IDS approaches	737
[231]	2021	Analysis on the application of FL to edge computing by considering security/privacy, efficiency, and migration scheduling	In the current work, security/privacy and efficiency aspects are addressed in Section 6	167
[148]	2021	A comprehensive overview around security and privacy in FL, and description of the main technologies in this domain	While security and privacy aspects are examined as challenges in our work, in Sections 5 and F.2, we additionally consider some of the aggregation functions and FL implementations in our analysis for FL-enabled IDS	830
[31]	2021	Analysis of security and privacy aspects affecting FL setting, including the description of potential attacks and countermeasures, as well as an evaluation of such methods	The described challenges and mitigation techniques are considered in our work in Section 6 regarding their relationship with FL-enabled IDS	86
[94]	2021	Comprehensive analysis on the use of FL for IoT scenarios, including a proposed taxonomy, and open research challenges	While it is focused on IoT settings, some of the described challenges, e.g., those related to security and data heterogeneity, and taxonomy aspects are also considered in our work in the scope of FL-enabled IDSs in Section 2.3	493
[239]	2021	Comprehensive analysis of privacy concerns and mitigation techniques for FL, including a proposed taxonomy around those aspects	With reference to Section 6, privacy aspects are considered as one of the main challenges around the development of FL-enabled IDSs	362
[83]	2022	Analysis of the main challenges of FL when considering its application on IoT constrained scenarios, as well as potential solutions	Some of the described challenges are also taken into account in our work in Section 6 regarding their impact on the development of FL-enabled IDSs	419
[153]	2022	Analysis of the requirements of using FL in smart healthcare, and overview of the main applications, as well as the main challenges and research directions	With reference to Section 6 the description of several challenges around FL in this context, such as communication issues, or security/privacy aspects are considered in our work with reference to the application of FL for IDSs	387
[124]	2022	Description of the evolution of distributed ML and, in particular, FL architectures and techniques	Our work also provides a description of existing FL implementations (given in Section F.2), and research directions, e.g., around data heterogeneity, that are considered in Section 6 for FL-enabled IDSs	174
[259]	2023	Analysis of blockchain as enabling technology in FL settings, as well as description of challenges and future directions	The application of blockchain in FL scenarios is addressed as a potential approach to mitigate the issues associated with typical FL settings where the aggregator could become a bottleneck (see Section 6.3)	118
[84]	2023	Analysis of blockchain technology to address some of the main security and privacy challenges related to FL in IoT scenarios	Security and privacy challenges in FL (and FL-enabled IDS in particular) are analyzed in Section 6. Furthermore our proposed taxonomy in Section 2.3 considers some of the aspects of this work, such as data partitioning and aggregation function	151
[211]	2023	Description of the main principles of trustworthy FL, including security/privacy, fairness and interpretability	Our work also considers some of the main aspects related to security and privacy in the context of FL-enabled IDS in Section 6	Not in Scopus

Table 2. Comparison of This Work Based on Seven Criteria with Existing Surveys on FL-Enabled IDS

Reference	Year	Taxonomy	ML models	Datasets	Implementations	Aggregation functions	Challenges	Systematic	Analyzed works
[10]	2021	✗	✗	✗	✗	✗	✓	✗	15
[57]	2021	✗	Partially	Partially	✗	✗	✓	✗	14
[32]	2022	✗	Partially	✗	Partially	✗	Partially	✗	12
[55]	2022	✗	✗	✗	✗	Partially	Partially	✗	11
[29]	2022	✗	Partially	Partially	✗	Partially	Partially	✗	18
[110]	2022	✓	Partially	✗	✗	Partially	Yes	✓	22
[22]	2022	✓	Partially	✗	✗	✗	Partially	✓	4
[219]	2023	✓	Partially	✗	✗	✗	Partially	✗	11
[246]	2024	✗	✗	Partially	✗	✗	Partially	✗	21
[97]	2024	✗	✗	✗	Partially	Partially	✓	✗	8
Our work	2025	✓	✓	✓	✓	✓	✓	✓	104

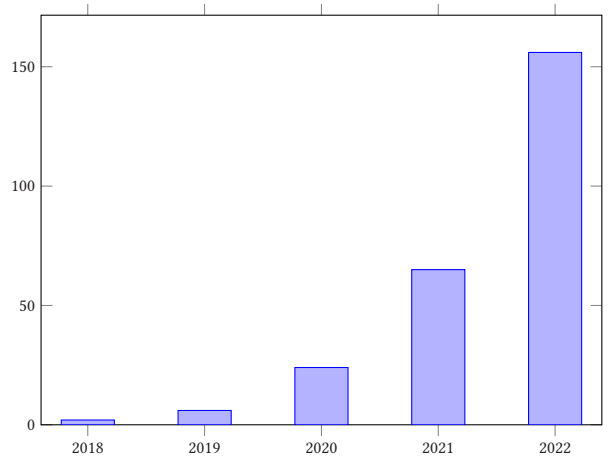


Fig. 2. Number of FL-enabled IDS publications through time.

B Supplemental Content About Background Around IDS and FL

B.1 Intrusion Detection Systems

According to NIST SP 800-94, intrusion detection refers to *"the process of monitoring the events occurring in a computer system or network and analyzing them for signs of possible incidents"*. Therefore, an IDS is usually considered as a *"software that automates the intrusion detection process"* [157]. An IDS represents a widely deployed security mechanism to identify potential malicious entities and actions targeting a certain IT or **operational technology** (**OT**) system. Although the initial development of IDSs was focused on *signature-based* approaches in which network traffic was compared with previously stored network patterns, *anomaly-based* IDSs quickly attracted significant interest, especially with the application of ML techniques [98]. That is, an anomaly-based IDS builds a normal behavior model of a certain system so that any detected deviation from that behavior is likely to be considered an intrusion. Compared to signature-based IDSs, one of the main advantages of anomaly-based approaches is that they can detect previously unseen attacks. Additionally, an IDS can also be classified depending on the source of information for intrusion detection, for example, if the data originates from end nodes or network traffic. However, in practice, it is becoming commonplace to use hybrid approaches where information from both the network and end nodes is used to identify possible intrusions related to network and application layers.

The application of ML in the development of anomaly-based IDS has represented a hot topic in cybersecurity research by using both supervised and unsupervised approaches [122]. In the

former case, the identification of cybersecurity attacks relies on labeled data and a set of features that characterize the information (usually network traffic) for the training process. That is, each network flow or other type of information is already marked for the training as an attack or benign traffic, depending on whether it corresponds to an intrusion or not. In the scope of IDS, several different supervised techniques have been considered, including DT, SVM, as well as different types of NN [98]. However, it is not always feasible find or create such labeled information in real-world scenarios. Therefore, the use of unsupervised approaches [34] has been proposed for identifying potential cybersecurity attacks using unlabeled data. In this direction, typical approaches based on *clustering* are employed to group samples from the same type (attack or benign traffic) in a certain cluster. Additionally, there are hybrid or semi-supervised approaches [252], in which supervised and unsupervised learning techniques are used alongside. That is, as described by [156], hybrid approaches can take advantage of the performance of supervised approaches for known attacks, as well as the ability of unsupervised approaches to detect new attacks.

The performance of the ML technique is key in the operation of an ML-enabled IDS; a poor performance could have severe consequences on the system where the IDS is deployed. While a *false negative* would represent an attack instance that is not detected by ML-enabled IDS, a *false positive* could lead the system to deploy countermeasures against a false attack, which would cause unnecessary use of resources that could be exploited by opponents to trigger a real attack (e.g., a DoS attack). Furthermore, as widely recognized in the literature [99], the time required to detect a certain attack by an IDS is crucial to reducing the eventual damage caused to the system.

The development of ML-enabled IDSs has experienced a strong interest in recent years through centralized environments where the information is shared by end nodes to build the corresponding model. As previously mentioned, this approach raises privacy concerns that need to be addressed through decentralized settings. The next subsection provides an overview about FL, which is intended to mitigate these issues for the next generation of IDSs.

B.2 Federated Learning

FL was coined in 2016 [138] as a distributed and collaborative approach to ML, in which a set of entities is trained over their local data to build a global model. Unlike traditional ML approaches, end nodes do not need to share their data with typical data centers to be further processed. Figure 3 shows an overview of the centralized ML and FL approaches. Generally, the FL training process is divided into a set of *rounds* where each party trains a global model by using its local data until a certain number of rounds or the model converges. Each party is called an *FL client* and can be represented by different entities depending on the scenario, e.g., IoT device, smartphone, or vehicle. Furthermore, this local training can be based on a wide variety of ML techniques, including supervised and unsupervised approaches. The local processing is normally called *epoch* and can be repeated several times in each training round.

In a typical FL scenario, the local updates computed by the different FL clients are then shared with a central entity called *aggregator*, which is responsible for generating a new model based on the received updates using an *aggregation function*. While the commonest function is based on averaging (FedAvg [138]), as discussed in Section 5, other more sophisticated functions have also been proposed in recent years to cope with non-IID data distributions, which are common in FL settings (see Section 6). The new model generated by the aggregator is then again shared with FL clients to start a new round of training. It should be noted that, in each round, the aggregator can select a subset of the clients to participate, considering different aspects, such as connection/device status or impact of the updates on the model's convergence.

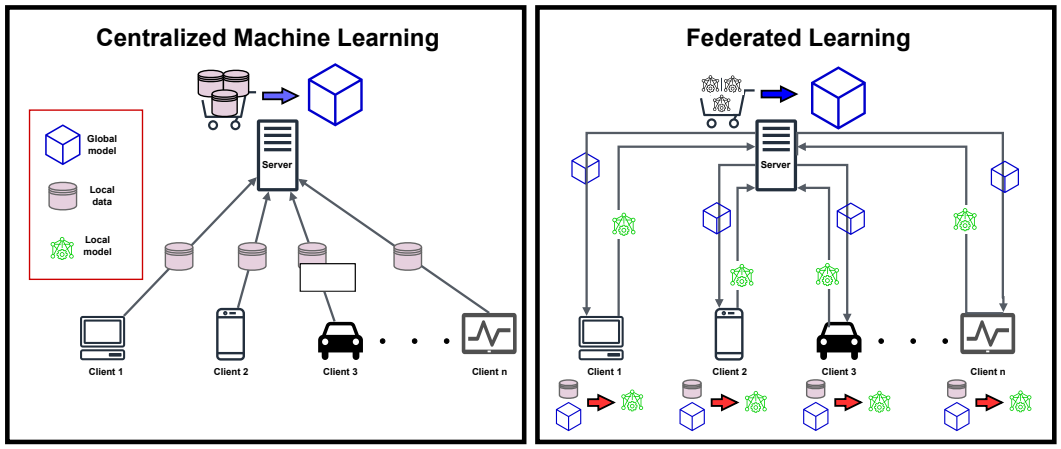


Fig. 3. A bird's-eye view of centralized ML vs. FL approach.

C Supplemental Content About IDS Datasets and Their Use in Existing FL-Enabled IDS

C.1 Internet Datasets

The **CIC-IDS2017** dataset [189] is one of the most utilized datasets for IDS purposes. It was created in an emulated networking environment and comprises five days of network traffic. This labeled dataset covers a broad range of attacks, including brute force, DoS, network infiltration, and botnets. It contains 80 features associated with the network traffic, and provides information such as timestamp, source/destination IP address and port. Furthermore, it can be downloaded in either pcap or CSV format.

The **CSE-CIC-IDS2018** dataset [189] is an extension of the CIC-IDS2017 one, that is, it contains the same methodology of exporting and labeling flow-based features, but with different attacks, including brute-force, TLS Heartbleed, botnet, DoS, DDoS, Web attacks, and network infiltration. Additionally, the network topology is bigger, i.e., the attacker controls 50 machines and the targeted organization comprises five departments, where each one contains 420 client machines and 30 servers.

The **CIDDS-001** dataset [178] capitalizes on OpenStack as a tool to generate labeled and realistic benchmark datasets for IDS. This dataset is flow-based, anonymized, labeled, and the captured network traffic stems from a realistic testbed. It contains different assaults, including DoS, brute force, and port scanning. The **CIDDS-002** dataset [177] extends CIDDS-001 [178] for specific port scanning attacks, including SYN, ACK, UDP, FIN, and Ping cases. Like its predecessor, it is provided in a flow-based, anonymized, and labeled format based on a similar deployment.

The **CIC DoS** [87] dataset contains eight different application-layer DoS attacks launched against 10 web servers. Particularly, the focus is on DoS slow-rate attacks, like the well-known Slowloris. The dataset also contains attack-free traffic obtained from the ISCXIDS2012 dataset [194], and is offered in a packet-based format with a size of 4.6 GB.

The **CIC DDoS** [190] dataset considers DDoS attacks by using different well-known protocols, such as NTP, DNS, LDAP, or SNMP. Such attacks were triggered during two days over different machines with diverse Windows OS versions, and more than 80 features were obtained through the CICFlowMeter tool. The dataset is available in both pcap and CSV format containing labeled flows.

The **PUF** dataset [191] captures flow-based traffic. It focuses on DNS-based anomalies by labeling different flows, including DNS-over-TCP, DNS zone transfer, and DNS2TCP (a tool for relaying TCP connections over DNS).

Table 3. Comparison of Contemporary Internet Datasets which Can Be Used for Assessing FL-Enabled IDS

Dataset	Year	# features	# samples (packets/flows)	Attack-/benign traffic ratio	Attacks	Data labelled?	Realistic testbed?	Division	FL-enabled IDS works using the dataset
CIC-IDS2017 [189]	2017	80	≈28M f	0.17:1	Brute force, DoS, DDoS, Heartbleed, XSS, SQL Injection, Data exfiltration, Botnet, Scan	✓	✓	IP address	[27, 40, 41, 54, 70, 106, 136, 139, 159–161, 168, 170, 171, 210, 226, 227, 235, 247, 255]
TRAbID [220]	2017	50	≈207M p	0.15:1	DoS (SYN flood, UDP flood, ICMP flood, TCP keepalive, SMTP flood, HTTP flood), Port scan, Vulnerability scan	✓	✓	N/A	-
Unified host and network [214]	2017	21	N/A	N/A	Privilege escalation	✗	✓	N/A	-
CIDDS-001 [178]	2017	14	≈32M f	0.12:1	DoS, Brute force, Port scan	✓	✓	IP address	-
CIDDS-002 [177]	2017	14	≈16M f	0.036:1	Port scan (SYN, ACK, UDP, FIN, Ping)	✓	✓	IP address	-
CSE-CIC-IDS2018 [189]	2018	80	≈4,5M f	0.58:1	DoS, Heartbleed, DoS Slowloris, DDoS, Brute-force (SSH, FTP), Scan, Injection	✓	✓	IP address	[40, 59, 167, 193, 250]
PUF [191]	2018	12	≈0.3M f	0.14:1	DNS attacks	✓	✓	N/A	-
CIC-DoS [87]	2017	2	≈4,6GB p	N/A	High-volume HTTP attacks: DoS improved GET (Goldeneye), DDoS GET(ddossim), DoS GET (hulk). Low-volume HTTP attacks: slow-send body (Slowhttptest), slow send body (RUDY]), slow-send headers (Slowhttptest), slow send headers (Slowloris), slow-read (Slowhttptest)	✓	✓	N/A	-
CIC-DDoS2019 [190]	2019	80	≈50M f	879.4:1	DDoS attacks over the following protocols: NTP (DDoS_NTP), DNS (DDoS_DNS), LDAP (DDoS_LDAP), MSSQL (DDoS_MSSQL), NetBIOS (DDoS_NetBIOS), SNMP (DDoS_SNMP), SSDP (DDoS_SSDP), UDP (DDoS_UDP), UDP-Lag, WebDDoS, SYN and TFTP	✓	✓	IP address	[48, 51, 113, 132, 171, 242, 247]
AWID3 [36]	2021	254	≈37M p	0.18:1	Deauthentication flooding, Deauthentication flooding broadcast, Disassociation flooding, Disassociation flooding broadcast, Amok, Association Request flooding, Reassociation Request flooding, Beacon flooding, Rogue AP, Channel MitM attack, Key Reinstallation (Krack), TK Reinstallation (Kr00k), SSH Brute force, Botnet, WannaCry plus (Malware), TeslaCrypt (Malware), SQL Injection, SSDP Amplification, Captive portal (Evil Twin), Eaphammer (Evil Twin), ARP spoofing, DNS spoofing	✓	✓	IP address	-
H23Q [37]	2022	200	≈10M p	0.08:1	HTTP3-flood, Fuzzing, HTTP3-loris(DDoS), HTTP3-stream, QUIC-flood(DDoS), QUIC-loris(DDoS), QUIC-enc, HTTP-smuggle, HTTP2-concurrent, HTTP2-pause	✓	✓	Device	-

The **TRAbID** dataset [220] concentrates on legacy assaults, including SYN flood, UDP flood, and ICMP flood. Ubuntu 16.04 was used for client machines and Ubuntu 14.04 for vulnerable servers. The dataset was also evaluated by the authors in terms of anomaly detection.

The **Unified host and network** dataset [214] comprises MS Windows-based event logs, which can be analysed through ML techniques. Different host-based events were captured from the log files, including the creation and use of Kerberos authentication tickets, user log in or log out events, and if a workstation was locked or not.

The **AWID3** dataset [36] contains attacks exercised over an enterprise Wi-Fi network. It considers 21 different assaults, which are grouped into 801.11-specific and higher layer ones mounted against the local network or external targets. It was created from a physical lab that realistically emulates a typical enterprise network infrastructure, and it is offered in both CSV and pcap formats.

The **H23Q** dataset [37] comprises HTTP/2, HTTP/3, and QUIC protocol traffic involving six diverse HTTP/3-enabled servers residing on Azure cloud infrastructure. Ten different attacks were recorded, including DDoS and Slowloris-alike ones. The testbed also included 12 legitimate clients

who were accessing these servers from different public networks. Packet capture took place on each server, therefore each of them can be considered as a client under an FL setting.

C.2 IoT Datasets

The **N-BaIoT** dataset [140] focuses on botnet attacks on IoT networks. Specifically, it simulates two different botnets, namely, Bashlite and Mirai. Both these botnets are capable of launching TCP and UDP flooding attacks, along with other ones, including port scanning. The dataset was evaluated by the authors by means of AE models.

The **Kitsune** dataset [142] was obtained based on the traffic provided by a video surveillance network with eight IP cameras, as well as an IoT deployment comprising nine devices, including a thermostat, a baby monitor, and two doorbells. More than 100 features were identified, and the attacks include MiTM, reconnaissance, and DoS. The dataset is available in both pcap and CSV formats.

The **Bot-IoT** dataset [102] also targets botnet attacks against IoT networks. It contains mostly DoS and DDoS assaults based on network flow features originating from each participating device. In this respect, the dataset focuses on IoT-specific protocols like MQTT having several devices residing on **Amazon Web Services** (**AWS**).

The **WUSTL-IIoT** dataset [260] concentrates on an **industrial IoT** (**IIoT**) scenario considering 41 features to characterize network traffic and different attacks related to command injection, DoS, reconnaissance, and backdoor traffic. The data was obtained over 53 hours by using a testbed in an industrial plant with several IIoT systems.

The **IoT network intrusion dataset** [90] contains various types of network attacks against an IoT infrastructure. For capturing the traffic, two typical smart home devices were used, namely, SKT NUGU (NU 100) and EZVIZ Wi-Fi Camera. The dataset comprises 42 raw network packet files (pcap) collected at different time points. The packet files were captured by using the monitor mode of the wireless network adapter. This dataset contains zero wireless headers as these were removed through the Aircrack-ng tool. The **IoTID20** [215] is built from the previous dataset by applying CICFlowMeter to extract 83 features from the network traffic, so that it is released in CSV format.

The **IoT-23** dataset [187] is released through 20 malware captures and three captures related to benign network traffic. In particular, malicious traffic was obtained by using a Raspberry Pi device, while benign traffic was associated to three different IoT devices. The attacks in this dataset include traffic related to different botnets, such as Mirai and Okiru, as well as port scanning and DDoS.

The **IoT DoS and DDoS Attack Dataset** [80] was created by applying data preprocessing techniques over the CIC DDoS dataset (described in subsection C.1). Although the original dataset was in CSV format, to efficiently utilize the potential of CNNs for DoS and DDoS attack detection, the network traffic data has been converted to an image format.

The **MQTT-IoT-IDS2020** dataset [75] is derived from a simulated MQTT-based network. The simulated network comprised 12 sensors, a broker, a simulated camera, and an attacker. Five scenarios were recorded in an equal number of pcap files: normal operation, aggressive scan, UDP scan, Sparta SSH brute-force, and MQTT brute-force attack. Three types of features were extracted from the raw pcap files: packet features, unidirectional flow features, and bidirectional flow features. The dataset is also available in CSV format.

The **MQTTset** dataset [217] is another dataset focused on the MQTT protocol created from the traffic generated by different IoT devices, such as door locks, temperature sensors, motion sensors, and fan speed controllers. It uses 33 features and provides data from different attacks, including flooding DoS, MQTT publish flood, and malformed data.

The **IoT Healthcare Security Dataset** [79] was generated with the assist of IoT-Flock, an open-source tool for IoT traffic generation. This dataset contains one use-case corresponding to an

IoT-based intensive care unit with the capacity of two beds, where each bed is equipped with nine patient monitoring devices (sensors) and one control unit called Bedx-Control-Unit. The dataset includes attacks related to MQTT and CoAP protocols.

The **CCD-INID-V1** dataset [127] was compiled and assessed on a hybrid setting, that is, a combination of fog and cloud computing. Specifically, authors generated features from the fog layer and utilized training and testing phases at the cloud layer. The data was collected in smart lab and smart home environments using Rainbow HAT sensor boards installed on Raspberry Pi. The dataset was evaluated by means of legacy ML algorithms, including KNN, NB, LR, and SVM.

The **X-IIoTID** [11] dataset captures recent attacker behavior targeted against industrial control loops devices, and edge, mobile, and cloud network domains. It contains device-agnostic features, making it suitable for IIoT systems. The dataset has more than 820K records and 68 features. These features were extracted from various sources, including network traffic, system logs, application logs, device resources, and commercial IDS logs.

The **Edge-IIoTset** dataset [56] can be used by ML-based IDS in two different modes, namely, centralized and FL. The IoT data were generated from an assortment of IoT devices, such as low-cost digital sensors, ultrasonic sensors, and others. The authors analyzed 14 attacks related to IoT and IIoT protocols, which are categorized into five categories, namely DoS/DDoS, information gathering, MitM, injection, and malware. Specifically, the testbed comprised seven layers, i.e., cloud computing, network functions virtualization, a blockchain network, fog computing, SDN, edge computing, and IoT/IIoT perception.

The **LATAM-DDoS-IoT dataset** [17] is a dataset centers on DoS and DDoS attacks against IoT devices. In particular, it contains 20 features and was created by using a production network with physical IoT devices, such as smart light bulbs and voice assistants.

The **CIC IoT dataset** [45] was recently created by using 81 IoT devices, such as cameras, smart lamps, speakers, doorbells, motion sensors, or weather stations which communicate through Wi-Fi, Zigbee, and Z-Wave technologies. While the dataset is focused on device identification/profiling aspects, the authors carried out six different experiments, including the simulation of DoS and RTSP brute force attacks.

The **Ton_IoT dataset** [19] was obtained through a deployment combining fog/edge and cloud components to simulate an IoT/IIoT production environment, considering different types of sensors. It provides network traffic information, as well as data related to sensor readings and telemetry using a set of 83 features. The data contains information regarding different types of attacks, including DoS/DDoS, ransomware, and XSS. Different parts of the dataset (e.g., related to network traffic or sensor readins) can be downloaded in CSV and pcap formats.

The **MedBIoT** dataset [66] is focused on IoT botnets, including Mirai, BashLite, and Torii. It was obtained through a network comprised of 83 real and emulated devices, such as locks, switches, fans, and lights. Authors used real malware, considering the mentioned botnets. The dataset employs 100 features to characterize network traffic and can be downloaded in the form of pcap files.

C.3 Other Datasets

The **InSDN** dataset [52] considers SDN scenarios by using 83 features to characterize network traffic in such environments. It contains information about different attacks, such as DoS, web attacks, and exploitation that was obtained through several tools, including Metasploit and Nmap.

The **OTIDS** IDS dataset [111] released back in 2017 targets **in-vehicle networking (IVN)**, and specifically the CAN bus. Along with CAN normal traffic, it contains three attack categories, namely, DoS, fuzzy, and impersonation. The two datasets of normal traffic have a total size of around 354 MB (≈4.6M messages), whereas the sizes of the three attack datasets are 60 MB (DoS), 50 MB (Fuzzy), and 84 MB (impersonation). The dataset was extracted from a KIA SOUL car by

Table 4. Comparison of Contemporary IoT Datasets which Can Be Used for Assessing FL-Enabled IDS

Dataset	Year	# features	# samples (packets/flows)	Attack-/benign traffic ratio	Attacks	Data labelled?	Realistic testbed?	Division	FL-enabled IDS works using the dataset
N-BaIoT [140]	2018	115	≈7M p	11.7:1	Mirai Bot, BashLite Bot	✓	✓	Device	[95, 166, 176, 243, 252]
Kitsune [142]	2018	115	≈21M p	0.9:1	Reconnaissance, MitM, DoS, and botnet	✓	✓	IP address	[222, 243]
Bot-IoT [102]	2019	46	≈73M p	9725.7:1	PortScan, OS Fingerprinting, DoS/DDoS, data theft, keylogging	✓	✓	IP address	[78, 166, 167]
WUSTL-IIoT [260]	2019	41	≈1.2M p	0.08:1	command injection, DoS, reconnaissance and backdoor	✓	✓	IP address	-
IoT network intrusion dataset [90]	2019	N/A	≈1.3M p	9:1	Mirai ACK flooding, Mirai brut force, Mirai HTTP flooding, Mirai UDP flooding, MitM, portscan, OS Fingerprinting	✗	✓	N/A	-
IoTID20 [215]	2020	83	625,784 p	14.6:1	Mirai ACK flooding, Mirai brut force, Mirai HTTP flooding, Mirai UDP flooding, MitM, portscan, OS Fingerprinting	✓	✓	IP address	-
IoT-23 [187]	2020	21	≈325M f	9.5:1	PortScan, Okiru botnet, DDoS, C&C, C&C-HeartBeat, C&C-File download, C&C-Heart-Beat-FileDownload, C&C-Tori, C&C-Mirai	✓	✗	IP address	-
MQTT-IoT-IDS2020 [75]	2020	44	≈22M p	8.7:1	Aggressive scan, UDP scan, Sparta SSH brute force, and MQTT brute-force attack	✓	✓	IP address	[26]
MQTTset [217]	2020	33	≈12M p	0.014:1	Flooding DoS, MQTT Publish flood, SlowITe, malformed data, brute force authentication	✓	✗	IP address	[59]
MedBIoT [66]	2020	100	≈18M p	0.42:1	Mirai botnet, BashLite botnet, Torii botnet	✓	✓	IP address	-
IoT Healthcare Security Dataset [79]	2021	10	56,609 p	0.7:1	MQTT Publish Flood, MQTT Authentication Bypass Attack, MQTT Packet Crafting Attack, and COAP Replay Attack	✓	✓	N/A	-
ToN_IoT [19]	2021	83	≈5.3M p	0.88:1	Backdoor, DoS, DDoS, Injection, MITM, Password, Ransomware, Scanning, XSS	✓	✓	IP address	[32, 46, 105, 106, 167, 181, 199]
CCD-INID-V1 [127]	2021	83	N/A	N/A	ARP poisoning, APR DoS, UDP Flood, Brute force, Slowloris	✓	✗	IP address	-
IoT DoS and DDoS Attack Dataset [80]	2021	80	N/A	N/A	CIC DDoS attacks	✓	✓	IP address	-
X-IIoTID [11]	2021	68	≈0.8M p	0.5:1	Generic scanning, scanning vulnerabilities, WebSocket fuzzing, discovering CoAP resources, brute force, dictionary attack, malicious insider, reverse shell, MitM, MQTT cloud broker-subscription, Modbus-register reading, TCP relay attack, data exfiltration, poisoning of cloud data (i.e., false data injections), fake notification, ransomware, and Ransom DoS	✓	✓	IP address	-
Edge-IIoTset [56]	2022	61	≈21M f	1.15:1	TCP SYN DDoS, UDP flood DDoS, HTTP flood DDoS, ICMP flood DDoS, Port scanning, OS Fingerprinting, Vulnerability scanning, DNS spoofing, ARP spoofing, XSS, SQL Injection, Remote file inclusion, Backdoor, Password cracking, Ransomware	✓	✓	IP address	[56]
LATAM-DDoS-IoT [17]	2022	20	≈30M f (DoS), ≈50M f (DDoS)	50.25:1	DoS, DDoS	✓	✓	IP address	-
CIC IoT [45]	2022	48	30k	N/A	DoS, RTSP brute force	✗	✓	IP address	-

logging the CAN traffic via the **On-Board Diagnostics** (**OBD**) port. Additionally, the dataset is partially labeled, that is, only for the DoS attack, and is offered in text format.

The **Vehicular Reference Misbehavior Dataset (VeReMi)** dataset [73] was built through a simulated vehicular environment considering five different attacks in which the vehicle's position is forged. The dataset also contains information about several scenarios in which vehicle density (i.e., number of vehicles) and attacker density are changed. It is a labeled dataset in which 17 features are used to characterize the vehicles' messages.

The **UAV Attack Dataset** [228] is one of the few datasets containing ad-hoc network traffic. It targets UAV environments and was created with the aid of five different simulation platforms. The dataset contains two attack scenarios, namely, **Global Positioning System** (**GPS**) spoofing and jamming. Although it seems to be the only one available for studying UAV IDS, it has a rather small size of less than 19 MB.

Table 5. Comparison of Other Contemporary Datasets which Can Be Used for Assessing FL-Enabled IDS

Dataset	Year	# features	# samples (packets/flows)	Attack-/benign traffic ratio	Attacks	Data labelled?	Realistic testbed?	Division	FL-enabled IDS works using the dataset
InSDN [52]	2020	83	≈0.3M f	4.03:1	Botnet, DoS, DDoS, web attacks, password brute-forcing, probe, exploitation	✓	✓	IP address	[59]
OTIDS [111]	2017	11	≈4.6M p	0.9:1	DoS, Fuzzy, impersonation	✓	✓	Vehicle	[15]
VeReMi [73]	2018	17	≈2.2M p	0.35:1	Misbehavior (position forging) attacks	✓	✓	Vehicle	[216]
Intrusion Detection in CAN bus [183]	2019	N/A	N/A	N/A	DoS, fuzzy	✗	✓	Device	-
UAV Attack Dataset [228]	2020	85	786,728 (sensor values)	0.02:1	GPS spoofing, and jamming	✓	✓	Device	-
Car Hacking [201]	2020	5	≈17.5M messages	0.17:1	Flooding, spoofing, replay, fuzzing	✓	✓	Vehicle	[7]
Car Hacking A&D Challenge [91]	2021	6	≈1.2M messages	16.8:1	DoS, fizzy, spoofing (drive gear), spoofing (RPM gauze)	✓	✓	Vehicle	[50]

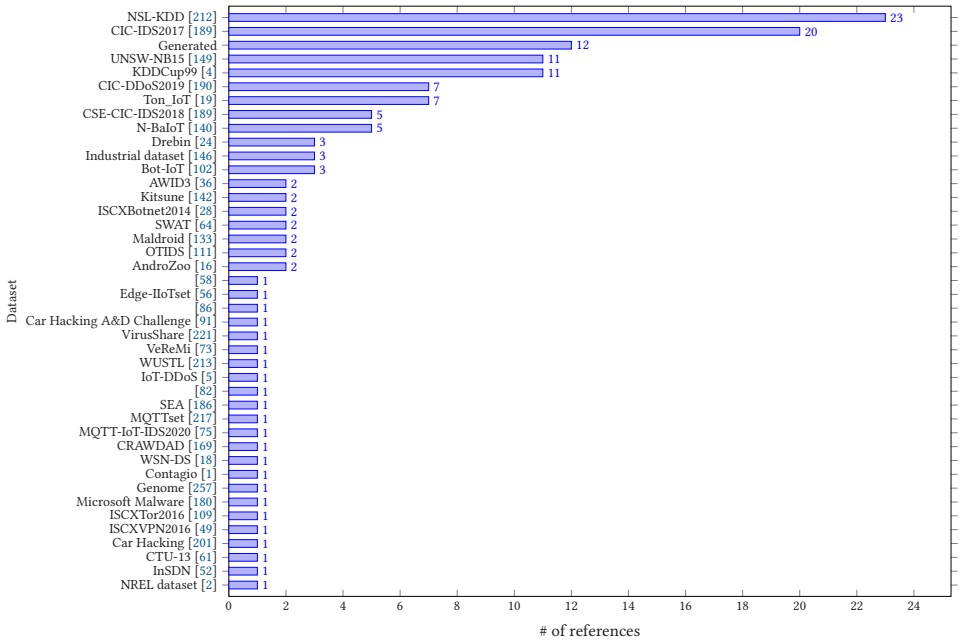


Fig. 4. Datasets employed by existing FL-enabled IDS proposals.

The **Intrusion Detection in CAN bus** [183] dataset comprises a fusion of three other datasets. The first is the Car Hacking Dataset v1 [111], from which the authors picked the "fuzzy" attack. The other two datasets were new collections, utilizing the ML350 and CL2000 Mercedes models. Overall, the dataset contains two attacks, namely DoS and "fuzzy", and has an unzipped size of 43 MB.

The **Car Hacking** dataset [201] focuses on different attacks on CAN bus, including DoS, fuzzy, and spoofing (RPM/gear). It considers five features, namely timestamp, CAN ID, DLC, Data, and flag, and was built by logging CAN traffic via the OBD-II port from a real vehicle.

The **Car Hacking: Attack & Defense Challenge** dataset [91] was created through a competition in Korea to develop attack and detection techniques of CAN by using Hyundai Avante CN7 as target vehicle. The dataset is composed of different attacks, including data about flooding, spoofing, replay, and fuzzing attacks on CAN messages, and similar features compared to the Car Hacking dataset.

D Supplemental Content for ML Models for FL-Enabled IDS

Table 6. Analysis on the Use of ML Techniques in FL-Enabled IDS Approaches

ML approach	Technique	Core	Works using this technique	Suitability	Challenges	Performance
Supervised. It relies on the use of labeled data to train a model. The training dataset consists of inputs and correct outputs, which enable the model to learn by adjusting its hyperparameters to the data. The dataset guides the model so that it can learn the relationship between inputs and outputs and make predictions on new data	Based on NN	Typically organized into interconnected layers (input, hidden, output). Neurons are weighted, and those weights adjust based on training data. Commonly, the NNs used in IDS employ a number of features in the dataset as the number of neurons in the input layer.	[9, 26, 56, 59, 70, 78, 93, 107, 108, 143, 150, 160, 161, 166, 167, 171, 173, 174, 182, 188, 216, 225, 243, 254, 255]	Effective for handling large-scale, high-dimensional IDS data.	Computationally expensive, requires large labeled datasets.	High accuracy with sufficient training data, scalable.
	GRU	A form of RNN that is computationally more efficient using fewer parameters than LSTMs. Additionally, GRUs aim to solve the vanishing gradient problem which comes with a standard RNN using the update gate and reset gate.	[41, 50, 105, 112, 113, 147, 154, 209, 210, 222]	Suitable for sequential intrusion detection, such as time-series anomaly detection.	Sensitive to hyperparameters and requires careful tuning.	Efficient and computationally less expensive than LSTMs.
	CNN	These are based on the shared-weight architecture of the convolution kernels or filters to extract relevant features from the input data. They can be considered as regularized versions of MLPs intended to reduce overfitting by leveraging the hierarchical structure of data.	[27, 39, 51, 54, 59, 70, 81, 112, 114, 125, 130, 132, 135, 163, 193, 195, 196, 203–206, 222, 237, 248, 250, 253, 256]	Ideal for feature extraction from network traffic images.	High resource consumption, prone to adversarial attacks.	Excels in spatial pattern recognition, high accuracy.
	SVM	The goal of this technique is to find a hyperplane to separate different classes with the largest possible margin, so that the samples of each class are as far as possible	[41, 56, 72, 76, 104]	Good for binary/multi-class classification tasks in IDS.	Computationally expensive for large-scale datasets.	High accuracy, but computationally intensive.
	MLP	It is the most well-known example of FNN, and it is characterized by one or several hidden layers between the input and output layers. Furthermore, an MLP is a fully connected network.	[12, 48, 72, 112, 123, 125, 176, 195, 203]	General-purpose, suitable for small to mid-sized datasets.	Prone to overfitting with high-dimensional data.	Moderate accuracy, depends on dataset size.
	BNN	They are characterized by using binary (+1 or -1) representations for weights and activations. Therefore, the sign function becomes the most suitable activation function under this setting	[170]	Efficient for constrained environments with binary computations.	May suffer from information loss due to binary constraints.	Energy-efficient, suitable for edge computing.
	LSTM	It is one of the main examples of RNN that maintains an internal memory that is updated with new inputs. Specifically, this memory is maintained by each neuron using three different gates: input, output, and forget.	[70, 147, 199, 202, 248, 251, 253]	Handles sequential data well, suitable for detecting long-term dependencies.	Requires large training datasets to prevent overfitting.	High accuracy in sequence-based anomaly detection.
	DT	Based on a hierarchical structure where each node represents a decision (on the basis of a certain variable) and each leaf represents a prediction.	[48, 56, 72, 158, 242]	Simple and interpretable, useful for rule-based intrusion detection.	Lacks scalability for large datasets, prone to overfitting.	Interpretable results, moderate accuracy.
	RF	Unlike GBDT that builds trees one after another, RF creates several completely independent trees that are then combined to improve the model's accuracy	[15, 50, 56, 72, 104, 136, 195, 196]	Works well for ensemble learning, reducing overfitting.	Sensitive to noisy data, requires careful feature selection.	Good generalization, suitable for large datasets.

(Continued)

Table 6. Continued

ML approach	Technique	Core	Works using this technique	Suitability	Challenges	Performance
	RNN	It allows signals to flow in both directions; that is, signals feed back to neurons in the different layers	[59, 247]	Handles sequential or time-series data effectively, making it suitable for detecting temporal attack patterns.	Susceptible to vanishing or exploding gradients. Requires careful tuning of hyperparameters. Computationally expensive for long sequences.	Can achieve moderate to high detection accuracy if trained with enough sequential data.
	LR	It uses a set of variables and a logistic function to model the probability that a sample belongs to a certain class	[32, 43, 46, 181, 188, 195, 196]	Effective for simple linear classification tasks.	Limited in handling complex non-linear decision boundaries.	Performs well in binary classification tasks.
	NB	It is one of simplest types of Bayesian network that assumes independence among the observed features, which makes it efficient for certain applications	[104]	Well-suited for simple, discrete-feature datasets. Requires relatively low computational resources and smaller training sets.	Strong independence assumption among features often reduces accuracy in complex, correlated datasets.	Very fast classification with moderate accuracy, but can produce higher false positives when the independence assumption is violated.
	KNN	It is based on assigning a certain class to a sample using its K nearest neighbors in the feature space	[56, 196]	Effective where attack patterns resemble previous known patterns. Suitable for low-dimensional data classification.	Struggles with high-dimensional data due to increased computational cost.	Simple and interpretable. Performs well when sufficient labeled data is available but suffers in high-dimensional spaces.
	Transformers	A transformer is based on an encoder-decoder architecture and uses a self-attention mechanism to give assign different weights to each part of the input data	[7]	Particularly effective in scenarios requiring the capture of long-range dependencies or contextual relationships.	Computationally and memory-intensive, often requiring substantial labeled data and specialized hardware.	High accuracy on sequence-based tasks if sufficiently trained. Can outperform traditional RNNs in capturing global context, but at the cost of higher resource consumption.
Unsupervised. This type of learning mainly consists of finding patterns from unlabeled data, while some well-known techniques are *clustering* and *dimensionality reduction*	AE	It is represented as a specific case of NN composed by an input layer (encoder), several hidden layers and an output layer (decoder), where the objective is to learn a compressed representation of certain input data. The encoder maps the input data into a hidden representation, and the decoder is intended to reconstruct the input data from such representation.	[13, 35, 40, 62, 85, 168, 176, 229, 235]	Best for anomaly detection in unlabeled data.	Sensitive to noise, struggles with complex attack patterns.	Detects unknown attacks, performs well on anomalies.
	GAN	A GAN consists of two NNs called generator and discriminator, which compete during the training process. Specifically, the generator creates synthetic data to deceive the discriminator so that the latter learns to distinguish between synthetic and real data	[208]	Useful for generating realistic network attack scenarios.	Difficult to stabilize training, may generate unrealistic data.	Capable of generating realistic attack data.

(Continued)

Table 6. Continued

ML approach	Technique	Core	Works using this technique	Suitability	Challenges	Performance
	DBN	They can be considered as a structure composed of several layers or processing units, such as AEs	[95, 232]	Suitable for feature extraction and anomaly detection. Works well for hierarchical feature learning in FL.	Computationally expensive. Requires large training datasets and careful tuning of hyperparameters	Capable of deep feature representation learning. Performs well in detecting previously unseen attack patterns.
	Isolation Forest	It is based on DTs representing a subset of the input data	[227]	Ideal for detecting outliers and novel attack types. Works well in unsupervised anomaly detection.	Limited interpretability. Can struggle with imbalanced datasets.	Fast anomaly detection with a low false positive rate in balanced datasets. Effective for attack instances with high accuracy.
	Elliptic Envelope	It fits an ellipse to the dataset and evaluates the distance of each sample to the ellipse's centroid	[227]	Useful for anomaly detection in moderate-dimensional data with roughly elliptical distribution.	Assumes data follow near-elliptical contours. Performance degrades if real-world distributions are multimodal or strongly skewed.	Provides moderate outlier detection at a relatively low computational cost.
	SOM	A SOM is represented by a NN in which the neuron weights are adjusted so that data with similar characteristics are grouped in specific areas of the map	[139]	Useful for clustering network behaviors. Works well for visualizing high-dimensional intrusion data.	Requires careful tuning of learning rate and neighborhood size. Can struggle with overlapping clusters.	Intuitive way to analyze attack patterns. Suitable for detecting anomalous behavior but may require post-processing for attack classification.
	K-means	It partitions a dataset into K distinct non-overlapping subgroups (clusters) where each observation/subject belongs to only one group. The objective is to minimise the intra-cluster distance and maximize the inter-clusters one. It assigns data points to a cluster such that the sum of the squared distance between the data points and the cluster's centroid is minimized.	[106, 233, 234]	Effective for clustering network traffic behavior patterns.	Sensitive to initial cluster assignment, may not scale well.	Useful for detecting evolving attack patterns.
Semi-supervised. It is based on the use of both labeled and unlabeled data to build a certain model.	AE-NN	AEs are initially used to learn representations in the data so that they feed a NN to enhance the performance of the supervised task	[20, 21, 172]	Enhances IDS with feature learning and classification.	Performance depends on feature extraction quality.	Moderate accuracy, improves data representation.
	GAN-CNN	Combination of both techniques to train in certain scenarios	[198]	Improves IDS feature extraction and synthetic data utility.	Prone to mode collapse, requires adversarial training.	Improves classification in hybrid training scenarios.
	SACN	This models improves the efficiency of CNNs, in which capsules are used instead of neurons to represent object features. In a semi-supervised approach, SACN can be beneficial in scenarios with limited labeled data.	[164]	Enhances CNN-based intrusion detection with capsule networks. Suitable for limited labeled data in semi-supervised learning.	High computational cost. Requires extensive training and careful tuning of capsule parameters.	More robust to spatial hierarchies in data compared to traditional CNNs. Performs well in limited-label scenarios with improved feature extraction.

(Continued)

Table 6. Continued

ML approach	Technique	Core	Works using this technique	Suitability	Challenges	Performance
	CNN	The classification of unlabeled data is based on a pre-training approach using a CNN over labelled data	[252]	Suitable for semi-supervised learning with limited labeled data.	Lacks generalization without sufficient labeled data.	Pre-trained models boost performance on real-world IDS.
Reinforcement. In this ML branch, an agent learns to maximize a certain numerical reward by interacting with the environment.	Q-learning	In this technique, a table of values is maintained to choose the best action in each state. This table of "q-values" is updated using the rewards obtained from the actions taken by the agent. The term Q refers to the function used to calculate the reward, considering a specific action and state.	[159]	Reinforcement-based IDS learning, adapts to evolving threats.	Convergence time is slow, requires fine-tuning.	Capable of dynamic adaptation to new threats.
	DQN/D-DQN	DQN can be considered a variant of Q-learning, where a NN is used to approximate the Q function. Additionally, it incorporates prior knowledge using the *replay experience* technique. DDQN uses two NNs to select the best action and evaluate the quality of that action respectively in order to reduce the possible overestimation of the reward in DQN	[155, 226]	Learns attack patterns dynamically for robust IDS.	Overestimation issues, requires dual Q-networks.	Mitigates overestimation, stable reinforcement IDS learning.

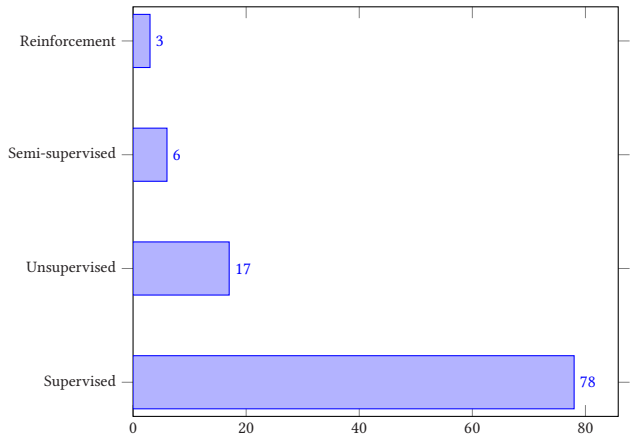


Fig. 5. Number of publications according to the ML type.

E Supplemental Content About Aggregation Functions and Their Use in Existing FL-Enabled IDS

E.1 FedAvg

FedAvg was initially proposed in the original FL paper [138], in which each client's local update is aggregated on the server by calculating the average, depending on their data length. Specifically, the FedAvg objective function is calculated using the equation $\min [F(W) = \frac{D_k}{D} \sum f_k(W^k)]$ subject to $w_i = w_i^k$, where $W = (w_i)_{i=1}^n$ are the weights of the server's model, $W^k = (w_i^k)_{i=1}^n$ the weights of client k, f_k is the loss function of client k, and D_k and D represent the length of the client k dataset and the total length of the clients, respectively. Based on this function, the weights are updated according to $W = \frac{D_k}{D} \sum W^k$. The operation of FedAvg is determined by the

number of participating clients C in each training round, the number of local training epochs E that each client performs on its data, as well as the size of the local minibatch B used for local training. Actually, FedAvg can be considered a generalization of FedSGD, which is based on **Stochastic Gradient Descent** (**SGD**), where $B = \infty$ and $E = 1$. Therefore, in FedSGD, each client is trained with the complete dataset and only performs one local training epoch in each round.

E.2 FedPAQ

FedPAQ [175] follows a similar aggregation approach to FedAvg, while providing a higher degree of efficiency in environments with certain computing and network constraints. In particular, FedPAQ adds three main features: periodic averaging, partial device participation, and quantized message-passing. First, periodic averaging means that aggregation is performed at certain intervals, allowing each client to perform multiple epochs of local training before sending its updates to the server. Second, it allows only a subset of nodes to participate in each training round, reducing network overhead and improving scalability. Third, FedPAQ uses quantization to reduce the size of models sent to the server in each training round. Specifically, each node employs quantized operators on the difference between the local model created by the client and the global model received during a training round. Overall, the server weights are updated based on equation $W_{r+1} = W_r + \frac{1}{|S_n|} \sum_{k \in S_n} Q(W_r^k - W_r)$, where W_r and W_r^k represent the weights of the global model and client k at round r, respectively, S_n is the subset of the n clients chosen in that round, and Q is the quantized function.

E.3 FedMA

FedMA [224] was designed to address the issue of permutation invariance of NN parameters. Unlike FedAvg, which performs coordinate-wise averaging, FedMA conducts a matching process of the clients' NNs to perform layer-wise averaging. In addition to improving performance, this reduces the required communication rounds. In FedMA, after each client trains its model, the server sets the first layer's weights using one-layer matching, i.e., finding the permutations $\Pi_{k,l,\cdot}$, where k is the client, and l represents the layer, which minimizes equation $\sum_{i=1}^{L} \sum_{k,l} min_{\theta_i} \Pi_{k,l,i} c(w_{kl}, \theta_i)$. The equation of FedMA is $W_n = \frac{1}{K} \sum_k W_{k,n} \Pi_k^T$, where Π_k^T are the solution of the matched average of the previous equation. Note that θ_i are the global model's weights, and $c(\cdot, \cdot)$ is a similarity function, such as the Euclidean distance.

E.4 FedProx

FedProx [117] is a variant of FedAvg, whose main goal to address system and statistical heterogeneity of common FL settings. Especially, FedProx adds a *proximal term* to restrict the local models to be closer to the central one by limiting the impact of local updates. Furthermore, it also addresses device heterogeneity to deal with scenarios involving devices with different computation capabilities. In particular, the approach is intended to address the problem of *stragglers*, which represent devices with low capabilities that could have a negative impact on system convergence. The objective function of FedProx is calculated as $min\, F_k(W^k) + \frac{\mu}{2} ||W^k - W||^2$, where F_k is the loss function of client k, W^k and W are the client's weights and aggregated weights, respectively, and μ is the proximal term chosen by the user.

E.5 Fed+

Fed+ [240] (or Fedplus) is a set of algorithms developed to handle data heterogeneity, and in particular, non-IID data. Fed+ takes the loss function of FedAvg and adds a penalty function on each client to remove the restriction that all clients' weights converge to the same point. That is, the Fed+

objective function is calculated as min $\left[F_\mu(W) = \frac{1}{D}\sum[f_k(W^k) + \mu B(W^k, A(W))]\right]$, where $\mu > 0$ is a penalty constant chosen by the user, A is an aggregation function (e.g., the average) of the clients' weights, and $B(\cdot,\cdot)$ is a distance function that penalizes the deviation of a local model W^k from the output of $A(\cdot)$.

Furthermore, the corresponding weights are updated by using equation $W^k_{r+1} \longleftarrow \theta[W^k_r - \nu\nabla f_k(W^k_r)] + (1-\theta)Z^k$, where r is the round, ν the learning rate, $\theta = \frac{1}{1+\nu\mu}$ a constant that controls the degree of regularization, i.e., how the local weights are influenced by the distant function, and Z^k the result of $B(W^k, \tilde{W})$. This weighted average between their own weights and the one provided by the previous equation leads to each client having their own model through the federated process. This removes the restriction that all clients' weights converge to the same point.

E.6 Others

While previous approaches are widely considered in FL settings for different use cases and scenarios, a significant number of other aggregation functions have been also defined in the literature. The following subsections provide a brief overview of these techniques for reasons of completeness.

E.6.1 Turbo-Aggregate. Turbo-Aggregate [200] stems from the idea of securing the communication of FedAvg between clients and server through secure aggregation techniques. The clients are separated into several groups, and model updates are shared among groups by using an additive secret sharing approach. Secret sharing is used to mask each local model by adding randomness in a way that can be cancelled out once the models are aggregated. In particular, the weights of each cluster are encoded and sent to the next cluster, where it will be decoded and aggregated with the weights of that cluster. The process is repeated for all the clusters. Finally, the last cluster sends the final aggregated model to the server, where it reconstructs the encoded model for obtaining the actual weights and sends them to the clients, where a new round starts.

E.6.2 FedCD. Similar to FedAvg, FedCD [101] begins with a global model on a centralized server that all devices are updated with. In each round, clients train different models and calculate a score related to the performance of these models. To face non-IID scenarios, FedCD, starting with one model in certain rounds, clones the models with high performance based on its score, deleting the ones with low scores. Finally, the model with the highest score is selected. However, this cloning can overload the devices' resources, thus delaying the whole process.

E.6.3 Krum. Krum aggregation [30] is intended to deal with scenarios including f malicious peers. It is defined by $Kr(W_1, \ldots, W_n) = W_k$, where certain weights W_k are selected from the clients' weights W_i to minimize their distance between its $n - f - 2$ closest neighbors, where n is the number of clients. Such weights W_k are sent to the server at every round. However, the number of malicious clients should be known in advance, which makes Krum aggregation impractical.

E.6.4 SAFA. **Semi-Asynchronous Federated Averaging** (**SAFA**) [230] was intended to cope with scenarios characterized by extremely weak communication conditions, where some clients may be disconnected depending on their capabilities. To this end, it is based on an asynchronous aggregation approach to mitigate the impact of stragglers. Furthermore, the authors propose a cache structure in the cloud to reduce the cost of communication, as well as a post-training client selection approach to improve convergence.

E.6.5 CDA-FedAvg. **Concept-Drift-Aware Federated Averaging** (**CDA-FedAvg**) [33] applies FedAvg to concept drifts, that is, situations where data could change due to a certain event or circumstance. In this approach, the server aggregates weights asynchronously, namely, there is no predefined sequence to upload the global model. On the other hand, the clients have two different

data storages (short and long term) to check if a drift occurs. If so, additional training rounds are performed to reflect the changes derived from the concept drift.

E.6.6 SecureBoost. SecureBoost [42] is a framework for GBDT on vertically-partitioned data. SecureBoost enhances clients' privacy by sharing a GBDT model through multi-party data under privacy constraints. Specifically, SecureBoost determines inter-database intersections with a privacy-preserving protocol based on different techniques (e.g., Paillier encryption [162]), which are employed by clients to calculate the weights at every round prior to sending them to the server.

E.6.7 CMFL. CMFL [131] follows the idea of removing irrelevant updates from certain clients. To calculate the level of relevance of a certain model update, each client is forced to compare its local update with the global model by calculating the percentage of parameters with a different sign (positive or negative). The higher the percentage , the more irrelevant the update, so it can be discarded by considering a certain threshold.

E.6.8 DW-FedAvg. **Dynamic weighted federated averaging (DW-FedAvg)** [38] is based on FedAvg, with the goal of rewarding best performing clients. In particular, DW-FedAvg updates the weights following the equation $W = \sum \beta_k \cdot W^k$, where β_k represents the adjusting factor associated with clients' weights. These β_k are updated based on the local performance of each client. Initially, β_k are initialized as the $\frac{1}{D_k}$, where D_k is the length of the dataset of client k. Then, if client's accuracy is higher than the average accuracy, $\beta_k = \beta_k + \beta_k \cdot \alpha$, where α is a reward/penalty factor chosen by the user. In case the user's accuracy is lower than the average accuracy, $\beta_k = \beta_k - \beta_k \cdot \alpha$. Once β_k are updated, these values are rescaled by $\beta_k = \frac{\beta_k}{\sum_k \beta_k}$. Finally, this process is repeated throughout all training rounds.

Table 7. Comparison Among Main Aggregation Methods Given in Subsections E.1 to E.5

Technique	Core	Advantages	Disadvantages	Works using this technique
FedAvg	Based on the weighted average of the updated weights provided by clients	It is widely used due to its low level of complexity	It poses convergence issues in FL settings with non-IID data distributions	[7, 12, 13, 15, 20, 21, 26, 27, 32, 35, 39–41, 46, 48, 50, 51, 54, 56, 59, 62, 70, 72, 76, 78, 81, 85, 93, 95, 104–108, 112–114, 123, 130, 132, 135, 136, 139, 143, 147, 150, 154, 155, 159–161, 163, 164, 166–168, 171–174, 176, 181, 182, 188, 193, 198, 199, 202–206, 208–210, 216, 222, 225–227, 229, 232–235, 237, 242, 243, 247, 248, 250–256]
FedPAQ	It is based on averaging model updates but providing a higher degree of efficiency through periodic averaging, partial device participation and quantized message-passing	It reduces communication and computation overhead	It requires additional functionality on both aggregator and clients, and still poses issues with non-IID settings	-
FedMA	It is based on the neurons' permutation invariance through a matching process of the clients' NNs to perform layer-wise averaging	It adapts to global model size and data heterogeneity and improves convergence, while requiring fewer training rounds	The computation of the permutation matrix may increase the computation time, and it is specific for CNN and LSTM	-
FedProx	It adds a proximal term to limit the impact of the different local updates	It addresses both data heterogeneity considering non-IID settings and device heterogeneity	It increases computation requirements and its performance largely depends on the appropriate election of the proximal term	[195, 196, 202]
Fed+	Similar to the previous proximal term, a penalty constant is also added but different aggregation functions can be employed (beyond average)	It adds an increasing level of flexibility by considering different aggregation functions beyond average	A higher flexibility comes with a cost in complexity, and the performance also depends on the right choice of the penalty constant	[32, 167, 181]

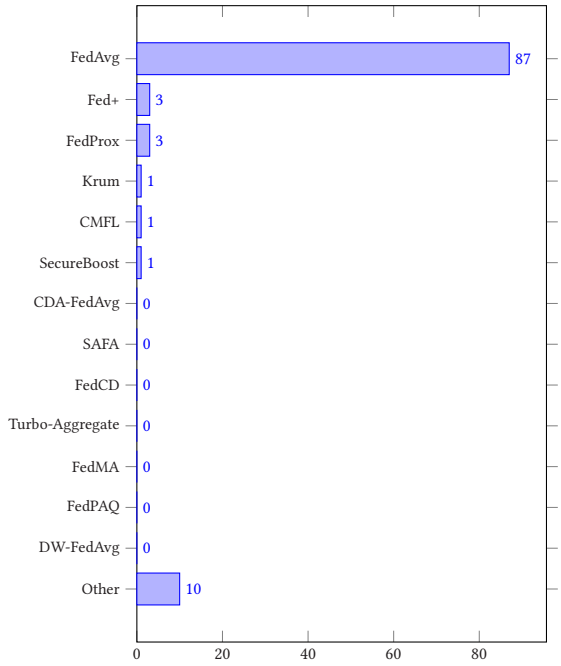


Fig. 6. Number of publications according to the use of a certain aggregation method.

F Supplemental Content About Evaluation Aspects in Existing FL-Enabled IDS

F.1 Evaluation Metrics

Evaluating classification scenarios involves the use of various metrics to assess the performance of the model. These metrics provide insights into how well a given model is performing and aid in comparing different models or tuning their hyperparameters. Annex X provides

F.1.1 Binary Classification Metrics.

- True Positive (TP): number of positive instances correctly classified as positive
- True Negative (TN): number of negative instances correctly classified as negative
- False Positive (FP): number of negative instances incorrectly classified as positive
- False Negative (FN): number of positive instances incorrectly classified as negative
- **False Positive Rate** (**FPR**): ratio of positive cases incorrectly classified as negative: $(\frac{FP}{FP+TN})$
- **False Negative Rate** (**FNR**): ratio of positive cases incorrectly classified as negative: $(\frac{FN}{FN+TP})$

F.1.2 General Classification Metrics. For reason of brevity, in the following, legacy/most common evaluation metrics, such as accuracy, precision, F1-Score, AUC, or TP rate are omitted.

- Accuracy measures the overall correctness of the model's predictions and is calculated as the ratio of correct predictions to the total number of predictions. $(\frac{TP+TN}{TP+TN+FP+FN})$
- Precision is a measure that quantifies the accuracy of positive predictions made by the model. Precision is calculated by dividing the number of TP by the sum of TP and FP. $(\frac{TP}{TP+FP})$
- **TP Rate** (**TPR**) (also known as sensitivity, recall, or hit rate) is the ratio of TP to the total number of actual positive instances. It was used in 61% of the surveyed studies. $(\frac{TP}{TP+FN})$
- **False Omission Rate** (**FOR**) is a performance metric used in authentication tasks to assess the likelihood of incorrectly rejecting genuine users or samples. It measures the rate at which

the system erroneously classifies genuine instances as impostors or unauthorized users. ($\frac{FN}{FN+TN}$)
- F1-score combines the precision and TPR into a single value, providing a balanced measure of a model's performance. The F1-score is commonly used when there is an imbalance between the positive and negative classes or when both precision and TPR are equally important. ($2 * \frac{TRP*Precision}{TRP+Precision}$)
- The AUC quantifies the overall quality of the model's predictions across different thresholds. To calculate the AUC, the ROC curve is generated by plotting the TPR on the y-axis against the FPR on the x-axis at different classification thresholds. The AUC is then computed by calculating the integral (area) under this curve; it ranges from 0 to 1. An AUC of 1 represents a perfect classifier that achieves a TPR of 1 while maintaining an FPR of 0. An AUC of 0.5 indicates a classifier that performs no better than random guessing.
- MCC takes into account all four values in the confusion matrix (TP, TN, FP, FN): $\frac{TP \times TN - FN \times FP}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$. MCC values have a range between -1 and 1. A value close to 1 means that both classes are predicted well, even if one class is disproportionately under- or over- represented. If $MCC = 1$ then $FP = FN = 0$ which means that the model is perfect. When $MCC = -1$ then $TP = TN = 0$, indicating a negative correlation; consequently, the model produces the opposite of the true value. If $MCC = 0$, it means that the model is totally random.
- CKS takes values between -1 and 1, and is also used to measure the reliability of the classifier's predictions; nevertheless, it was not found in any of the reviewed works. Practically, CKS considers the possibility of agreeing by chance, and measures the number of predictions it makes that cannot be explained by a random guess. CKS is defined by the formula $\frac{a-p_c}{1-p_c}$, where a is the accuracy of the model, and p_c is the probability of having the correct prediction by chance. The calculation of p_c is based on the confusion matrix. Let p be the multiplication of the total sum of the elements of a column divided by the total number of elements, and the total sum of the elements of a row divided by the total number of elements. Then, p_c is the sum of the previous p applied to each column.

F.1.3 Regression Metrics. Unlike previous indicators, regression metrics are employed to evaluate the predictions of regression models. The most widespread metrics are:

- **Mean Square Error** (**MSE**) is the average squared difference between the predicted values and the true values in a regression problem. ($\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$)
- **Mean Absolute Error** (**MAE**) is used to calculate the average absolute difference between the predicted and actual values. ($\frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$)
- **Mean Absolute Percentage Error** (**MAPE**) calculates the average percentage difference between the predicted and actual values. ($\frac{1}{n}\sum_{i=1}^{n}|\frac{y_i - \hat{y}_i}{y_i}| \times 100$)
- **Root Mean Squared Error** (**RMSE**) is used to measure the average magnitude of errors between the predicted and actual values, taking into account both the magnitude and the direction of the errors. RMSE is calculated by taking the square root of the average of the squared differences between the predicted and actual values.(RMSE = $\sqrt{MSE}$)
- Pearson correlation coefficient measures the strength and direction of the linear association between two variables, ranging from -1 (perfect negative correlation) to 1 (perfect positive correlation), with 0 indicating no correlation. It is computed as $r = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}}$, where n is the number of pairs of observations, x_i and y_i are the values of the first and second

variables, respectively, for the ith pair of observations, while $\bar{x}$ and $\bar{y}$ are the means of the variables x and y, respectively.

F.2 FL Implementations

In this section, a brief overview of FL implementations is provided. The interested reader could also refer to [96] for more detailed descriptions and comparison of the various implementations. Regarding framework selection, [126] provides a unified benchmark for evaluating the most popular FL frameworks in terms of functionality, usability, and performance.

TensorFlow Federated (TFF) is an open-source library developed by Google for training models in federated settings. TFF offers two interfaces to apply the included implementations of federated training on TensorFlow models,[1] as well as to define new algorithms to be used in federated environments. Additionally, it provides diverse implementations to reduce the communication overhead between devices and the server, as well as tools to apply DP techniques. As of June 2023, TFF is still in version 0.59, that is, in a pre-release status.[2]

PySyft[3] is an open-source Python library designed for secure and private DL. PySyft decouples private data from model training, using FL, DP, and SMPC. It was developed by the OpenMined community and works mainly with DL frameworks, such as PyTorch [3] and TensorFlow. PySyft supports two types of computations: dynamic computations over hidden data, and static computations, that is, graphs of computations that can be executed later in a different environment. PySyft is not production-ready as it is still in a beta release in version 0.8.1, as of June 2023.

The **Federated AI Technology Enabler (FATE)**[4] is an industrial grade open-source framework based on HE and SMPC. It provides various ML algorithms, including logistic regression, tree-based algorithms, and additional techniques for DL and TL. FATE can be installed on Linux or Mac platforms and supports standalone and cluster deployments. Unlike previous implementations, FATE can already be used in production environments; its current version is 1.11 (June 2023).

Paddle Federated Learning (PaddleFL)[5] is an open-source FL framework based on the PaddlePaddle platform. PaddleFL is Python-based and can be used in production environments; currently, as of June 2023, it is in version 1.2. The data partitioning types supported by PaddleFL are horizontal and vertical FL, whereas support for federated TL will be developed in the future.

IBM Federated Learning (IBMFL)[6] is an open-source Python library designed to support easy set up of FL in productive environments. IBMFL is an enterprise-level solution that provides a basic FL layer over which more advanced features could be added. It supports conventional ML algorithms, such as linear regression and k-means, as well as DNNs, while facilitating the easy implementation of new FL algorithms. In IBMFL, supervised and unsupervised, as well as RL approaches are included. The IBMFL code repository is maintained and its latest version is 1.1 (as of June 2023).

FedML[7] is an open research library and benchmark written in Python, which attempts to address limitations of other FL approaches and promote FL research. Its creators argue that FedML overcomes the main limitations of other FL libraries by: (i) supporting diverse FL computing paradigms and diverse FL configurations, (ii) providing standardised FL algorithm implementations and benchmarks, and (iii) being open and continuously evolving. It supports three main computing

[1] https://www.tensorflow.org/
[2] https://github.com/tensorflow/federated/releases
[3] https://github.com/OpenMined/PySyft
[4] https://fate.fedai.org/
[5] https://github.com/PaddlePaddle/Paddle
[6] https://github.com/IBM/federated-learning-lib
[7] https://github.com/FedML-AI/FedML

paradigms: on-device training for edge devices (for example, mobile and IoT), distributed computing, and standalone, single-machine simulation. At the time of writing this article, it has a pre-release version (0.8).

Flower[8] is a Python-based, open source FL framework focusing on large-scale experiments with heterogeneous devices. Indicatively, it can support experiments with up to 15M clients with a pair of high-end GPUs. Among Flower advantages are stability, wide support of languages, operating systems and ML frameworks, support of scenarios with diverse privacy requirements, as well as flexibility to enable the implementation of novel approaches with low engineering overhead. Moreover, it facilitates the extension of FL implementations to mobile devices with diverse hardware specifications and the transition of existing ML training setups to an FL environment. The code base of Flower is actively maintained with the latest release being version 1.5 in June 2023.

FederatedScope[9] is a relatively novel platform for FL that was designed to be flexible enough to manage heterogeneous data, resources, behaviors and learning goals, using an event-driven architecture. Two further objectives of this platform are usability and extensibility. The former is fulfilled by providing different programming interfaces to users, whereas the latter by extensible <event, handler> pairs thanks to its event-driven architecture. FederatedScope is open-source, Python-based and is intended for both research and production environments. Its source code is maintained but given its novelty there is no stable release yet; currently, its latest version is 0.3.0, as of June 2023.

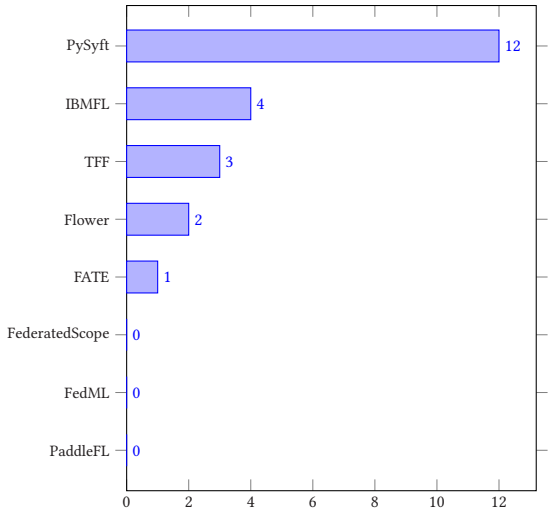


Fig. 7. Use of FL implementations in surveyed publications.

[8]https://github.com/adap/flower
[9]https://github.com/alibaba/FederatedScope

G Supplemental Content About Challenges in Existing FL-Enabled IDS

Table 8. Classification of Existing Works on FL-Enabled IDS

Challenge	Reference	Approach	Future aspects
Security	[198]	An Auxiliary Classifier GAN is proposed for generating malware attacks to increase the system's robustness	— Precise definition of a comprehensive list of attacks considering different configurations and attack parameters. — Exhaustive evaluation of aggregation mechanisms to assess their resilience in federated environments with malicious nodes. — Evaluation of trust and reputation approaches that allow weighting. — Analysis of the impact of cryptographic approaches. — Use of statistical approaches for identifying malicious nodes to ensure a training process based on legitimate clients.
	[208]	A GAN is used to generate poisoning attacks and different aggregation functions are exploited to evaluate the robustness in the scope of Android platform	
	[41]	Different aggregation functions are considered to measure their impact in scenarios with non-IID data distributions and against poisoning attacks	
	[26]	Label-flipping attacks are simulated to analyze the impact on the proposed approach	
	[234]	A reward-based mechanism is proposed to detect model poisoning attacks, so malicious nodes can be removed	
	[112]	Proposed cryptographic system to encrypt model parameters with the aggregation server	
	[196]	Proposed robust version of FedProx as aggregation function to detect malicious updates during the training process	
	[108]	Implementation of an approach to assess the trust level of each node based on the variations of each local model throughout the training rounds	
	[222]	Implementation of a mechanism to reject malicious update generated through label-flipping and GANs	
	[176]	Analysis of the impact of different data/-model poisoning attacks by considering different aggregation functions	
	[250]	Implementation of a model-level defensive mechanism to address poisoning attacks	
Privacy	[12]	Use of mimic learning where a certain model is used to train sensitive data	— Analysis of privacy-preserving techniques' impact regarding the compliance with existing data protection regulations, such as GDPR. — Identification of the limits on the required level of privacy and effectiveness for a certain IDS depending on the deployment scenario. — Exhaustive analyses to find trade-offs between the privacy level, as well as the effectiveness and performance of the deployed IDS. — Analysis of GAN-based approaches for federated training using synthetic data instead of clients' real data.
	[76]	Implementation of a SMPC scheme for privacy-preserving aggregation	
	[51]	Use of DP during the training process in an SDN-enabled FL setting by using TensorFlow Privacy	
	[48]	Use of data masking to further protect the privacy of clients' data	
	[104]	Integration of HE and DP techniques for privacy-preserving model updates in an edge-based FL-enabled IDS	
	[70]	Use of HE and DP techniques to protect the model updates to be aggregated in each training round	
	[181]	Analysis of different DP techniques on the impact of the effectiveness for intrusion detection	
	[105]	Perturbation-based encoding combined with an LSTM-AE model to transform original data used for the federated training	
	[81]	Implementation of a DP mechanism by adding a noise layer in the proposed CNN model	
Aggregator as bottleneck	[72]	Clients' model updates are shared through a blockchain infrastructure to keep track of them	— Comprehensive evaluations on the applicability of blockchain in FL settings, particularly in the context of IDS, considering different deployment options such as consensus mechanisms, number of nodes, and frequency of model updates. — Definition and implementation of smart contracts for the aggregation and validation process of model updates produced by both the aggregator and FL clients. — Analysis of asynchronous aggregation approaches to alleviate network and computation overhead, which could have negative implications on the IDS efficiency. — Analysis of the applicability of fully decentralized approaches and comparison with hierarchical solutions where certain nodes are responsible for performing partial aggregations.
	[15]	Use of blockchain to store the models generated during the federated training process in a vehicular scenario	
	[7]	Capitalize on blockchain technology to come up with a decentralized validation process of clients' local updates and evaluation, considering different settings with malicious nodes	
	[168]	The authors integrate a blockchain infrastructure to add accountability to the model updates	
	[39]	Partial aggregation is carried out by intermediate nodes with the weights providing the best performance	
	[123]	Collaborative aggregation approach where several RSUs in a vehicular context are responsible of performing partial aggregations	

(Continued)

Table 8. Continued

Challenge	Reference	Approach	Future aspects
	[173]	Analysis of the impact of different data distributions in the accuracy of the intrusion detection process	
	[256]	Analysis of the impact of non-IID data distributions in the system's classification accuracy	
	[181]	Comparison between FedAvg and Fed+ as aggregation functions considering a setting with a non-IID data distribution and a subset of clients with the highest entropy	
	[254]	Use of a TL scheme to simulate scenarios where certain nodes suffer from data scarcity	
	[247]	Use of a data resampling algorithm based on SMOTE for dealing with highly unbalanced data distributions	
Data heterogeneity	[32]	A client selection approach based on entropy values and use of Fed+ to deal with non-IID data distributions	– Analysis of the requirements and the impact of vertical FL scenarios for the development of FL-enabled IDS approaches. – Evaluation on the impact of aggregation functions in FL-enabled IDS under non-IID settings. – Analysis on the integration of TL approaches to deal with non-IID settings where certain devices are unable to access data.
	[113]	An aggregation function based on FedAvg in which the difference between data distributions is taken into account	
	[56]	Evaluation of a proposed dataset considering the impact of non-IID data distributions	
	[113]	An approach to expand feature spaces to deal with non-IID data distributions	
	[242]	A system based on LGBM, which is evaluated in both IID non-IID settings	
	[172]	Devices are grouped according to their targeting attack types to carry out the federated training by considering a certain feature set	
	[251]	Evaluation of an IDS for false data injection attacks considering both IID and non-IID settings	
	[167]	Analysis of the impact of different aggregation functions on the performance of intrusion detection considering different datasets	
	[132]	A personalization approach to address the impact of non-IID data distributions	
	[202]	A FedProx-based aggregation approach to handle non-IID data distributions	
	[225]	Analysis of the proposed FL-enabled contrastive learning approach by considering different data distributions	
	[125]	An aggregation mechanism (FedBatch), which is analyzed and compared with FedAvg under non-IID data distributions	
	[54]	A TL scheme to obtain personalized models by using different datasets	
	[40]	Implementation of a two-phase optimization strategy to deal with non-IID data distributions	
	[59]	The authors evaluate the proposed IDS by considering different non-IID data distributions and number of clients	
	[252]	An FD-oriented aggregation mechanism which is evaluated under different scenarios with non-IID data distributions	
	[164]	A TL approach to build a semi-supervised approach where nodes leverage on the labelled data used by a cloud model that is used by nodes to train on unlabelled data	
	[161]	Use of a TL approach to create customized models based on a global model previously obtained from a federated training process	
	[182]	Analysis on the impact of including edge nodes between end devices and aggregator in settings with non-IID data distributions	
	[93]	Analysis on the impact of performing local training after the FL process to obtain personalized local models	
	[206]	A GAN is proposed to increase the number of samples especially from rare classes in order to deal with data imbalance	

(Continued)

Table 8. Continued

Challenge	Reference	Approach	Future aspects
Device heterogeneity	[150]	Application of the Dempster-Shafer theory [?] for client prioritization	— Precise definition of device characteristics influencing the FL training process. — Study of reinforcement learning-based approaches to maximize the effectiveness and adaptability of the FL environment through appropriate client selection. — Analysis of the impact of changing environmental conditions and device dynamics throughout their lifecycle. — Definition of resource orchestration approaches to address device heterogeneity considering the inherent dynamism in IDS environments.
	[204]	FL clients with similar performance are dynamically grouped throughout the training rounds	
	[27]	The evaluation process considers different aspects (e.g., the type of attack) for client selection during the training process	
	[232]	A client selection approach based on communication quality, computing power, and attack risk aspects	
	[43]	An RL-based approach to implement a client selection mechanism towards increasing system's accuracy	
	[163]	A TL-based approach to create personalized models based on FL considering smartphone devices	
	[9]	An aggregation function to determine the waiting time for the server to initiate a new training round	
	[81]	Analysis of a client selection approach by including adversarial samples	
Computation and communication requirements	[170]	Use of BNN [77] to reduce communication and memory overhead	— Analysis and evaluation of lightweight ML techniques in FL environments using TinyML-based implementations. — Design and implementation of quantization and pruning schemes to address bandwidth limitations in specific scenarios and networks. — Analysis of trade-offs between communication and processing overhead regarding training rounds and epochs and the achieved IDS performance.
	[135]	An FedAvg-based aggregation function to reduce the number of training rounds required for model convergence	
	[78]	A lightweight NN structure based on the reduction of number of layers and neurons in hidden layers	
	[243]	Use of a NN to build a memory efficient version destined to constrained scenarios	
	[48]	A data binning approach to group traffic statistics so that the communication overhead is reduced	
	[195]	Design of a bit-wise encoding technique to reduce the communication cost during the training process	
	[252]	Use of a FD mechanism to reduce the communication overhead during the training process	

N/S stands for not specified.

H List of Works Analyzed

Table 9. Classification of Existing Works on FL-Enabled IDS

Ref.	Year	Network architecture	Data availability	Data partitioning	Dataset	ML model	Aggregation function	FL implementation	Training parties	Training rounds	Evaluation metrics
Based on supervised models											
[154]	2019	Cross-device	Horizontal	Centralized	Generated	GRU	FedAvg	Based on Flask and Keras	2-15	3/51	FPR, TPR, time
[255]	2019	Cross-device	Vertical	Centralized	CIC-IDS2017, ISCXVPN2016, ISCXTor2016	NN (multi-task)	FedAvg	Based on Pytorch	N/S	N/S	Accuracy, precision, recall, training time
[114]	2020	Cross-device	Horizontal	Centralized	Generated [6]	CNN	N/S	TensorFlow Federated	N/S	1-8/1	FPR, TPR, training time, CPU use
[41]	2020	Cross-device	Horizontal	Centralized	WSN-DS, KDDCup99, CIC-IDS2017	GRU-SVM	FedAvg, CMFL and based on FedAvg	PySyft	10-50	1-50/1	Accuracy, FAR, recall
[254]	2020	Cross-device	Transfer Learning	Centralized	UNSW-NB15	DNN	FedAvg	N/S	10	N/S-N/S	Accuracy, precision, recall
[150]	2020	Cross-device	Horizontal	Centralized	CRAWDAD [169]	NN	FedAvg	N/S	6	1-10/25	Accuracy
[12]	2020	Cross-device	Horizontal	Centralized	NSL-KDD	MLP	FedAvg	TensorFlow, Keras	10	20/10	Accuracy, precision, recall, F1-score
[170]	2020	Cross-device	Horizontal	Centralized	CIC-IDS2017, ISCXBotnet2014	BNN	signSGD	TensorFlow, Keras	2-8	N/S-N/S	Accuracy, precision, recall, F1-score, latency, overhead
[205]	2020	Cross-device	Horizontal	Centralized	Generated	CNN	FedAvg	N/S	20	1-60/1	Accuracy, precision, recall, F1-score
[76]	2020	Cross-device	Horizontal, Vertical	Centralized	Generated	SVM	FedAvg	Based on Python	2-30	10/30	Accuracy, precision, recall, FPR, F1-score
[253]	2020	Cross-device	Horizontal	Centralized	SEA [186]	LSTM, CNN	FedAvg	TensorFlow, Keras	4	1-50/1	Accuracy, precision, recall, F1-score, latency, overhead
[173]	2020	Cross-device	Horizontal	Centralized	NSL-KDD	NN	FedAvg	N/S	4	1-5/1	Accuracy
[256]	2020	Cross-device	Horizontal	Centralized	[82, 146]	CNN	FedAvg	Based on Pytorch	2-10	5/20	Accuracy
[54]	2020	Cross-device	Transfer Learning	Centralized	CIC-IDS2017, NSL-KDD, Kitsune, IoT network intrusion dataset	CNN	FedAvg	N/S	4	1-20/1-10	Accuracy, TPR, FPR
[72]	2020	Cross-device	Horizontal	Centralized	KDDCup99	MLP, DT, SVM, RF	FedAvg	N/S	4	N/S-N/S	Accuracy, Precision, Recall, F1-score, AUC, delay
[143]	2021	Cross-device	Horizontal	Centralized	NSL-KDD	DNN	FedAvg	N/S	16	200/50	Accuracy, Precision, Recall, F1-score, AUC, FPR, delay
[243]	2021	Cross-device	Horizontal	Centralized	N-BaIoT, Kitsune, IoT-DDoS, WUSTL [213]	Fully connected NN	FedAvg	Pytorch, Pysyft	3	30/4	Accuracy, Precision, Recall, F1-score, memory use
[171]	2021	Cross-device	Transfer	Centralized	CIC-IDS2017, CIC-DDoS2019	NN	N/S	N/S	N/S	1-200/1	Precision, FPR, FNR
[247]	2021	Cross-device	Horizontal	Centralized	NSL-KDD, CIC-IDS2017, CIC-DDoS2019	RNN	Based on FedAvg	Pytorch, Pysyft	21	1-8000/100	Accuracy, Precision, Recall, F1-score
[113]	2021	Cross-device	Horizontal	Centralized	CIC-DDoS2019	GRU	Based on FedAvg	Pytorch	5	1-10/200	Accuracy, loss
[242]	2021	Cross-device	Horizontal	Centralized	CIC-DDoS2019	DT	N/S	N/S	1-30	1-200/1	Accuracy, Precision, Recall, F1-score, FPR, communication overhead, time
[107]	2021	Cross-device	Horizontal	Centralized	UNSW-NB15-v2	NN	FedAvg	N/S	10	55/1	Accuracy, Precision, Recall, F1-score, TP

(Continued)

Table 9. Continued

Ref.	Year	Network architecture	Data availability	Data partitioning	Dataset	ML model	Aggregation function	FL implementation	Training parties	Training rounds	Evaluation metrics
Based on supervised models											
[51]	2021	Cross-device	Horizontal	Centralized	CIC-DDoS2019	CNN	FedAvg	Flower, TensorFlow Privacy	3	1-10/1	Accuracy, Precision, Recall, F1-score, communication overhead
[161]	2021	Cross-device	Transfer	Centralized	CIC-IDS2017	DNN	FedAvg	Pytorch, TensorFlow	N/S	N/S-5	Accuracy, Precision, F1-score, time
[188]	2021	Cross-device	Horizontal	Centralized	NSL-KDD	LR, DNN	FedAvg	scikit-learn, PySyft	4-8	1-4/1	Accuracy, Precision, Recall, F1-score
[48]	2021	Cross-device	Horizontal	Centralized	CIC-DDoS2019	MLP, GBDT	Based on FedAvg (DataBin)	Based on Python	21,66	1000-2000/1	Accuracy, FOR
[39]	2021	Cross-device	Horizontal	Centralized	KDDCup99	CNN	Based on FedAvg	Pytorch	1-20	50-300/1	Accuracy, time
[27]	Cross-device	Horizontal	Centralized	2021	CIC-IDS2017	CNN	FedAvg	N/S	10	7/10	Accuracy, loss
[70]	2021	Cross-device	Horizontal	Centralized	CIC-IDS2017	LSTM, NN, CNN	FedAvg	Flask	2-6	3-7/10	Accuracy, precision, recall, F1
[222]	2021	Cross-device	Horizontal	Centralized	Kitsune	CNN, GRU	FedAvg?	TensorFlow, Keras	3-9	6-12/1	Accuracy, precision, recall, F1
[167]	2021	Cross-silo	Horizontal	Centralized	Ton_IoT, UNSW-NB15, BoT-IoT, CSE-CIC-IDS2018	DNN	FedAvg, CM, Fed+	N/S	4	1-10/1-10	Accuracy, precision, recall, F1-score, MCC
[9]	2021	Cross-device	Horizontal	Centralized	NSL-KDD	NN	Temporal Weighted Averaging (proposed)	TensorFlow	30	15/1	Accuracy, loss
[181]	2021	Cross-device	Horizontal	Centralized	Ton_IoT	LR	FedAvg, Fed+	IBMFL	4	50/1	Accuracy, Pearson correlation
[135]	2021	Cross-device	Horizontal	Centralized	NSL-KDD	CNN	Based on FedAvg (proposed)	Pytorch	10	40-200/40-200	Accuracy, precision, recall, F1-score
[26]	2021	Cross-device	Horizontal	Centralized	MQTT dataset [75]	NN	FedAvg	PySyft	10	1-10/1	Accuracy, precision, recall, F1-score
[204]	2021	Cross-device	Horizontal	Centralized	Generated	CNN	FedAvg	N/S	20	1-60/1	Accuracy, precision, recall, F1-score
[15]	2021	Cross-device	Horizontal	Centralized	CAN-intrusion dataset (OTIDS)	RF	FedAvg	Based on sklearn	5-20	N/S-N/S	Accuracy, precision, recall, F1-score
[78]	2021	Cross-device	Horizontal	Centralized	BoT-IoT	NN	FedAvg	Based on Python	4	1000/1	Accuracy, Precision, Recall, F1, ROC curve, memory/CPU use
[216]	2021	Cross-device	Horizontal	Centralized	Veremi	NN	FedAvg	TensorFlow	10	1-500/1	Accuracy, precision, recall, communication overhead
[193]	2021	Cross-device	Horizontal	Centralized	UNSW-NB15, CSE-CIC-IDS2018	CNN	FedAvg	N/S	10, 15	15,20/1	Accuracy, loss
[123]	2021	Cross-device	Horizontal	Decentralized	KDDCup99	MLP	FedAvg	Pytorch, Pysyft	2-6	10-40/1	Accuracy, precision, recall
[182]	2021	Cross-device	Horizontal	Centralized	NSL-KDD	NN	FedSGD, FedAvg	N/S	4-8	1-20/1	Accuracy, loss
[112]	2021	Cross-device	Horizontal	Centralized	[146]	CNN, GRU, MLP	FedAvg	Flask, Keras	3-7	2-10/1	Accuracy, precision, recall, F1-score
[93]	2021	Cross-device	Horizontal	Centralized	Generated	NN	FedAvg	N/S	3	1-3/1	Accuracy, precision, F1-score
[104]	2021	Cross-device	Horizontal	Centralized	KDDCup99	RF, SVM, NB	FedAvg?	N/S	N/S	N/S-1	Accuracy, overhead
[196]	2021	Cross-device	Horizontal	Centralized	Generated from VirusTotal	CNN, RF, LR, KNN	Based on FedProx	N/S	10-50	200/20-40	Accuracy, time, power, energy
[163]	2022	Cross-silo	Transfer	Centralized	Maldroid, Drebin, AndroZoo	CNN	FedAvg	TensorFlow Lite, keras	3	1-4000/1-8000	Accuracy, loss, F1, time, CPU, memory

(Continued)

Table 9. Continued

Ref.	Year	Network architecture	Data availability	Data partitioning	Dataset	ML model	Aggregation function	FL implementation	Training parties	Training rounds	Evaluation metrics
Based on supervised models											
[132]	2022	Cross-device	Horizontal	Centralized	CIC-DDoS2019	CNN	FedAvg	Pytorch	50	1-1000/1	Accuracy, precision, recall, F1-score
[248]	2022	Cross-device	Horizontal	Centralized	Generated	CNN-LSTM	FedAvg?	N/S	4	N/S-N/S	Accuracy
[202]	2022	Cross-device	Horizontal	Centralized	KDDCup99	LSTM	FedAvg, Based on FedProx	Based on Python	N/S	1-200/1-20	Accuracy, precision, recall, F1
[199]	2022	Cross-device	Horizontal	Centralized	NSL-KDD, Ton_IoT	LSTM	Based on FedAvg (hierarchical)	Keras, Scikit-learn, TensorFlow, Numpy	N/S	1-100/1	Accuracy, precision, recall, F1
[130]	2022	Cross-device	Horizontal	Centralized	NSL-KDD	CNN	FedAvg	Pytorch	N/S	N/S-N/S	Accuracy, Recall, FPR, time
[237]	2022	Cross-device	Horizontal	Centralized	NSL-KDD	CNN	FedAvg	Pytorch	N/S	N/S-N/S	Accuracy, Recall, FPR, time
[136]	2022	Cross-device	Horizontal	Centralized	KDDCup99, NSL-KDD, UNSW-NB15, CIC-IDS2017	RF	FedAvg	N/S	3-14 (depending on the dataset)	N/S	Accuracy
[225]	2022	Cross-device	Horizontal	Centralized	NSL-KDD	NN	FedAvg	Pytorch	50	15/4	Accuracy, recall, precision, F1, FPR, time
[125]	2022	Cross-device	Horizontal	Centralized	NSL-KDD	CNN-MLP	FedBatch (proposed)	Pytorch	100	N/S	Accuracy, precision, recall F1-score
[50]	2022	Cross-device	Horizontal	Centralized	Car Hacking [92]	RF, GRU	FedAvg	TensorFlow, Keras	N/S	N/S-100	Accuracy, precision, recall F1-score, time
[108]	2021	Horizontal	Centralized	Cross-device	UNSW-NB15-v2	NN	FedAvg	N/S	10	1-18/1	Accuracy, Precision, Recall, F1-score
[174]	2022	Cross-silo	Horizontal	Centralized	Generated (Based on [86])	DNN	FedAvg	N/S	2	1-3/20	Accuracy, precision, recall, F1, loss, ROC
[160]	2022	Cross-device	Horizontal	Centralized	CIC-IDS2017	NN	FedAvg	sklearn	1-100	1-10/1	Accuracy, DR, power consumption
[56]	2022	Cross-device	Horizontal	Centralized	Edge-IIoTset (proposed)	DT, RF, KNN, SVM, DNN	FedAvg	Based on sklearn	5-15	1-10/1	Accuracy
[105]	2022	Cross-device	Horizontal	Centralized	Ton_IoT	GRU	FedAvg?	PySyft	2	1-100/1-100	Accuracy, precision, F1, detection rate
[32]	2022	Cross-device	Horizontal	Centralized	Ton_IoT	LR	FedAvg, Fed+	IBMFL	4-10	1-300/1	Accuracy, precision, recall, F1, FPR
[147]	2022	Cross-device	Horizontal	Centralized	[58]	LSTM, GRU	FedAvg	Pytorch, Pysyft	1-40	N/S-N/S	Accuracy, precision, recall, F1, time
[46]	2022	Cross-device	Horizontal	Centralized	Ton_IoT	LR	FedAvg	sklearn	13	50/10	F1
[43]	2022	Cross-device	Transfer	Centralized	NSL-KDD, UNSW-NB15	CNN	FedSGD	Pytorch, Pysyft	10	1-40/1	Accuracy
[7]	2022	Cross-device	Horizontal	Centralized	Car Hacking [92]	Transformer	FedAvg	PySyft	20	30/50	Accuracy, precision, recall, F1
[251]	2022	Cross-device	Horizontal	Centralized	Generated	LSTM	FedAvg	Flower, Pytorch	3	300/1	Accuracy, precision, recall, F1, loss, time, communication overhead
[166]	2022	Cross-device	Horizontal	Centralized	BoT-IoT, N-BaIoT	DNN	FedAvg	IBMFL, TensorFlow, Keras	5	1-8/5	Accuracy, precision, recall, F1, time, memory
[158]	2022	Cross-device	Vertical	Centralized	SWaT [64]	GBDT	SecureBoost	FATE	6	N/S-N/S	Accuracy, time
[81]	2022	Cross-device	Horizontal	Centralized	MNIST, CTU-13	CNN	FedAvg	TensorFlow, IBMFL	200	N/S-N/S	Accuracy, time
[210]	2022	Cross-device	Horizontal	Centralized	CIC-IDS2017	GRU	FedAvg	Pytorch, Pysyft	2	1-500/1	Accuracy, precision, recall, F1
[195]	2022	Cross-device	Horizontal	Centralized	Generated	CNN, MobileNet, RF, LR, KNN, MLP	Based on FedProx	N/S	10-50	200/20-40	Accuracy, power, energy, and area on-chip
[59]	2022	Cross-device	Horizontal	Centralized	CSE-CIC-IDS2018, InSDN [52], MQTTset [217]	DNN, CNN, RNN	FedAvg	Sherpa.ai, TensorFlow, Keras	5-15	1-50/1	Accuracy, precision, recall, F1, time, energy

(Continued)

Table 9. Continued

Ref.	Year	Network architecture	Data availability	Data partitioning	Dataset	ML model	Aggregation function	FL implementation	Training parties	Training rounds	Evaluation metrics
Based on supervised models											
[206]	2022	Cross-device	Horizontal	Centralized	NSL-KDD, KDDCup99, UNSW-NB15	CNN	FedAvg	N/S	N/S	1-30/1-200	Accuracy, precision, recall, F1, AUC
[250]	2022	Cross-device	Horizontal	Centralized	UNSW-NB15, CSE-CIC-IDS2018	CNN	FedAvg?	Pytorch	100	1-50/1	Accuracy, recall
[209]	2022	Cross-device	Horizontal	Centralized	Based on [2]	GRU	FedAvg, Fed-AA, DeepFed-Avg, DeepFed-AA	Matpower, Pytorch	N/S	1-50/1	Accuracy, precision, recall, F1, time
[203]	2022	Cross-device	Horizontal	Centralized	NSL-KDD	MLP, CNN	FedAvg	TensorFlow	10	1-100/1-10	Accuracy, precision, F1, overhead
Based on unsupervised models											
[168]	2018	Cross-device	Horizontal	Centralized	CIC-IDS2017	AE	FedAvg	Based on Keras, Deeplearning4j and Spring Boot 2.0	2-12	1/1-50	Accuracy, loss, time
[35]	2019	Cross-device	Horizontal	Centralized	AWID	Stacked AE	FedAvg	LEAF	933	1-20/1	Accuracy, communication overhead
[208]	2020	Cross-device	Horizontal	Centralized	Drebin [24], [257], [1]	GAN based on CNN	FedAvg, Krum	TensorFlow, Keras	5-15	1-300/1-300	Accuracy
[62]	2020	Cross-silo	Horizontal	Centralized	KDDCup99	Deep AE-GMM	FedAvg	N/S	2	1-60/1	Precision, recall, F1-score
[95]	2020	Cross-device	Horizontal	Centralized	KDDCup99, NSL-KDD, UNSW-NB15, N-BaIoT	DBN	FedAvg	N/S	2, 3	N/S-N/S	Accuracy, precision, TPR
[85]	2021	Cross-device	Horizontal	Centralized	SWaT	Stacked AE	FedAvg	N/S	N/S	N/S-N/S	Accuracy, precision, recall, F1
[235]	2021	Cross-device	Horizontal	Centralized	CIC-IDS2017	AE, NN	FedAvg	N/S	N/S	1-100/1-100	Accuracy, MSE
[227]	2021	Cross-device	Horizontal	Centralized	CIC-IDS2017	Elliptic Envelope, Isolation Forest	FedAvg?	N/S	N/S	5/1	Accuracy, F1-score
[139]	2021	Cross-device	Horizontal	Centralized	CIC-IDS2017	SOM	FedAvg?	PySyft	N/S	N/S-N/S	Accuracy, precision, recall F1-score
[233]	2021	Cross-device	Horizontal	Centralized	AWID	K-means clustering	FedAvg	Based on Python	10	2-10/1	Accuracy, FAR
[234]	2021	Cross-device	Horizontal	Centralized	NSL-KDD	Based on K-means	FedAvg	Based on Python	10	1-100/1	Accuracy, WCSS
[13]	2022	Cross-device	Horizontal	Centralized	Generated	AE	FedAvg?	Pytorch	N/S	N/S-N/S	Accuracy, precision, recall, F1, FPR, FNR, memory use
[232]	2022	Cross-device	Horizontal	Centralized	NSL-KDD	DBN	FedAvg	TensorFlow	100	1-30/30-50	Accuracy, precision, recall, F1, time
[176]	2022	Cross-device	Horizontal	Centralized	N-BaIoT	MLP, AE	FedAvg, Coordinate-wise median/trimmed mean	Own library	8	1-29/1-4	Accuracy, F1, FPR, TPR
[106]	2022	Cross-device	Horizontal	Centralized	MNIST, CIFAR-10, CIC-IDS2017, TON_IoT	K-means clustering	FedAvg	Based on Pytorch	N/S	N/S-N/S	AUC, F1
[40]	2022	Cross-silo	Horizontal	Centralized	KDDCup99, NSL-KDD, CIC-IDS2017, CSE-CIC-IDS2018	AE	FedAvg	N/S	2	1-120/200	Precision, recall, F1
[229]	2022	Cross-silo	Horizontal	Centralized	NSL-KDD	Variational AE	FedAvg	TensorFlow, Keras	5-1000	100/1	Accuracy, Precision, recall, F1
Based on semi-supervised models											
[172]	2021	Cross-device	Horizontal	Centralized	NSL-KDD	AE	FedAvg	N/S	8	N/S-N/S	Accuracy
[198]	2020	Cross-device	Horizontal	Centralized	Microsoft Malware [180]	CNN, Auxiliary Classifier GAN	FedAvg	TensorFlow Federated	5	50-100/10	Accuracy, precision, recall, F1
[21]	2022	Cross-device	Horizontal	Centralized	UNSW-NB15	AE-NN	FedAvg	Pytorch	100	6/5	F1

(Continued)

Table 9. Continued

Ref.	Year	Network architecture	Data availability	Data partitioning	Dataset	ML model	Aggregation function	FL implementation	Training parties	Training rounds	Evaluation metrics
Based on supervised models											
[20]	2022	Cross-device	Horizontal	Centralized	[146]	AE-NN	FedAvg	Pytorch	100	18/20-100	Accuracy, precision, recall, F1, communication overhead
[164]	2022	Cross-device	Transfer	Centralized	Drebin, MalDroid, AndroZoo, VirusShare	SACN	FedAvg	Keras	100-500	N/S-N/S	FPR, TPR, ROC curve, F1-score
[252]	2022	Cross-device	Horizontal	Centralized	N-BaIoT	CNN	FedAvg	Pytorch	27, 89	1-200/100	Accuracy, precision, F1-score, communication overhead
Based on reinforcement models											
[226]	2021	Cross-device	Horizontal	Centralized	CIC-IDS2017	DQN	Based on FedAvg	Pytorch	1-10	1-4/1-1000	Error rate, reward
[155]	2021	Cross-device	Horizontal	Centralized	N/S	DDQN, SOM	FedAvg	TensorFlow Federated	6	1-200/1	Reward, loss, communication overhead
[159]	2021	Cross-device	Horizontal	Centralized	CIC-IDS2017	Q-learning	FedAvg	MATLAB/Simulink	50	1-10/1-10	Accuracy, DR, FPR, FNR

N/S stands for not specified.

I List of Acronyms

Acronym	Definition
AE	Autoencoder
ARP	Address Resolution Protocol
AUC	Area Under the Curve
BNN	Binarized Neural Network
C&C	Command-and-Control
CAN	Controller Area Network
CKS	Cohen Kappa Score
CM	Coordinate Median
CMFL	Communication-Mitigated Federated Learning
CNN	Convolutional Neural Network
CoAP	Constrained Application Protocol
CSV	Comma-separated values
DBN	Deep Belief Network
DDoS	Distributed Denial of Service
DDQN	Double Deep Q-Network
DL	Deep Learning
DLC	Data Length Code
DNN	Deep Neural Network
DNP	Distributed Network Protocol
DNS	Domain Name System
DoS	Denial of Service
DP	Differential Privacy
DQN	Deep Q-Network
DT	Decision Tree
EAP	Extensible Authentication Protocol
ENN	Edited Nearest Neighbors
FAR	False Acceptance Rate
FD	Federated Distillation
FL	Federated Learning
FNN	Feedforward Neural Network
FN	False Negative
FNR	False Negative Rate
FOR	False Omission Rate
FP	False Positive
FPR	False Positive Rate
GAN	Generative Adversarial Network
GBDT	Gradient Boosting Decision Tree
GBRM	Gaussian Binary Restricted Boltzmann Machine
GMM	Gaussian Mixture Model
GPS	Global Positioning System
GRU	Gated recurrent unit
HE	Homomorphic Encryption
IDS	Intrusion Detection System
IIoT	Industrial Internet of Things
IoT	Internet of Things
IRL	Inverse Reinforcement Learning
KNN	K-Nearest Neighbors
LAN	Local Area Network
LDAP	Lightweight Directory Access Protocol
LLM	Large Language Model
LR	Logistic Regression
LSTM	Long short-term memory
MAE	Mean Absolute Error

Acronym	Definition
MAPE	Mean Absolute Percentage Error
MCC	Matthews correlation coefficient
MEC	Mobile Edge Computing
MSSQL	Microsoft SQL Server
ML	Machine Learning
MLP	Multilayer perceptron
MQTT	Message Queuing Telemetry Transport
MSE	Mean Square Error
MTL	Multi-Task Learning
MTNN	Multi-Task Neural Network
NB	Naive Bayesian
NetBIOS	NETwork Basic Input Output System
NLP	Natural Language Processing
NN	Neural Networks
NTP	Network Time Protocol
OBD	On-Board Diagnostics
OS-ELM	Online Sequential Extreme Learning Machine
PFNM	Probabilistic Federated Neural Matching
RaNN	Random Neural Network
RL	Reinforcement Learning
RF	Random Forest
RMSE	Root Mean Squared Error
RNN	Recurrent Neural Network
ROC	Receiver operating characteristic
RPM	Revolutions per minute
RTSP	Real Time Streaming Protocol
SACN	Subgraph Aggregated Capsule Network
SAFA	Semi-Asynchronous Federated Averaging
SDN	Software-Defined Networking
SGD	Stochastic Gradient Descent
SMC	Secure Multi-party Computation
SMOTE	Synthetic Minority Over-sampling Technique
SMPC	Secure Multi-Party Computation
SNMP	Simple Network Management Protocol
SOC	Security Operation Centers
SOM	Self-organizing map
SSDP	Simple Service Discovery Protocol
SVM	Support-vector machine
TEE	Trusted Execution Environment
TFF	TensorFlow Federated
TFTP	Trivial File Transfer Protocol
TL	Transfer Learning
TN	True Negative
TNR	True Negative Rate
TP	True Positive
TPR	True Positive Rate
UAV	Unmanned Aerial Vehicle
UDP	User Datagram Protocol
VAE	Variational Autoencoder
VPN	Virtual Private Network
WCSS	Within Cluster Sum of Squares
WSN	Wireless Sensor Network
XSS	Cross-site scripting

Received 29 August 2023; revised 12 February 2025; accepted 22 March 2025